Monday 3:43 AM

Uma música de 1m a 120 BPM tem quantos compassos?

Assumindo **4/4**:

- **120 BPM** = 120 batidas por minuto
- 1 minuto = **120 batidas**
- Cada compasso 4/4 = **4 batidas**

Então:

$$\frac{120}{4} = \boxed{30 \text{ compassos}}$$

Resposta: 1 minuto a 120 BPM = 30 compassos de 4/4. 🎵

Se você estiver pensando em representar isso em **MIDI/piano roll**, posso também calcular quantos **ticks** ou passos de piano roll correspondem a esses 30 compassos.

Quantos tempo cada compasso?

A 120 BPM:

- **1 batida (tempo)** = 60 / 120 = **0,5 segundo**
- Um compasso **4/4** tem 4 tempos
- Portanto, **1 compasso = 4 × 0,5 = 2 segundos**

Então:

1 compasso = 2 segundos 🎵

Para 1 minuto:

- 30 compassos × 2 s = **60 segundos**.

Eu tentei fazer um algoritmo genético onde o genótipo era a música inteira e gerava notas aleatórias mas ficou muito ruim. Evolução lenta, resultado ruim. Então tô pensando numa ideia melhor, espera que vou te explicar...

Claro. Pode explicar a ideia nova — estou acompanhando.

E concordo com o diagnóstico: **usar a música inteira como um único genótipo e preencher notas aleatoriamente cria um espaço de busca gigantesco**. O algoritmo genético passa muito tempo produzindo indivíduos musicalmente ruins, e pequenas mutações podem destruir estruturas que já estavam funcionando.

Pode mandar sua ideia; aí analisamos **a representação do genótipo, operadores de mutação/crossover e principalmente como reduzir o espaço de busca**.

Vou criar um padrão de blocos de música fixos que no final sejam concatenadoa gerando uma música... Espera...

Sim, pode continuar. Estou esperando a ideia completa.

Vai ter 5 instrumentos:

Bateria
Strings
Baixo
Guitarra
Piano Solo

E vai ter um tempo fixo (Ex.: 2min)

E cada instrumento terá restrições claras para limitar o espaço de busca...

Aguarde que vou detalhar mais

Perfeito. A ideia está ficando bem mais estruturada.

Vou aguardar você detalhar as **restrições de cada instrumento** antes de sugerir a representação do genótipo ou os operadores do algoritmo genético.

Vou definir blocos fixos primeiro para cada instrumento...

Sim. Essa abordagem pode ser **bem melhor** que gerar a música inteira nota por nota.

Pode definir primeiro os blocos de cada instrumento. Depois podemos pensar em como o algoritmo genético vai **selecionar, ordenar e combinar esses blocos** para formar os 2 minutos.

Pode continuar.

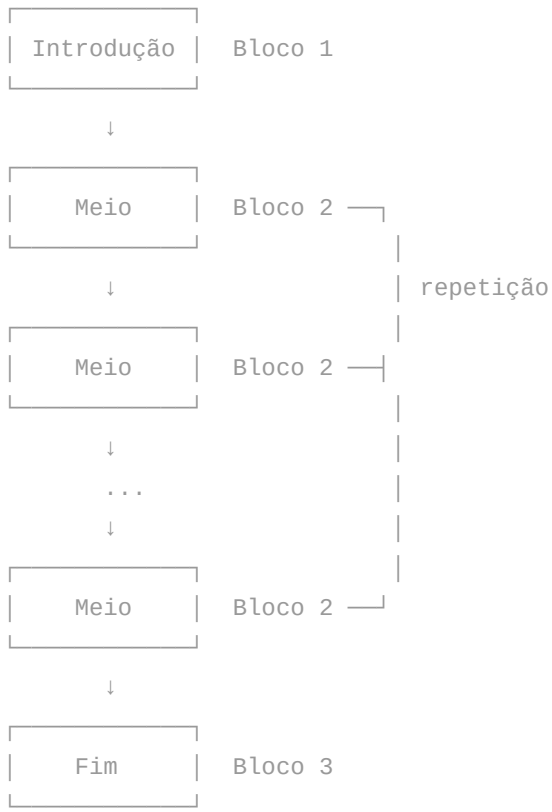
A bateria terá 3 blocos:

Bloco 1: Introdução
Bloco 2: Meio
Bloco 3: Fim

Porém o meio será algo que fica repetindo ao longo dos 2min desconsiderando o bloco de início e de fim.

Entendi. Então, para a **bateria**, você está propondo uma estrutura hierárquica:

Plain text



Isso é interessante porque o **genótipo da bateria não precisa conter 2 minutos de notas**. Ele pode simplesmente ser algo como:

Plain text

```
Bateria = [Bloco_1, Bloco_2, Bloco_3]
```

E o algoritmo sabe que:

Plain text

```
Música = Bloco_1 + Bloco_2 × N + Bloco_3
```

onde `N` é determinado pela duração total desejada.

Por exemplo, se cada bloco tiver **2 compassos**, a 120 BPM e 4/4:

- 1 compasso = 2 s
- 2 compassos = 4 s
- 120 s / 4 s = 30 blocos

Então poderia ser:

Plain text

```
Intro (4 s)
+
Meio × 28 (112 s)
+
Fim (4 s)
=
120 s
```

E o mais importante: **o padrão musical do bloco 2 já é válido por construção**. O GA não precisa descobrir sozinho como criar um ritmo coerente a partir de notas aleatórias.

Pode continuar detalhando os outros instrumentos. Vou esperar antes de propor a estrutura final do genótipo.

> Além disso, os itens de percussão da bateria devem ser bem restritos. 5 no máximo.

Sim. Isso torna o espaço de busca **ainda mais controlável**.

Podemos definir a bateria inicialmente como, por exemplo:

Percussão	Permitido
Kick	✓
Snare	✓

Percussão	Permitido
Closed Hi-Hat	✓
Open Hi-Hat	✓
Crash	✓
Outros	×

Mas os **5 itens não precisam ser exatamente esses**. Podemos escolher os 5 instrumentos de percussão mais adequados ao estilo que você quer gerar.

E há uma vantagem importante: o GA poderia **não escolher quais instrumentos existem**. Essa decisão seria fixa. Ele apenas evoluiria os padrões dentro das restrições.

Por exemplo:

```
Plain text

BATERIA
├── Instrumentos: {Kick, Snare, HiHat, Crash, Tom}
├── Bloco Intro
│   └── padrão restrito
├── Bloco Meio
│   └── padrão restrito
└── Bloco Fim
    └── padrão restrito
```

Assim você vai progressivamente transformando o problema de:

> **"Crie qualquer música possível."**

em:

> **"Encontre a melhor combinação dentro de um conjunto musicalmente plausível."**

Para algoritmo genético, isso é uma diferença enorme.

Pode continuar com as regras dos blocos/instrumentos.

Para o String vou assumir que minha música sempre vai ter 4 acordes fixos de 3 notas. E que eles ficam tocando sequencialmente

1 2 3 4 1 2 3 4 ...

O algoritmo genético poderá apenas variar as notas entre esses 4 acordes, limitando o espaço de busca neles.

Sim — e aqui sua ideia fica **muito mais interessante para um algoritmo genético**.

Para **Strings**, você está fixando a estrutura harmônica e deixando o GA procurar apenas dentro de um espaço controlado.

Strings

Você teria 4 acordes, cada um com 3 notas:

```
Plain text

Acorde 1 → Acorde 2 → Acorde 3 → Acorde 4
    ↓          ↓          ↓          ↓
   3 notas    3 notas    3 notas    3 notas

Repetição:

1 → 2 → 3 → 4 → 1 → 2 → 3 → 4 → ...
```

Por exemplo, **apenas como ilustração**:

```
Plain text

Acorde 1 = [C4, E4, G4]
Acorde 2 = [A3, C4, E4]
Acorde 3 = [F3, A3, C4]
Acorde 4 = [G3, B3, D4]
```

O ponto importante é que o GA **não precisa decidir quando tocar os acordes**. Isso já está definido.

Ele só precisa encontrar **quais 4 acordes** serão utilizados.

Isso reduz absurdamente o espaço de busca

Em vez de:

```
Plain text
```

```
2 minutos
↓
milhares de posições
↓
cada posição pode ter qualquer nota
```

você passa para algo como:

```
GENÓTIPO
│
├── Acorde 1 = [nota, nota, nota]
├── Acorde 2 = [nota, nota, nota]
├── Acorde 3 = [nota, nota, nota]
└── Acorde 4 = [nota, nota, nota]
```

E o fenótipo gera:

```
1 2 3 4 1 2 3 4 1 2 3 4 ...
```

durante os 2 minutos.

E dá para restringir ainda mais

Por exemplo, cada nota de cada acorde poderia ser escolhida de um conjunto pequeno:

```
Acorde 1:
nota 1 ∈ {C3, D3, E3}
nota 2 ∈ {E3, F3, G3}
nota 3 ∈ {G3, A3, B3}
```

Ou, melhor ainda, você pode impor regras musicais:

- notas dentro de determinada escala;
- limite de oitavas;
- distância máxima entre as três notas;
- evitar intervalos específicos;
- acordes sempre com 3 notas;

- determinada nota como baixo;
- etc.

Assim, o GA passa a procurar **uma combinação musical**, em vez de procurar uma música completamente do zero.

E isso combina muito bem com a ideia dos seus **blocos de bateria + acordes fixos + outros instrumentos com restrições**.

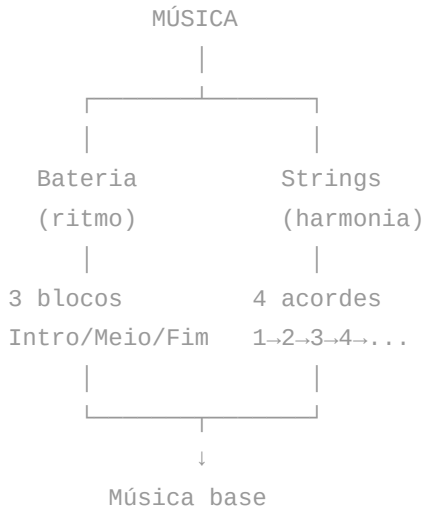
Pode continuar — quero ver como você está pensando **baixo, guitarra e piano solo**, porque aí provavelmente conseguimos montar uma representação de genótipo bastante elegante.

> Aí já posso implementar um GA básico para gerar a bateria e strings... Que serão os elementos base da música, a bateria define o ritmo e o String define os acordes da melodia. Aguarde...

Exatamente. 👍

Você já teria uma **primeira versão do problema bem delimitada**:

Plain text



E isso permite testar o GA **antes de adicionar complexidade**.

Você pode inclusive avaliar se a representação genética funciona usando apenas esses dois elementos. Depois, **baixo, guitarra e piano solo** podem ser adicionados progressivamente.

Aguardo a continuação.

Pensando em algo simples pra início, assumindo sequências de 4s, pode ser um compasso de 4s ou 4 de 1s.

O genótipo que será representado em piano roll terá os seguintes itens:

Bat1
SChord1 SChord2 Schord3 Schord4

O Bat1 é uma sequência de itens de bateria de 4s que o GA poderá variar.
SChord1 a Schord4 são os acordes de 3 notas do String que o GA pode variar também porém eles sempre tem que ser acordes musicais válidos, não apenas mistura de 3 notas aleatórias.

Entendeu?

Sim, entendi perfeitamente. E acho que essa representação é **bem mais adequada para começar**.

O genótipo inicial seria um conjunto de **5 componentes independentes**, todos representados em piano roll:

```
Plain text

GENÓTIPO
│
├── Bat1
│    └── padrão de bateria com duração = 4 s
│
├── SChord1
│    └── acorde válido de 3 notas
│
├── SChord2
│    └── acorde válido de 3 notas
│
├── SChord3
│    └── acorde válido de 3 notas
│
└── SChord4
     └── acorde válido de 3 notas
```

Depois, o fenótipo poderia ser expandido para:

```
Plain text
4 segundos:

Bateria:
[ Bat1 ]

Strings:
[ Chord1 ]

Próximos 4 segundos:
Bateria:
[ Bat1 ]

Strings:
[ Chord2 ]

Próximos 4 segundos:
Bateria:
[ Bat1 ]

Strings:
[ Chord3 ]
```

E isso se repete até completar os **2 minutos**.

Uma diferença importante

Para os Strings, eu **não faria o gene armazenar diretamente três notas independentes**:

```
Plain text
SChord1 = [C4, F#4, A#4] ❌
```

O gene deveria representar **um acorde pertencente a um conjunto de acordes válidos**:

```
Plain text
SChord1 = Am
SChord2 = F
SChord3 = C
SChord4 = G
```

E então o piano roll é gerado a partir desses acordes:

```
Am = [A, C, E]
F  = [F, A, C]
C  = [C, E, G]
G  = [G, B, D]
```

Isso é muito interessante para o GA porque uma mutação poderia ser simplesmente:

```
SChord2: F → Dm
```

em vez de alterar três notas aleatoriamente.

Assim, **toda mutação continua produzindo um acorde musicalmente válido**.

E o mesmo princípio pode ser aplicado à bateria: em vez de permitir qualquer combinação absurda de notas, `Bat1` pode ser escolhido/mutado dentro de um conjunto de padrões rítmicos válidos.

Acho que você está chegando a uma representação genética **muito mais compacta e evolutivamente tratável**.

O genótipo portanto pode ter esses 5 itens, ok?

Sim, exatamente. **O genótipo pode ter apenas esses 5 itens**:

```
Genótipo = [Bat1, SChord1, SChord2, SChord3, SChord4]
```

Onde:

- **Bat1** → padrão de bateria de 4 segundos.
- **SChord1** → acorde válido de 3 notas.
- **SChord2** → acorde válido de 3 notas.
- **SChord3** → acorde válido de 3 notas.
- **SChord4** → acorde válido de 3 notas.

O GA evolui esses **5 genes**, e o algoritmo de decodificação transforma o genótipo na música de 2 minutos.

Isso é uma representação **bem menor e mais estruturada** do que ter a música inteira como genótipo.

> Vamos assumir que cada item tem 4s de duração e que a música terá 32s, com exatamente os 4 Schords concatenadoa enquanto o Bat1 fica repetindo em paralelo a cada um deles.
>
> Como ficaria o genótipo?

Sim. Nesse caso, o **genótipo continua tendo somente 5 genes**:

```
Plain text
Genótipo = [Bat1, SChord1, SChord2, SChord3, SChord4]
```

Como cada item dura **4 segundos** e a música tem **32 segundos**, teremos 8 segmentos de 4 s.

A decodificação seria:

Tempo	Bateria	Strings
0–4 s	Bat1	SChord1
4–8 s	Bat1	SChord2
8–12 s	Bat1	SChord3
12–16 s	Bat1	SChord4
16–20 s	Bat1	SChord1
20–24 s	Bat1	SChord2
24–28 s	Bat1	SChord3
28–32 s	Bat1	SChord4

Portanto:

Plain text

```
             0    4    8   12   16   20   24   28   32s
             |----|----|----|----|----|----|----|----|
Bateria      Bat1 Bat1 Bat1 Bat1 Bat1 Bat1 Bat1 Bat1
Strings      C1   C2   C3   C4   C1   C2   C3   C4
```

E o ponto fundamental é que **o genótipo não precisa armazenar essas 8 repetições**. Ele armazena apenas:

Plain text

```
┌──────────────────────────────────────────┐
│              GENÓTIPO                    │
├──────┬──────────┬──────────┬──────────┬──────┤
│ Bat1 │ SChord1 │ SChord2 │ SChord3 │ SChord4 │
└──────┴──────────┴──────────┴──────────┴──────┘
```

O **fenótipo/música resultante** é que expande esses 5 genes para os 32 segundos.

Isso significa que uma mutação pode ser extremamente simples, por exemplo:

Plain text

```
Antes:
[Bat1_A, C, Am, F, G]

Mutação:
[Bat1_A, C, Dm, F, G]
                ↑
             mudou 1 gene
```

E automaticamente os 32 s passam a usar `Dm` nos segmentos correspondentes.

Essa é uma boa primeira versão do GA, porque o espaço de busca fica muito menor e cada indivíduo já possui uma estrutura musical válida por construção.

Boa.

A fitness pode continuar sendo o que já tô usando que é medir a qualidade baseada em teoria musical.

Sim. **Eu manteria a fitness que você já está usando**, pelo menos nessa primeira versão.

Na verdade, essa nova representação deve permitir que a mesma fitness funcione **melhor**, porque agora o GA não desperdiça tanta capacidade avaliando músicas completamente aleatórias.

O fluxo ficaria:

```
Genótipo
[Bat1, SChord1, SChord2, SChord3, SChord4]
              ↓
          Decodificação
              ↓
        Música de 32 s
              ↓
     ┌────────┴────────┐
     │                 │
  Bateria          Strings
  Bat1×8            1→2→3→4×2
     │                 │
     └────────┬────────┘
              ↓
       Fitness musical
              ↓
        Algoritmo Genético
```

A grande vantagem é que agora a **fitness consegue concentrar-se em avaliar qualidade**, enquanto as restrições do genótipo já garantem uma estrutura mínima.

Por exemplo, se sua fitness considera coisas como:

- consonância;
- progressão de acordes;
- intervalos;
- repetição;
- variedade;
- ritmo;
- coerência harmônica;
- regras de teoria musical;

o GA pode otimizar esses aspectos sem precisar simultaneamente descobrir **como construir uma música do zero**.

E eu faria uma distinção importante:

As restrições definem o espaço de busca; a fitness define o que é melhor dentro desse espaço.

Isso é exatamente o que você está fazendo agora.

Depois podemos comparar objetivamente o **GA antigo vs. esse GA baseado em blocos**, inclusive calculando o tamanho aproximado do espaço de busca de cada um.

> Só que ao invés de chamar de SChord, vamos chamar se MChord, pois definira acordes da música e não do string apenas.

Sim, faz mais sentido. **`MChord` = Music Chord**, representando os acordes harmônicos da música, independentemente de qual instrumento irá executá-los.

Então o genótipo passa a ser:

```
Plain text
Genótipo = [Bat1, MChord1, MChord2, MChord3, MChord4]
```

Com:

- `Bat1` → padrão de bateria de 4 s.
- `MChord1` → acorde musical válido de 3 notas.
- `MChord2` → acorde musical válido de 3 notas.
- `MChord3` → acorde musical válido de 3 notas.
- `MChord4` → acorde musical válido de 3 notas.

E a sequência de 32 s:

```
Plain text
Tempo:    0    4    8   12   16   20   24   28   32
          |----|----|----|----|----|----|----|----|

Bat1:     B    B    B    B    B    B    B    B

MChord:   1    2    3    4    1    2    3    4
```

Assim, posteriormente, **Baixo, Guitarra e Piano Solo podem usar os `MChord` s como referência harmônica**, em vez de cada instrumento ter sua própria definição de acordes.

A nomenclatura fica bem mais coerente para o projeto.

Da pra representar isso no piano roll fácil?

Sim, **muito facilmente**. Na verdade, essa representação combina muito bem com um piano roll.

Você pode pensar no piano roll como a **representação do fenótipo**, enquanto o genótipo continua sendo apenas:

Plain text

```
[Bat1, MChord1, MChord2, MChord3, MChord4]
```

Exemplo

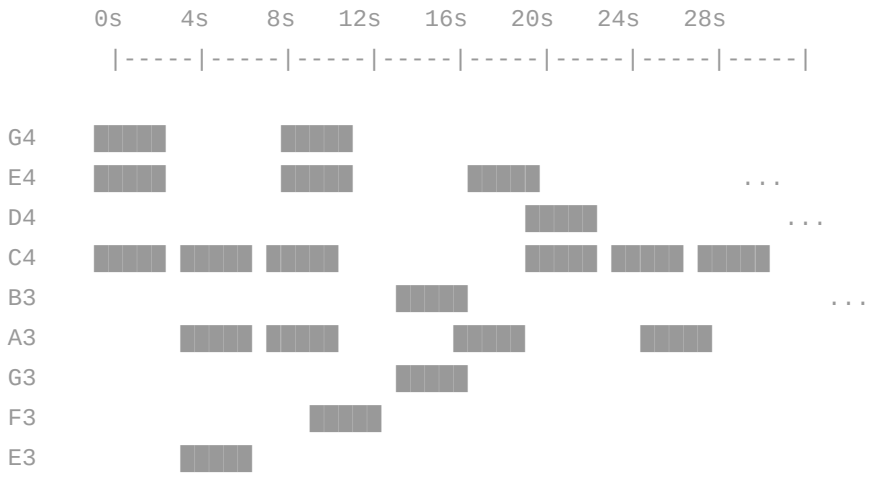
Suponha:

Plain text

```
MChord1 = C4 E4 G4
MChord2 = A3 C4 E4
MChord3 = F3 A3 C4
MChord4 = G3 B3 D4
```

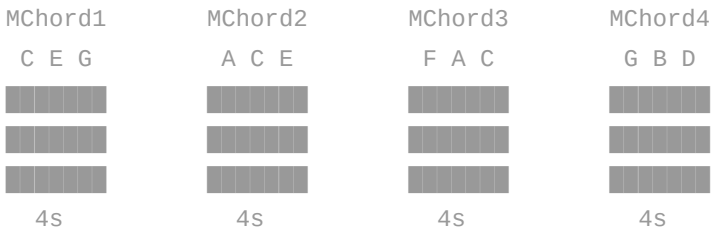
No piano roll, os acordes aparecem como três notas simultâneas:

Plain text



Conceitualmente:

Plain text



E a bateria pode ocupar **outra região/canal do piano roll**:

```
Plain text

Piano Roll
_____________________________________________
Strings / Harmonia
    MChord1 → MChord2 → MChord3 → MChord4


_____________________________________________
Bateria
    Bat1 → Bat1 → Bat1 → Bat1 → ...
_____________________________________________
```

E isso é ótimo para o GA

Porque você consegue ter uma representação computacional muito simples:

```python
genotype = [
    Bat1,
    MChord1,
    MChord2,
    MChord3,
    MChord4
]
```

E uma função:

```python
genotype_to_pianoroll(genotype)
```

que simplesmente expande os genes para os **32 segundos**.

Depois, sua fitness recebe o piano roll:

Plain text

```
Genótipo
   ↓
Piano Roll de 32s
   ↓
Fitness baseada em teoria musical
   ↓
Score
```

Isso também deixa **muito fácil visualizar os indivíduos do GA**. Você pode salvar o piano roll de cada geração e literalmente enxergar se a evolução está produzindo padrões mais interessantes.

De um exemplo de genótipo iniciado aleatoriamente

Claro. Considerando que cada gene é escolhido aleatoriamente, mas que `MChord` **obrigatoriamente precisa ser um acorde válido**, um indivíduo poderia começar assim:

Plain text

```
Genótipo = [
    Bat1,
    MChord1,
    MChord2,
    MChord3,
    MChord4
]
```

Um exemplo concreto:

Plain text

```
Bat1    = [Kick, HiHat, Snare, HiHat, Kick, HiHat, Snare, HiHat]

MChord1 = [C4, E4, G4]    → C Major
MChord2 = [A3, C4, E4]    → A Minor
MChord3 = [F3, A3, C4]    → F Major
MChord4 = [G3, B3, D4]    → G Major
```

Portanto:

Plain text

```
[
    [K, HH, S, HH, K, HH, S, HH],
    [C4, E4, G4],
    [A3, C4, E4],
    [F3, A3, C4],
    [G3, B3, D4]
]
```

Expandido para os 32 segundos

Plain text

```
Tempo      0-4s    4-8s    8-12s   12-16s   16-20s   20-24s   24-28s   28-32s
           |       |       |       |        |        |        |        |
Bateria   Bat1     Bat1     Bat1     Bat1     Bat1     Bat1     Bat1     Bat1
           |       |       |       |        |        |        |        |
Acorde    C Major  A Minor  F Major  G Major  C Major  A Minor  F Major  G Major
```

A partir daí, uma **mutação** poderia alterar apenas um gene:

Plain text

```
Antes:
MChord3 = F Major = [F3, A3, C4]

Depois:
MChord3 = D Minor = [D3, F3, A3]
```

E o restante do indivíduo permanece igual.

Esse é justamente o tipo de representação que eu acho interessante para o seu experimento: **o GA começa com indivíduos já musicalmente estruturados**, em vez de começar com uma enorme quantidade de notas aleatórias.

O que é um gene nesse caso?

Nesse caso, **um gene é um componente do genótipo que o algoritmo genético pode modificar**.

Seu genótipo tem **5 genes**:

Plain text

```
Genótipo
│
├── Gene 1 → Bat1
├── Gene 2 → MChord1
├── Gene 3 → MChord2
├── Gene 4 → MChord3
└── Gene 5 → MChord4
```

Por exemplo:

```
Plain text

Bat1    = padrão de bateria
MChord1 = C Major
MChord2 = A Minor
MChord3 = F Major
MChord4 = G Major
```

Então, nesse indivíduo:

```
Plain text

Gene 1 = Bat1
Gene 2 = C Major
Gene 3 = A Minor
Gene 4 = F Major
Gene 5 = G Major
```

Uma observação importante

O **gene não precisa ser uma única nota ou um único valor**.

No seu caso, `Bat1` é um gene que contém um **padrão inteiro de 4 segundos**, e `MChord1` é um gene que contém um **acorde inteiro de 3 notas**.

Isso é perfeitamente válido em um algoritmo genético.

Por exemplo, uma mutação poderia ser:

```
Plain text

Antes:
[Bat1, C, Am, F, G]
```

```
Depois:
[Bat2, C, Am, F, G]
       ↑
     mutação
```

ou:

```
Plain text
```

```
Antes:
[Bat1, C, Am, F, G]

Depois:
[Bat1, C, Dm, F, G]
           ↑
         mutação
```

Ou seja, **gene = unidade básica que você permite que o GA altere**.

No seu projeto, essa definição é especialmente útil porque você está escolhendo deliberadamente **o nível de abstração dos genes**.

E um exemplo de cruzamento...

Claro. Como seu genótipo tem **5 genes**, podemos fazer um crossover simples de **um ponto**.

Imagine dois pais:

```
Plain text
```

```
PAI 1
[ Bat1_A | C   | Am  | F   | G   ]
             1     2     3     4

PAI 2
[ Bat1_B | Dm  | Bb  | C   | Am  ]
```

Escolhemos aleatoriamente um ponto de corte, por exemplo, depois do terceiro gene:

```
Plain text
```

```
PAI 1
[ Bat1_A | C | Am | F | G ]
                 ↑
```

```
                corte

PAI 2
[ Bat1_B | Dm | Bb | C | Am ]
                  ↑
                corte
```

O filho recebe a primeira parte do PAI 1 e a segunda parte do PAI 2:

```
</> Plain text

FILHO
[ Bat1_A | C | Am | C | Am ]
```

Ou seja:

```
</> Plain text

           PAI 1       PAI 2
             ↓           ↓
Bat1       Bat1_A
MChord1       C
MChord2       Am
MChord3                   C
MChord4                   Am
             \           /
              \         /
               ↓       ↓
              FILHO
       [Bat1_A,C,Am,C,Am]
```

Uma vantagem importante

Como cada `MChord` já é um acorde válido, o crossover **não precisa criar notas inválidas**.

Por exemplo, não acontece:

```
</> Plain text

[C4, F#4, G#4] ❌
```

por misturar pedaços de dois acordes.

Você sempre herda o **gene inteiro**:

Plain text

```
MChord3 = C Major
```

ou:

Plain text

```
MChord3 = Bb Major
```

etc.

Isso deixa o crossover muito simples e mantém a restrição musical.

No futuro, você também poderia fazer crossover específico dentro do `Bat1`, caso queira evoluir partes do padrão de bateria, mas **para a primeira implementação eu manteria o gene inteiro**, exatamente como você está propondo.

> Está garantido que o Bat1 vai sempre existir? Não vai misturar bateria com acorde né?

Sim. **Com a representação que definimos, isso fica garantido**, desde que o crossover seja feito **por gene inteiro**.

O genótipo tem uma estrutura fixa:

Plain text

```
[ Bat1, MChord1, MChord2, MChord3, MChord4 ]
```

O `Bat1` ocupa sempre a **posição 0**. Os acordes ocupam sempre as posições 1–4.

Por exemplo:

Plain text

```
PAI 1 = [Bat1_A, C,  Am, F, G]
PAI 2 = [Bat1_B, Dm, Bb, C, Am]
```

Um crossover:

```
Plain text
```

```
       Bat1    MChord1  MChord2  MChord3  MChord4
        ↓         ↓        ↓        ↓        ↓
PAI 1 [ Bat1_A |   C   |   Am   |   F    |   G   ]
PAI 2 [ Bat1_B |  Dm   |   Bb   |   C    |   Am  ]
```

Pode gerar:

```
Plain text
```

```
FILHO [ Bat1_A |   C   |   Am   |   C    |   Am  ]
```

ou:

```
Plain text
```

```
FILHO [ Bat1_B |  Dm   |   Bb   |   F    |   G   ]
```

Nunca:

```
Plain text
```

```
[ C, Am, F, G, Bat1_A ] ❌
```

nem:

```
Plain text
```

```
[ Bat1_A, C, Am, F, alguma-coisa-de-outro-gene ] ❌
```

porque o significado de cada posição é fixo.

Na prática

Eu definiria explicitamente a estrutura:

```
Python
```

```
genotype[0]  # Bat1
genotype[1]  # MChord1
genotype[2]  # MChord2
```

```
genotype[3]  # MChord3
genotype[4]  # MChord4
```

E o crossover simplesmente preserva essas posições.

Portanto, **sim: todo indivíduo sempre terá exatamente 1 Bat1 + 4 MChords**.

Isso é uma das vantagens dessa representação: o próprio formato do genótipo funciona como uma **restrição estrutural do problema**.

Gere um código python no chat implementando esse genótipo

Sim. Abaixo está uma implementação inicial **somente da representação genética**, já com:

- 5 genes: `Bat1 + MChord1..4`
- `Bat1` com 4 s
- cada `MChord` com 3 notas
- lista de acordes válidos
- população aleatória
- crossover de um ponto
- mutação
- garantia de que `Bat1` nunca será misturado com `MChord`
- expansão do genótipo para uma representação de **32 s**

```
</> Python

import random
from dataclasses import dataclass
from typing import List, Tuple


# ============================================================
# CONFIGURAÇÕES
# ============================================================

BLOCK_DURATION = 4          # duração de cada gene em segundos
MUSIC_DURATION = 32         # duração total da música

NUM_MCHORDS = 4


# ============================================================
# TIPOS DE DADOS
# ============================================================
```

```
# Uma nota MIDI
```

A estrutura central

O que eu considero mais importante nesse código é que o indivíduo é explicitamente:

```python
Individual(
    bat1=BatPattern(...),
    mchords=[
        MChord1,
        MChord2,
        MChord3,
        MChord4
    ]
)
```

Portanto, **não existe possibilidade conceitual de o crossover transformar `Bat1` em um acorde**, porque eles são campos diferentes do indivíduo.

E o fluxo do seu experimento agora fica:

```
              INDIVÍDUO
                  |
          ┌───────┴────────┐
          |                |
       Bat1            MChord × 4
          |                |
     4 segundos        4 segundos
          |                |
          └───────┬────────┘
                  ↓
              DECODER
                  ↓
          ┌────────────────┐
          |   32 segundos  |
          |                |
          | Bat1  MChord1  |
          | Bat1  MChord2  |
          | Bat1  MChord3  |
          | Bat1  MChord4  |
          | Bat1  MChord1  |
```

Observação: neste primeiro código eu deixei os acordes como uma lista de acordes válidos e a bateria como uma lista de padrões válidos. Isso é proposital: depois podemos substituir essas listas por uma representação de **piano roll**, que será provavelmente a próxima etapa mais interessante do seu GA.

Já faz com a representação em piano roll

Sim. Eu faria o código já com o **genótipo armazenando diretamente matrizes de piano roll**, mas mantendo a estrutura de 5 genes:

```
Plain text
[Bat1, MChord1, MChord2, MChord3, MChord4]
```

Abaixo está uma versão inicial completa. Cada `Bat1` e cada `MChord` ocupa **4 segundos**, e o fenótipo final tem **32 segundos**.

GA com genótipo em piano roll

```
import random
from dataclasses import dataclass
from typing import List
```

A estrutura principal agora é exatamente a que você descreveu:

```
Plain text
GENÓTIPO
┌──────┬──────────┬──────────┬──────────┬──────────┐
| Bat1 | MChord1 | MChord2 | MChord3 | MChord4 |
└──────┴──────────┴──────────┴──────────┴──────────┘
  ↓        ↓         ↓         ↓         ↓
 4s       4s        4s        4s        4s
```

E cada gene já é um **piano roll**, não apenas um nome de padrão.

Um detalhe que eu deixei propositalmente simples: **cada MChord ocupa as 16 colunas inteiras**, ou seja, as três notas ficam sustentadas durante os 4 segundos. Depois podemos modificar isso para permitir que o GA altere **o ritmo/duração das notas dentro do acorde**, sem perder a restrição de que o acorde seja musicalmente válido.

Depois eu quero gerar as notas do baixo totalmente restritas aos MChord. Serão sempre 4 notas de baixo pra cada MChord com notas dentro do acorde.

Sim. Essa é uma ótima restrição para o **baixo**, porque reduz bastante o espaço de busca do algoritmo genético e garante coerência harmônica.

A ideia pode ficar assim:

Estrutura do genótipo

Continuamos com:

```
Plain text
[Bat1, MChord1, MChord2, MChord3, MChord4]
```

O baixo **não precisa ser um gene independente**. Ele pode ser **derivado dos MChord**.

Para cada `MChord`, teremos exatamente **4 notas de baixo**:

```
Plain text
MChord1 → Bass1
MChord2 → Bass2
MChord3 → Bass3
MChord4 → Bass4
```

E cada Bass terá 4 notas, todas pertencentes ao respectivo acorde.

Por exemplo:

```
Plain text
MChord1 = C major = [C, E, G]

Bass1 = [C, G, E, C]
```

Outro indivíduo poderia ter:

```
Plain text
```

```
Bass1 = [E, E, G, C]
```

Ambos são válidos porque todas as notas pertencem ao acorde.

Na música completa

Como os 4 MChords são repetidos duas vezes:

Plain text

```
Tempo     Acorde       Baixo
0–4s      MChord1      Bass1
4–8s      MChord2      Bass2
8–12s     MChord3      Bass3
12–16s    MChord4      Bass4

16–20s    MChord1      Bass1
20–24s    MChord2      Bass2
24–28s    MChord3      Bass3
28–32s    MChord4      Bass4
```

Portanto, temos **32 notas de baixo** na música de 32 segundos.

Isso é particularmente interessante para o GA

Você pode fazer o baixo ser uma função dos acordes:

Plain text

```
MChord → gera conjunto permitido de notas → Bass
```

Por exemplo:

Plain text

```
C major
    ↓
{C, E, G}
    ↓
Bass = [C, G, E, G]
```

O GA nunca poderá produzir algo como:

```
Plain text
Bass = [C, F#, E, G]
```

porque `F#` não pertence ao MChord.

Isso significa que **não precisamos penalizar o GA por produzir notas harmonicamente inválidas**. Elas simplesmente não existem no espaço de busca.

Eu faria ainda uma pequena distinção: o acorde pode usar notas em uma oitava, enquanto o baixo usaria as mesmas classes de notas em uma oitava mais grave.

Por exemplo:

```
Plain text
MChord:
C4 - E4 - G4

Bass:
C2 - G2 - E2 - C2
```

Assim você mantém:

MChord = harmonia
Bass = linha melódica restrita à harmonia

E depois podemos fazer o mesmo para **guitarra, piano solo e strings**, mas com restrições diferentes. Isso deixa o genótipo pequeno e faz o **fenótipo musical ser muito mais rico**.

> A mesma lógica será feita para a guitarra porém gerando 8 notas para cada MChord.

Sim. E isso mantém uma estrutura muito consistente para o algoritmo genético.

Para a **guitarra**, podemos usar exatamente a mesma lógica do baixo, mas com **8 notas por MChord**.

Estrutura

Para cada acorde:

```
Plain text
```

```
MChord1 → Bass1 → 4 notas
        → Guitar1 → 8 notas

MChord2 → Bass2 → 4 notas
        → Guitar2 → 8 notas

MChord3 → Bass3 → 4 notas
        → Guitar3 → 8 notas

MChord4 → Bass4 → 4 notas
        → Guitar4 → 8 notas
```

Todas as notas da guitarra devem pertencer ao respectivo `MChord`.

Por exemplo:

Plain text

```
MChord1 = C major
         = {C, E, G}

Guitar1 = [C, E, G, E, C, G, E, G]
```

Outro indivíduo poderia ter:

Plain text

```
Guitar1 = [G, G, E, C, E, G, C, E]
```

Ambos são válidos.

Na música de 32 segundos

Teríamos:

Plain text

```
0–4s     MChord1 + Bass1 + Guitar1
4–8s     MChord2 + Bass2 + Guitar2
8–12s    MChord3 + Bass3 + Guitar3
12–16s   MChord4 + Bass4 + Guitar4

16–20s   MChord1 + Bass1 + Guitar1
20–24s   MChord2 + Bass2 + Guitar2
```

```
24–28s   MChord3 + Bass3 + Guitar3
28–32s   MChord4 + Bass4 + Guitar4
```

Portanto:

- **Baixo:** 4 notas × 4 MChords = **16 notas únicas**
- **Guitarra:** 8 notas × 4 MChords = **32 notas únicas**
- Como os MChords são repetidos duas vezes, a execução terá **32 notas de baixo + 64 notas de guitarra**.

E eu manteria isso fora do genótipo

Inicialmente, eu faria:

```
Plain text

Genótipo:
[Bat1, MChord1, MChord2, MChord3, MChord4]
```

E durante o **decode/phenotype**, geraríamos:

```
Plain text

MChord
   ├── Bass → 4 notas
   └── Guitar → 8 notas
```

Assim o GA continua trabalhando com apenas **5 genes**, mas cada indivíduo gera uma música com bateria + acordes + baixo + guitarra.

Isso é interessante porque você está efetivamente **embutindo conhecimento musical no espaço de busca**: o GA não precisa aprender que baixo e guitarra devem respeitar o acorde — isso já é garantido pela representação.

> Aí como cada indivíduo tem 32 seg eu posso pegar os 5 melhores indivíduos e concatena-los gerando uma música completa no final.

Sim. **Essa estratégia faz bastante sentido** para o seu problema, principalmente porque cada indivíduo representa um bloco musical de apenas **32 segundos**.

Você pode tratar o GA como um gerador de **segmentos musicais**, e depois montar a música final a partir dos melhores indivíduos.

Exemplo

Suponha que, ao final da evolução, você tenha:

```
Plain text
Indivíduo 1 → fitness = 0.94
Indivíduo 2 → fitness = 0.92
Indivíduo 3 → fitness = 0.90
Indivíduo 4 → fitness = 0.88
Indivíduo 5 → fitness = 0.87
```

Cada um possui:

```
Plain text
32 segundos
```

Então:

```
Plain text
Indivíduo 1 → 0s  – 32s
Indivíduo 2 → 32s – 64s
Indivíduo 3 → 64s – 96s
Indivíduo 4 → 96s – 128s
Indivíduo 5 → 128s – 160s
```

Resultado:

5 × 32 = 160 segundos = 2min40s

Porém, eu faria uma pequena alteração

Não necessariamente usaria simplesmente:

```
Plain text
Top 1
Top 2
Top 3
```

```
Top 4
Top 5
```

Porque existe a possibilidade de os cinco indivíduos serem **muito parecidos**.

Por exemplo:

Plain text

```
I1 = [Bat_A, C, Am, F, G]
I2 = [Bat_A, C, Am, F, G]
I3 = [Bat_B, C, Am, F, G]
I4 = [Bat_A, C, Am, F, G]
I5 = [Bat_A, C, Am, F, G]
```

Você teria uma música final bastante repetitiva.

Uma alternativa interessante é selecionar os **melhores indivíduos diferentes entre si**:

Plain text

```
          Fitness
I1         0.94       ← selecionado
I2         0.93
I3         0.92       ← selecionado
I4         0.91
I5         0.90       ← selecionado
I6         0.89
...
```

Usando uma combinação de:

fitness + diversidade genética

E existe uma vantagem ainda maior

Como o seu indivíduo é:

Plain text

```
[Bat1, MChord1, MChord2, MChord3, MChord4]
```

podemos fazer a concatenação respeitando as estruturas.

Por exemplo:

Plain text

```
Segmento 1:
[Bat_A, C, Am, F, G]

Segmento 2:
[Bat_B, Dm, G, C, Am]

Segmento 3:
[Bat_C, Em, C, G, D]

Segmento 4:
[Bat_A, Am, F, C, G]

Segmento 5:
[Bat_D, C, G, Am, F]
```

A música final seria:

Plain text

```
┌──────────────── 32s ────────────────┐
│          Indivíduo 1                │
└─────────────────────────────────────┘
┌──────────────── 32s ────────────────┐
│          Indivíduo 2                │
└─────────────────────────────────────┘
┌──────────────── 32s ────────────────┐
│          Indivíduo 3                │
└─────────────────────────────────────┘
┌──────────────── 32s ────────────────┐
│          Indivíduo 4                │
└─────────────────────────────────────┘
┌──────────────── 32s ────────────────┐
│          Indivíduo 5                │
└─────────────────────────────────────┘
              ↓
       MÚSICA FINAL
          160 segundos
```

Isso também combina muito bem com a sua ideia de **restringir fortemente o espaço de busca**.

O GA não está tentando descobrir uma música inteira de 2–3 minutos de uma vez. Ele aprende a produzir **bons segmentos musicais de 32s**, e depois você usa esses segmentos como "blocos" para construir uma composição maior.

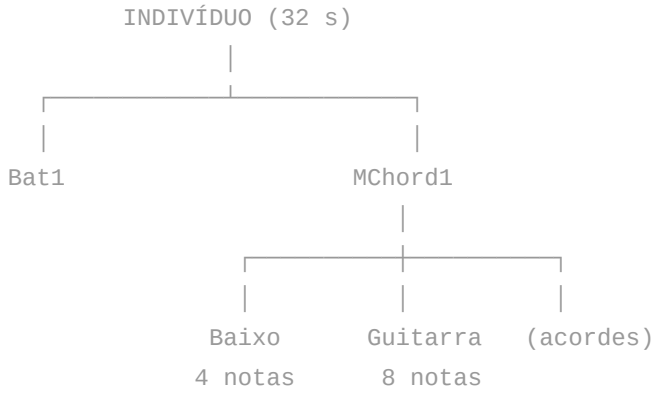
Eu só acrescentaria futuramente uma **fitness de transição entre segmentos**, porque o final do indivíduo 1 precisa combinar com o início do indivíduo 2. Isso pode ser feito sem aumentar muito a complexidade do GA.

> Acho que com isso já dá pra gerar uma música de 160seg bem legal né?

Sim — **é uma abordagem bastante plausível para gerar uma música de 160 s (2min40s) que seja musicalmente coerente**, especialmente como primeiro sistema de GA.

O ponto forte é que você não está deixando o algoritmo procurar no espaço gigantesco de "qualquer sequência de notas". Você está impondo uma estrutura musical:

Plain text



E:

Plain text

```
MChord1 → MChord2 → MChord3 → MChord4
   ↓          ↓          ↓          ↓
  baixo      baixo      baixo      baixo
  guitarra   guitarra   guitarra   guitarra
```

Depois:

Plain text

```
5 melhores indivíduos
        ↓
32s + 32s + 32s + 32s + 32s
        ↓
       160 s
```

O mais importante

Você está **separando geração e avaliação**.

O GA pode se concentrar em descobrir:

- bons padrões de bateria;
- boas progressões de acordes;
- boas linhas de baixo dentro dos acordes;
- boas linhas de guitarra dentro dos acordes.

E as próprias restrições garantem uma base musical razoável.

Por exemplo, se:

```
MChord = Am = {A, C, E}
```

o baixo nunca vai produzir `F#` por acidente, e a guitarra também não.

Mas há um detalhe que eu considero importante

Eu não esperaria que **simplesmente pegar os 5 melhores indivíduos** automaticamente produzisse uma música excelente.

O risco é:

```
[segmento muito bom]
        ↓
[segmento muito bom]
        ↓
[segmento muito bom]
```

mas as transições serem ruins.

Por isso, eu começaria exatamente como você propôs, **sem complicar o GA agora**.

Depois, se funcionar, podemos adicionar uma segunda etapa:

```
Plain text

GA
 ↓
100 indivíduos bons
 ↓
seleção dos 5 melhores/diversos
 ↓
ordenação dos segmentos
 ↓
160 s
```

E posteriormente avaliar:

```
Plain text

fitness do segmento
        +
fitness da transição
        +
diversidade
```

Isso pode melhorar bastante o resultado.

Para um primeiro experimento, eu acho que sua representação está muito mais promissora do que tentar evoluir uma música inteira nota por nota. Você reduziu drasticamente o espaço de busca mantendo uma estrutura musical explícita.

Que nome eu poderia dar a esse estilo musical que estou definindo?

Se você está falando do **estilo musical definido pela representação do seu GA**, eu evitaria chamar de um gênero musical tradicional, porque o que você está definindo é mais uma **estrutura/composição algorítmica** do que um gênero como rock, jazz ou metal.

Alguns nomes que combinam:

1. **Block-Based Music Generation** — simples e técnico.

2. **Chord-Constrained Genetic Music** — destaca a restrição pelos acordes.
3. **Structured Genetic Music** — destaca a estrutura rígida do genótipo.
4. **Hierarchical Genetic Music** — bom se você quiser enfatizar a hierarquia `MChord → baixo/guitarra`.
5. **Chord-Driven Genetic Music** — acho particularmente adequado ao seu caso.
6. **Modular Genetic Music** — destaca a ideia dos blocos de 32 segundos.

Eu escolheria:

Chord-Driven Modular Genetic Music

Porque descreve bem a ideia:

```
Chord-Driven
     ↓
MChord
/      \
Bass     Guitar
↓          ↓
4 notas     8 notas

Modular
    ↓
32-second blocks
    ↓
5 blocks → 160 seconds

Genetic
    ↓
Genetic Algorithm
```

Em português:

Geração Musical Genética Modular Orientada por Acordes

Ou, se quiser um nome mais curto para usar no código/artigo:

ChordGen

Mas se você estiver pensando em **um gênero musical propriamente dito**, como o nome de um estilo que a música resultante teria, aí precisamos definir também características como instrumentação, andamento, ritmo, progressões harmônicas etc. Nesse caso, podemos criar um nome de gênero específico para o estilo que você está construindo.

Mas a qual estilo ele se parece mais (Rock, classic, techno), considerando os instrumentos, acordes e restrições.

Considerando **a instrumentação e a estrutura que você definiu até agora**, eu diria que ele se aproxima mais de **Rock**, mas com uma estrutura bastante simplificada.

A comparação seria aproximadamente:

Característica	Rock	Clássica	Techno	Seu sistema
Bateria	✅	⚠️	✅	✅
Baixo	✅	⚠️	✅	✅
Guitarra	✅	×	×	✅
Acordes	✅	✅	⚠️	✅
Piano/teclas	⚠️	✅	✅	futuro
Estrutura em blocos	⚠️	⚠️	✅	✅
Repetição de padrões	✅	⚠️	✅	✅
Notas restritas aos acordes	✅	✅	⚠️	✅

🎸 Por isso, eu classificaria como Rock

Principalmente por causa da combinação:

```
Plain text
Bateria
   +
Baixo
   +
Guitarra
   +
Progressão de acordes
```

Essa é uma formação muito característica de rock.

Além disso, a sua guitarra e o baixo estão sendo derivados dos acordes, o que combina bem com uma abordagem de **rock harmônico simples**.

Mas há uma coisa interessante

A sua estrutura de:

```
Plain text

4 segundos
    ↓
MChord
    ↓
Bass + Guitar
```

e a repetição:

```
Plain text

MChord1 → MChord2 → MChord3 → MChord4
       ↘ repetição ↗
```

também tem uma característica bastante **eletrônica/Techno**: música construída a partir de padrões repetíveis.

Então eu diria:

> **Musicalmente, a instrumentação está mais próxima de Rock, enquanto a estrutura algorítmica é mais próxima de música eletrônica/Techno.**

Se você quiser que o resultado realmente soe como **rock**, eu acrescentaria algumas restrições específicas à bateria e à guitarra. Por exemplo, guitarra com padrões de **power chords (tônica + quinta)**, bateria com kick/snare em posições características e baixo enfatizando a tônica.

Isso provavelmente deixaria o resultado muito mais reconhecível como rock sem aumentar muito o espaço de busca do GA.

Monday 7:06 PM

> Ao invés de trabalhar com duração em segundos, vamos trabalhar com número de compassos da música colocando o BPM como variável global. Com BPM = 120, quanto tempo teria um compasso e quantas seminimas?

Sim. Isso fica **bem melhor para a representação musical**, porque o tamanho do indivíduo passa a ser definido musicalmente, e não por segundos.

Assumindo o compasso mais comum, **4/4**, e `BPM = 120` :

- BPM = 120 → **120 semínimas por minuto**
- Uma semínima dura:

$$\frac{60}{120} = 0{,}5 \text{ s}$$

- Um compasso 4/4 possui **4 semínimas**
- Portanto:

$$4 \times 0{,}5 = 2 \text{ s}$$

Resultado

BPM = 120 → 1 compasso de 4/4 = 2 segundos = 4 semínimas.

Então, se mantivermos os seus blocos:

```
Plain text
MChord1
MChord2
MChord3
MChord4
```

e cada MChord durar, por exemplo, **2 compassos**:

```
Plain text
MChord1 → 2 compassos = 4 s
MChord2 → 2 compassos = 4 s
MChord3 → 2 compassos = 4 s
MChord4 → 2 compassos = 4 s
```

Um ciclo completo teria:

```
Plain text
4 MChords × 2 compassos = 8 compassos
```

ou:

```
Plain text
8 compassos × 2 s = 16 s
```

Como você repete os quatro MChords duas vezes:

```
Plain text
8 compassos + 8 compassos = 16 compassos
```

ou:

```
Plain text
16 × 2 s = 32 s
```

Ou seja, **a representação anterior de 32 segundos pode ser mantida exatamente**, mas agora ela é definida musicalmente como **16 compassos em 4/4**, em vez de `32 segundos`.

Isso é uma mudança conceitualmente muito boa para o seu GA. A partir daí podemos definir coisas como:

```
Plain text
BPM = 120
TIME_SIGNATURE = 4/4
BARS_PER_MCHORD = 2
NUM_MCHORDS = 4
```

e o tempo em segundos passa a ser apenas uma **consequência** dessas variáveis.

Beleza então cada MChord dura um compasso (2s) e o Bat1 também dura um compasso (2s), os demais blocos seguem a restrição do MChord com 4 semínima cada, certo?

Exatamente. E isso deixa a representação **bem mais limpa**.

Com:

```
Plain text
BPM = 120
Compasso = 4/4
```

temos:

- **1 semínima = 0,5 s**
- **1 compasso = 4 semínimas = 2 s**

Então:

Estrutura de cada indivíduo

O indivíduo continua tendo:

```
Plain text
[Bat1, MChord1, MChord2, MChord3, MChord4]
```

Mas agora cada elemento representa **1 compasso**:

```
Plain text
Bat1    = 1 compasso = 4 semínimas = 2 s
MChord1 = 1 compasso = 4 semínimas = 2 s
MChord2 = 1 compasso = 4 semínimas = 2 s
MChord3 = 1 compasso = 4 semínimas = 2 s
MChord4 = 1 compasso = 4 semínimas = 2 s
```

Os MChords são executados sequencialmente:

```
Plain text
```

```
Compasso 1 → MChord1
Compasso 2 → MChord2
Compasso 3 → MChord3
Compasso 4 → MChord4
Compasso 5 → MChord1
Compasso 6 → MChord2
Compasso 7 → MChord3
Compasso 8 → MChord4
```

Como `Bat1` dura um compasso e se repete:

Plain text

```
Bat1 ──────────────────────────────────────────────►
       ↓       ↓       ↓       ↓       ↓
      MC1     MC2     MC3     MC4     MC1 ...
```

Baixo

Cada MChord gera **4 notas de baixo**, exatamente uma por semínima:

Plain text

```
MChord1
│
├── Beat 1 → Bass note 1
├── Beat 2 → Bass note 2
├── Beat 3 → Bass note 3
└── Beat 4 → Bass note 4
```

Portanto:

Plain text

```
4 notas × 0,5 s = 2 s
```

Guitarra

A guitarra terá **8 notas por MChord**.

Como o compasso tem apenas 4 semínimas, podemos colocar **2 notas por semínima**:

Plain text

```
1   2   3   4
↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
8 notas
```

Ou seja, cada nota de guitarra dura inicialmente:

$$\frac{0,5}{2} = 0,25s$$

Isso também corresponde a **colcheias**.

Resultado

Cada indivíduo agora representa:

Plain text

```
8 compassos × 2 s = 16 segundos
```

E os 5 melhores:

Plain text

```
5 × 16 s = 80 segundos
```

Então, **com a nova definição, 5 indivíduos geram uma música de 80 segundos**, não 160 s.

Para chegar aos **160 segundos**, bastariam concatenar os **10 melhores indivíduos**.

E eu acho essa definição melhor que a anterior: **o GA passa a trabalhar em compassos, sem depender diretamente de segundos**.

> Vou começar a implementar o GA só com a bateria (Só com o Bat1), se ficar bom eu acrescento os MChords, o que acha?

Acho **uma excelente estratégia**. Na verdade, eu começaria exatamente assim.

Você consegue validar primeiro se o **GA está realmente aprendendo padrões de bateria interessantes**, sem misturar ainda os problemas de harmonia, baixo e guitarra.

Etapa 1 — somente bateria

O indivíduo seria simplesmente:

```
Plain text
```

```
Individual = [Bat1]
```

Onde:

```
Plain text
```

```
Bat1 = 1 compasso
      = 4/4
      = 4 semínimas
      = 2 segundos @ 120 BPM
```

E a bateria pode ser representada como um piano roll, por exemplo:

```
Plain text
```

```
             1   &   2   &   3   &   4   &
Kick         ■       ■       ■       ■
Snare                ■               ■
Closed HH    ■   ■   ■   ■   ■   ■   ■   ■
Open HH                      ■
Crash        ■
```

Se usarmos **8 subdivisões por compasso**, cada posição corresponde a uma colcheia.

Mas há uma questão importante

Se o indivíduo tiver somente um `Bat1`, você precisa decidir como ele será usado na música.

Por exemplo:

```
Plain text
```

```
Bat1
  ↓
repete
  ↓
repete
  ↓
repete
  ↓
...
```

Assim, o GA aprende um **loop de bateria de 1 compasso**.

Isso é ótimo para testar:

- representação do indivíduo;
- crossover;
- mutação;
- seleção;
- fitness;
- evolução;
- qualidade musical.

Depois adicionamos os MChords

Quando a bateria estiver funcionando:

```
Plain text
Individual = [Bat1]
```

passamos para:

```
Plain text
Individual = [Bat1, MChord1, MChord2, MChord3, MChord4]
```

E posteriormente:

```
Plain text
Individual = [
    Bat1,
    MChord1,
    MChord2,
    MChord3,
    MChord4
]
```

com o fenótipo gerando:

```
Plain text
```

```
Bateria
   +
Acordes
   +
Baixo
   +
Guitarra
```

Uma vantagem metodológica

Você poderá mostrar claramente a evolução do projeto:

Experimento 1

> Genetic algorithm for drum pattern generation.

↓

Experimento 2

> Genetic algorithm for chord-constrained music generation.

↓

Experimento 3

> Multi-instrument genetic music generation.

Isso também facilita muito a análise experimental, porque se o sistema final não funcionar bem, você consegue descobrir **em qual etapa o problema apareceu**.

Eu começaria pelo Bat1 sem dúvida. E manteria desde já a representação em piano roll e o conceito de `BPM + 4/4`, para não precisar mudar a representação posteriormente.

> Vamos começar a implementar em python, porém vou te passar exemplos da versão anterior para você seguir como modelo (nomes das classes, funções, etc.), pois vou reaproveitar o código anterior...

Perfeito. Pode me passar os arquivos/códigos da versão anterior.

Vou **reaproveitar a estrutura existente** — nomes de classes, funções, organização do GA, seleção, crossover, mutação, fitness etc. — e alterar somente o necessário para a nova representação:

Plain text

```
Individual
    └── Bat1
        └── piano roll de 1 compasso
```

Com as premissas que definimos:

- **BPM = 120** como variável global;
- **compasso = 4/4**;
- `Bat1` = **1 compasso**;
- 1 compasso = **4 semínimas = 2 s**;
- inicialmente **somente bateria**;
- representação da bateria em **piano roll**;
- depois poderemos adicionar `MChord1...MChord4` sem precisar reconstruir o GA inteiro.

Pode mandar o código anterior, inclusive em partes se for grande.

Arquivo genotype.py:

```
class Gene:
    """
    Representa um gene musical.

    notes:
        Lista de 0 a 5 notas.
        Cada nota é representada por um pitch MIDI.
    duration:
        Duração da nota em PASSOS da grade (não em segundos).

        MIN_DURATION (1) é a duração normal: a nota ocupa só o
próprio
        passo (semicolcheia). Valores maiores fazem a nota sustentar e
        invadir os passos seguintes, até MAX_DURATION.
    """

    MAX_NOTES = 5

    # Duração em passos da grade
    MIN_DURATION = 1
    MAX_DURATION = 4
```

```
    def __init__(self, notes=None, duration=None):
        self.notes = [] if notes is None else list(notes)

        if duration is None:
            self.duration = self.MIN_DURATION
        else:
            if not isinstance(duration, int):
                raise TypeError(
                    "A duração deve ser um inteiro em passos."
                )

            self.duration = duration

        if len(self.notes) > self.MAX_NOTES:
            raise ValueError(
                f"Um gene pode conter no máximo {self.MAX_NOTES}
notas."
            )

        for note in self.notes:
            if not isinstance(note, int):
                raise TypeError("Cada nota deve ser um inteiro.")

            if not 0 <= note <= 127:
                raise ValueError(
                    f"Pitch MIDI inválido: {note}. "
                    "O intervalo permitido é 0-127."
                )

        if not self.MIN_DURATION <= self.duration <=
self.MAX_DURATION:
            raise ValueError(
                f"Duração inválida: {self.duration} passos. "
                f"O intervalo permitido é "
                f"{self.MIN_DURATION}-{self.MAX_DURATION}."
            )
```

```
    def __repr__(self):
        if self.duration != self.MIN_DURATION:
            return f"Gene({self.notes}, {self.duration})"

        return f"Gene({self.notes})"


class Genotype:
    """
    Genótipo de uma música.

    Configuração:
        - 2 trilhas: piano e drums
        - 30 segundos
        - 120 BPM
        - 4/4
        - 16 passos por compasso (semicolcheias)
        - 240 passos no total

    piano:
        Lista de STEPS genes.
        Cada gene pode conter de 0 a MAX_NOTES notas, e sua
duração
        vai de MIN_DURATION a MAX_DURATION passos.

    drums:
        Lista de STEPS genes.
        Cada gene representa os eventos de bateria daquele passo.
        Percussão não usa duração: fica sempre em MIN_DURATION.
    """

    BPM = 90
    DURATION = 10
    BEATS_PER_BAR = 4
    STEPS_PER_BEAT = 4
```

```
STEPS_PER_BAR = BEATS_PER_BAR * STEPS_PER_BEAT

# 120 BPM -> 0,5 s por batida
# 16 passos por compasso -> 0,125 s por passo
STEPS = int(
    DURATION * BPM / 60 * STEPS_PER_BEAT
)

def __init__(self, piano=None, drums=None):
    self.piano = (
        [Gene() for _ in range(self.STEPS)]
        if piano is None
        else list(piano)
    )

    self.drums = (
        [Gene() for _ in range(self.STEPS)]
        if drums is None
        else list(drums)
    )

    self._validate()

def _validate(self):
    if len(self.piano) != self.STEPS:
        raise ValueError(
            f"A trilha de piano deve possuir {self.STEPS} genes."
        )

    if len(self.drums) != self.STEPS:
        raise ValueError(
            f"A trilha de drums deve possuir {self.STEPS} genes."
        )

    if not all(isinstance(gene, Gene) for gene in self.piano):
        raise TypeError(
            "Todos os elementos de piano devem ser objetos Gene."
```

```
            )

        if not all(isinstance(gene, Gene) for gene in self.drums):
            raise TypeError(
                "Todos os elementos de drums devem ser objetos Gene."
            )

    def __repr__(self):
        return (
            f"Genotype("
            f"piano={self.piano}, "
            f"drums={self.drums})"
        )
```

Gere uma nova versão desse arquivo de modo que o genótipo agora tenha apenas o elemento Bat1 (Compasso de bateria com 8 colcheias), BPM = 120, com os 5 elementos de bateria que você recomendou acima. Com polifonia máxima de 3 por step (cada step uma colcheia).

Sim. Para essa primeira versão, eu deixaria o `Genotype` extremamente simples: ele contém somente um `Bat1`, formado por **8 steps**, cada um representando uma **colcheia**.

A polifonia máxima de 3 significa que em cada step podem ocorrer no máximo 3 elementos simultaneamente.

</> Python

```
class Genotype:
    """
    Genótipo de uma música contendo apenas um padrão de bateria (Bat1).

    Configuração:
        - 1 trilha: Bat1
        - 1 compasso
        - 4/4
        - BPM = 120
        - 8 steps por compasso
        - Cada step representa uma colcheia
        - 5 elementos de bateria
        - Polifonia máxima de 3 elementos por step
```

```
    Bat1:
        Lista de 8 steps.

        Cada step é uma lista contendo de 0 a 3 elementos de bateria.

        Elementos disponíveis:
            0 = Kick
```

A representação fica assim

Por exemplo:

Python

```
bat1 = [
    [0],          # 1: Kick
    [2],          # 2: Closed Hi-Hat
    [0, 2],       # 3: Kick + Closed Hi-Hat
    [2],          # 4: Closed Hi-Hat
    [1, 2],       # 5: Snare + Closed Hi-Hat
    [2],          # 6: Closed Hi-Hat
    [0, 2],       # 7: Kick + Closed Hi-Hat
    [2, 4]        # 8: Closed Hi-Hat + Crash
]
```

Visualmente:

Plain text

```
Step       1   2   3   4   5   6   7   8
           ♪   ♪   ♪   ♪   ♪   ♪   ♪   ♪
           --------------------------------
Kick       X       X           .       X
Snare                  .       X
Closed HH      X   X   X   X   X   X   X
Open HH
Crash                                      X
```

Cada step é uma **colcheia**. Como temos 8 colcheias:

Plain text

```
8 × 0,25 s = 2 segundos
```

pois, a 120 BPM:

Plain text

```
1 semínima = 0,5 s
1 colcheia = 0,25 s
1 compasso 4/4 = 2 s
```

Um detalhe importante: **eu removi completamente `Gene` nesta primeira versão**. Como agora cada `Bat1` é estruturalmente um padrão de bateria de 8 steps, a antiga ideia de `Gene = notas + duração` não se encaixa mais tão bem. Cada posição do `bat1` já é o nosso gene/step da bateria.

Isso também deixa o próximo `genetic_algorithm.py` bem mais simples de adaptar.

Por que você não colocou a classe Gene?

Você tem razão. **Foi uma decisão minha que não deveria ter tomado sem preservar a estrutura do código anterior.**

Eu removi `Gene` porque interpretei que cada `step` poderia ser diretamente uma lista de elementos de bateria. Mas, como você quer **reaproveitar o GA anterior**, é melhor manter a classe `Gene` e fazer o `Bat1` ser uma lista de `Gene`.

A estrutura deveria ser:

Plain text

```
Genotype
   └── bat1
        ├── Gene 1 → até 3 instrumentos
        ├── Gene 2 → até 3 instrumentos
        ├── Gene 3 → até 3 instrumentos
        ...
        └── Gene 8 → até 3 instrumentos
```

E podemos manter a semântica:

Python

```
Gene.notes
```

para representar os instrumentos presentes naquele step.

Por exemplo:

Python

```
Gene([0, 2])
```

significa:

Plain text

```
Kick + Closed Hi-Hat
```

Eu faria o `genotype.py` assim

Python

```
class Gene:
    """
    Representa um step de bateria.

    notes:
        Lista de elementos de bateria presentes no step.

        Elementos:
            0 = Kick
            1 = Snare
            2 = Closed Hi-Hat
            3 = Open Hi-Hat
            4 = Crash

        Um step pode possuir no máximo 3 elementos
        simultaneamente.
    """

    MAX_NOTES = 3

    def __init__(self, notes=None):
```

Assim preservamos a arquitetura anterior:

Plain text

```
Genotype
    ↓
```

```
Bat1
  ↓
Gene × 8
```

e cada `Gene` continua sendo uma unidade que o algoritmo genético pode manipular.

Isso é **bem melhor para o reaproveitamento do `genetic_algorithm.py` anterior**, principalmente porque provavelmente seu crossover e sua mutação já trabalham diretamente com objetos `Gene`.

Olhe como estava o código individual.py anterior:

```
from genotype import Genotype, Gene

class Individual:
    """
    Representa um indivíduo da população.

    Um indivíduo possui um genótipo completo, composto por:
        - 240 genes de piano
        - 240 genes de drums
    """

    def __init__(self, genotype=None):
        self.genotype = (
            Genotype()
            if genotype is None
            else genotype
        )

    @property
    def piano(self):
        return self.genotype.piano

    @property
    def drums(self):
        return self.genotype.drums

    def __repr__(self):
```

```
        return (
            f"Individual("
            f"piano={self.piano}, "
            f"drums={self.drums})"
        )
```

Gere uma nova versão adequada ao novo genotype.py

Sim. Mantendo a mesma estrutura da versão anterior, agora o `Individual` só precisa encapsular o `Genotype` e expor o `Bat1`.

Python

```
from genotype import Genotype, Gene


class Individual:
    """
    Representa um indivíduo da população.

    Um indivíduo possui um genótipo completo, composto por:
        - 1 padrão de bateria (Bat1)
        - 8 genes, sendo um para cada colcheia do compasso
    """

    def __init__(self, genotype=None):
        self.genotype = (
            Genotype()
            if genotype is None
            else genotype
        )

    @property
    def bat1(self):
```

A estrutura agora fica:

Plain text

```
Individual
    │
    └── Genotype
          │
          └── Bat1
                │
```

```
├── Gene 1 → colcheia 1
├── Gene 2 → colcheia 2
├── Gene 3 → colcheia 3
├── Gene 4 → colcheia 4
├── Gene 5 → colcheia 5
├── Gene 6 → colcheia 6
├── Gene 7 → colcheia 7
└── Gene 8 → colcheia 8
```

Assim, quando adicionarmos os `MChord` futuramente, podemos simplesmente expandir o `Individual` para expor os novos componentes, sem precisar mudar a ideia central da classe.

Verifique se este código de fitness.py vai funcionar com os novos códigos:

"""
fitness.py

Fitness por SIMILARIDADE com indivíduos de referência.

Em vez de rewarding estatísticas isoladas, esta fitness mede o quanto um
indivíduo se parece com a música REAL, da qual foram extraídos os indivíduos de referência (ver utils/generate_refs.py).

A comparação é feita sobre descritores INDEPENDENTES da duração:
  - repetição   : fração de compassos que repetem o mesmo padrão
  - perfil tonal : histograma das 12 classes de altura
  - registro   : histograma das OITAVAS (vê C3 vs C4, que o %12 não
…

Show more

Não. **Esse `fitness.py` não funcionará corretamente com o novo `genotype.py`**, principalmente porque ele ainda pressupõe duas trilhas (`piano` e `drums`) e várias métricas relacionadas ao piano.

Para a etapa atual, em que temos **somente `Bat1`**, eu simplificaria bastante a fitness.

O ponto importante é que podemos **preservar a ideia da fitness anterior**: comparar o padrão gerado com padrões de referência. Mas agora os descritores precisam ser específicos para bateria.

Por exemplo:

- **ritmo da bateria** — posições dos ataques;
- **densidade** — quantidade de eventos;
- **repetição** — neste momento será 1, porque o indivíduo é um único compasso;
- **distribuição dos instrumentos** — Kick, Snare, Hi-Hat etc.;
- futuramente podemos adicionar características mais musicais do padrão.

Além disso, existe um problema conceitual no `fitness.py` atual:

```python
SECONDS_PER_STEP = (60.0 / Genotype.BPM) / Genotype.STEPS_PER_BEAT
```

Isso está correto para a nova representação **se `STEPS_PER_BEAT = 2`** , resultando em:

```
120 BPM
↓
0,5 s / semínima
↓
0,25 s / colcheia
```

Então podemos manter essa parte.

Mas eu faria uma mudança importante

Como o seu `Gene` agora contém:

```python
Gene([0, 2])
```

isso significa **Kick + Closed Hi-Hat no mesmo step**.

Portanto, não devemos tratar simplesmente cada `Gene` como uma nota MIDI. Para a bateria, precisamos interpretar os números como **instrumentos de percussão**.

Por exemplo:

Python

```
DRUMS = {
    0: "Kick",
    1: "Snare",
    2: "Closed_HiHat",
    3: "Open_HiHat",
    4: "Crash",
}
```

E podemos calcular a fitness diretamente sobre:

Python

```
individual.bat1
```

em vez de:

Python

```
individual.piano
individual.drums
```

Eu sugiro fazer agora um `fitness.py` específico para Bat1

A arquitetura ficaria:

Plain text

```
Individual
    │
    └── Genotype
          │
          └── Bat1
                │
                ├── Gene
                ├── Gene
                ├── Gene
                ├── Gene
                ├── Gene
```

```
├── Gene
├── Gene
└── Gene
      ↓
   FITNESS
```

E não precisamos tentar adaptar artificialmente as métricas de piano.

Se você quiser manter o máximo possível do código antigo, também podemos reaproveitar as funções genéricas (`_normalize` , `_histogram_similarity` , `_ratio_similarity` , `_bucket_index` , carregamento das referências etc.) e substituir somente a parte dos descritores.

Eu recomendo essa segunda opção: **manter a estrutura do `fitness.py` anterior, mas adaptá-la para `Bat1`** , para que o restante do seu `genetic_algorithm.py` precise de pouquíssimas alterações.

> Gere um novo código de fitness.py para avaliar a qualidade somente da bateria considerando regras de teoria musical (use o código de fitness que te passei como exemplo)

Sim. Para essa primeira etapa eu faria a fitness **independente de referências reais** e baseada apenas em regras de construção de bateria.

Como o `Bat1` tem apenas **1 compasso / 8 colcheias**, algumas métricas do fitness anterior deixam de fazer sentido, principalmente repetição de compassos, tonalidade e contorno melódico.

Eu usaria:

- **Kick**: favorece posições fortes do compasso.
- **Snare**: favorece os tempos 2 e 4.
- **Hi-hat**: favorece regularidade.
- **Open Hi-hat**: evita excesso.
- **Crash**: evita excesso e favorece início do compasso.
- **Combinações**: evita combinações pouco naturais.
- **Densidade**: evita compasso vazio ou excessivamente carregado.
- **Variedade**: evita que todos os steps sejam iguais.
- **Polifonia**: penaliza excesso de elementos simultâneos.
- **Variação rítmica**: evita padrões completamente uniformes.

Abaixo está uma versão completa mantendo uma estrutura parecida com o seu `fitness.py` anterior:

Python

```
"""
fitness.py

Fitness para avaliação de padrões de bateria.

O indivíduo contém somente:

    Individual
        └── Genotype
              └── Bat1
                    └── 8 Genes

Cada Gene representa uma colcheia de um compasso 4/4.

Configuração musical:
    - BPM = 120
    - Compasso = 4/4
    - 8 colcheias por compasso
    - 5 elementos de bateria
    - Polifonia máxima = 3
```

Uma observação importante

Eu **não colocaria todas essas regras como restrições duras** no `genotype.py`. Por exemplo, não obrigaria o Snare a aparecer no step 3/7. O GA precisa poder experimentar:

```
Plain text

Kick em posições diferentes
Snare em posições diferentes
Hi-hat irregular
Open hi-hat
```

A fitness apenas dá **maior recompensa para padrões musicalmente mais plausíveis**.

Assim o GA continua tendo espaço para descobrir padrões interessantes.

E, principalmente, eu deixaria `Bat1` como **um único compasso de 8 colcheias**, como definimos. Depois que essa versão estiver funcionando, podemos observar os padrões que o GA realmente está produzindo e ajustar os pesos/regras em vez de tentar definir todas as regras musicais antecipadamente.

Agora veja este generate_midi.py:

```
import pretty_midi

#
============================================================
======
# CONFIGURAÇÃO
#
============================================================
======

BPM = 90
STEPS_PER_BEAT = 4

SECONDS_PER_BEAT = 60.0 / BPM
SECONDS_PER_STEP = SECONDS_PER_BEAT / STEPS_PER_BEAT



#
============================================================
======
# TRILHAS EM MEMÓRIA
#
============================================================
======

def build_tracks(individual,
seconds_per_step=SECONDS_PER_STEP) -> list:
    """
    Converte um Individual em trilhas em memória, sem tocar no
disco.

    Cada trilha é um dicionário:
        name    : nome da trilha
        program : programa General MIDI
        is_drum : True na trilha de percussão
        notes   : lista de tuplas (inicio, fim, pitch)
```

```
    Cada gene representa um passo. No piano as notas são pitches MIDI; na
    bateria são pitches de percussão General MIDI.

    A duração da nota vem do próprio gene: uma nota ocupa gene.duration
    passos a partir do seu passo inicial. Assim uma nota com duração 4
    (o máximo) atravessa quatro semicolcheias, em vez de soar picotada.

    seconds_per_step : duração de um passo na grade em que o indivíduo foi
    construído. Por padrão usa a grade do AG, mas quem gerar referências de
    uma música em outro andamento deve passar o valor correto, senão o
    ritmo sai distorcido.

    O algoritmo genético usa esta função para avaliar em memória, o que
    evita gravar e reler um arquivo MIDI para cada indivíduo.
    """
    tracks = []

    piano_notes = []

    for step, gene in enumerate(individual.piano):
        start = step * seconds_per_step
        end = start + gene.duration * seconds_per_step

        for pitch in gene.notes:
            piano_notes.append((start, end, pitch))

    tracks.append({
        "name": "Piano",
        "program": 0,
        "is_drum": False,
```

```
        "notes": piano_notes,
    })

    drum_notes = []

    for step, gene in enumerate(individual.drums):
        start = step * seconds_per_step

        # Percussão fica com a duração mínima: sustentar bumbo/caixa
não
        # faz sentido com os três sons usados (bumbo, caixa e hi-hat).
        end = start + gene.duration * seconds_per_step

        for pitch in gene.notes:
            drum_notes.append((start, end, pitch))

    tracks.append({
        "name": "Drums",
        "program": 0,
        "is_drum": True,
        "notes": drum_notes,
    })

    return tracks


#
==========================================================
======
# GERAÇÃO DO MIDI
#
==========================================================
======

def generate_midi(individual, output_path="generated.mid",
bpm=None):
    """
```

```
    Gera um arquivo MIDI a partir de um Individual.

    O indivíduo deve possuir:
        individual.piano
        individual.drums

    Cada gene representa um passo da grade em que ele foi construído.

    Piano:
        As notas do Gene são interpretadas como pitches MIDI.

    Drums:
        As notas do Gene são interpretadas como pitches
        de percussão General MIDI.

    bpm : andamento do arquivo. Por padrão usa BPM (120 do AG).

    IMPORTANTE: os tempos das notas são calculados a partir desse
    andamento, então o arquivo toca exatamente no ritmo em que o genótipo
    foi construído. Gravar com um andamento diferente do da grade
    distorce o ritmo.
    """
    tempo = float(BPM if bpm is None else bpm)

    seconds_per_step = (60.0 / tempo) / STEPS_PER_BEAT

    midi = pretty_midi.PrettyMIDI(
        initial_tempo=tempo
    )

    for track in build_tracks(individual, seconds_per_step):

        instrument = pretty_midi.Instrument(
            program=track["program"],
            is_drum=track["is_drum"],
```

```
            name=track["name"]
        )

        for start, end, pitch in track["notes"]:

            instrument.notes.append(
                pretty_midi.Note(
                    velocity=100,
                    pitch=pitch,
                    start=start,
                    end=end
                )
            )

            midi.instruments.append(instrument)

        # ----------------------------------------------------
        # SALVAR
        # ----------------------------------------------------

        midi.write(output_path)
```

E atualize-o se necessário.

Sim. Como agora o genótipo contém **somente `Bat1`**, o `generate_midi.py` antigo precisa ser simplificado. Ele ainda procura `individual.piano` e `individual.drums`, que não existem mais.

Também há uma diferença importante: agora temos **8 steps por compasso**, cada um correspondendo a uma **colcheia**, com **BPM = 120**.

Portanto:

- 120 BPM → semínima = 0,5 s
- 1 colcheia = 0,25 s
- 8 steps = 2 s
- `Bat1` é repetível posteriormente para formar músicas maiores.
- Os números `0..4` do `Gene` precisam ser convertidos para os **pitches MIDI General MIDI de bateria**.

Uma versão atualizada e compatível com o `genotype.py` atual seria:

```python
import pretty_midi


# ============================================================
# CONFIGURAÇÃO
# ============================================================

BPM = 120

BEATS_PER_BAR = 4
STEPS_PER_BEAT = 2

STEPS_PER_BAR = BEATS_PER_BAR * STEPS_PER_BEAT

SECONDS_PER_BEAT = 60.0 / BPM
SECONDS_PER_STEP = SECONDS_PER_BEAT / STEPS_PER_BEAT


# ============================================================
# MAPA DOS ELEMENTOS DE BATERIA
# ============================================================
```

Uma mudança importante

No código antigo havia isto:

```python
end = start + gene.duration * seconds_per_step
```

Agora isso não faz mais sentido, porque o novo `Gene` **não possui `duration`**. Cada evento de bateria simplesmente ocupa o próprio step.

Assim, para um compasso:

```
Step:       0     1     2     3     4     5     6     7
Tempo:     0.00  0.25  0.50  0.75  1.00  1.25  1.50  1.75
```

e o compasso termina em:

Plain text

```
2.00 s
```

Isso deixa o `generate_midi.py` coerente com o novo modelo:

Plain text

```
Individual
    │
    └── Genotype
          │
          └── Bat1
                │
                ├── Gene 0
                ├── Gene 1
                ├── ...
                └── Gene 7
```

E a conversão:

Plain text

```
Gene [0, 2]
       │  │
       │  └── Closed Hi-Hat → MIDI 42
       └─────  Kick          → MIDI 36
```

Isso também é interessante para o próximo passo do GA: **o cromossomo continua trabalhando com símbolos simples `0..4`, enquanto somente o gerador MIDI conhece os pitches General MIDI**. Assim, a representação genética fica independente do formato MIDI.

O código de fitness que eu te passei era baseado em similaridade com midis existentes, por favor gere novamente o código fitness.py considerando este como referência, baseado em teoria musical:

"""

fitness.py

Função de avaliação (fitness) para indivíduos musicais.

A função principal é:

calculate_fitness(individual, refs_dir=None, limit=MAX_REFS) -> float

Atualmente o fitness é baseado em regras musicais:
- tonalidade
- consonância
- comportamento melódico
- repetição
…

Show more

Sim. Como o genótipo atual contém **somente `Bat1`**, não faz sentido manter tonalidade, consonância e melodia. Mas podemos manter a mesma filosofia do seu `fitness.py` antigo: **vários critérios musicais independentes, normalizados em `[0,1]` e combinados por pesos**.

Para uma bateria de **1 compasso 4/4 com 8 colcheias**, eu usaria critérios de teoria musical/rítmica como:

- **strong_beats** — alinhamento com tempos fortes;
- **backbeat** — presença da caixa nos tempos 2 e 4;
- **kick_pattern** — comportamento musical do bumbo;
- **hihat_pattern** — regularidade das subdivisões;
- **syncopation** — presença controlada de síncopes;
- **density** — densidade adequada;
- **variation** — variação entre as duas metades do compasso;
- **combination** — combinações instrumentais plausíveis;
- **structure** — existência de estrutura/repetição interna.

Não usaria "polyrhythm" como no código anterior, porque ter vários instrumentos **não é propriamente polirritmia**.

Segue o `fitness.py` completo, mantendo uma estrutura bastante próxima do seu original.

```python
"""
fitness.py

Função de avaliação (fitness) para indivíduos musicais.
```

```
O indivíduo atual contém somente:

    Individual
        └── Genotype
              └── Bat1
                    └── 8 Genes

Cada Gene representa uma colcheia de um compasso 4/4.

Elementos de bateria:

    0 = Kick
    1 = Snare
    2 = Closed Hi-Hat
    3 = Open Hi-Hat
    4 = Crash
```

Uma diferença importante em relação ao código anterior

Eu **não colocaria `tonal`, `consonance` ou `melody`** nessa versão. Esses critérios faziam sentido para o indivíduo antigo:

```
Plain text

Piano + Drums
```

mas agora estamos testando exclusivamente:

```
Plain text

Bat1
 ├── Kick
 ├── Snare
 ├── Closed Hi-Hat
 ├── Open Hi-Hat
 └── Crash
```

E há uma questão particularmente importante para o seu experimento: **o fitness não deve simplesmente recompensar "mais eventos"**. Se fizermos isso, o GA provavelmente descobrirá rapidamente que colocar Kick + Snare + Hi-Hat em quase todos os steps é uma estratégia fácil para aumentar a pontuação.

Por isso incluí critérios que entram em conflito controlado:

Plain text

```
densidade
     ↓
estrutura ⟷ variação
     ↓
tempos fortes
     ↓
backbeat
     ↓
síncope
     ↓
kick / hi-hat
     ↓
combinação
```

Isso é mais adequado para o seu objetivo de testar primeiro se o **GA consegue aprender a produzir um groove de bateria plausível** antes de adicionar `MChord`.

Um detalhe que eu considero especialmente importante: **não tentaria fazer esse fitness reconhecer "boa música" perfeitamente**. Para o primeiro experimento, ele deve funcionar como um conjunto de restrições que elimina padrões claramente ruins e cria um gradiente de qualidade para o GA. Depois que o GA de `Bat1` estiver funcionando, podemos comparar os padrões gerados com grooves MIDI reais e acrescentar **similaridade estatística com referências** como uma segunda etapa.

Agora, seguindo este código do genetic_algorithm.py como exemplo, gere um novo genetic_algorithm.py considerando as novas versões de genotype, individual e fitness implementadas:

```
import random
from copy import copy
from concurrent.futures import ProcessPoolExecutor

from individual import Individual
from genotype import Genotype, Gene
from fitness import calculate_fitness
from generate_midi import generate_midi


# ========================
# Parâmetros do algoritmo
```

```
# ========================
```

…

Show more

Sim. Aqui a mudança é maior que apenas adaptar nomes, porque o novo cromossomo tem **somente 8 genes de `Bat1`** . Portanto, não precisamos mais de piano, duração, transposição ou mutações por compassos de várias trilhas.

Também mantive a estrutura geral do seu algoritmo: população → avaliação → seleção por torneio → crossover → mutação → elitismo → MIDI.

Uma decisão importante: como **um indivíduo = exatamente um compasso**, o crossover pode simplesmente cortar entre os 8 genes. Não precisamos de crossover alinhado a compassos maiores.

Segue o `genetic_algorithm.py` completo:

Python

```
import random
from copy import copy
from concurrent.futures import ProcessPoolExecutor

from individual import Individual
from genotype import Genotype, Gene
from fitness import calculate_fitness
from generate_midi import generate_midi


# ============================================================
# PARÂMETROS DO ALGORITMO
# ============================================================

POPULATION_SIZE = 500
GENERATIONS = 5000

# Número de processos usados para avaliar a população.
#
# Para a fitness atual, WORKERS=1 normalmente é mais eficiente,
# pois a avaliação é barata e o custo de serialização dos
```

O que foi removido

A nova versão não possui mais nada relacionado a:

- `piano`;
- `PIANO_PITCHES`;
- transposição de melodias;
- `duration`;
- `DURATION_MUTATION_RATE`;
- `copy_bar()` para piano;
- instrumentos que não existem no novo `Gene`, como Clap, Tom e Ride;
- 16 steps por compasso.

Agora o cromossomo é exatamente:

```
Individual
   │
   └── Bat1
       ├── Gene 0
       ├── Gene 1
       ├── Gene 2
       ├── Gene 3
       ├── Gene 4
       ├── Gene 5
       ├── Gene 6
       └── Gene 7
```

E cada Gene contém no máximo três elementos:

```
Gene([Kick, Closed_HiHat])
Gene([Closed_HiHat])
Gene([Snare, Closed_HiHat])
...
```

Um ponto que eu mudaria no seu experimento

Eu começaria **bem menor que `500 indivíduos × 5000 gerações`** para verificar se o comportamento está correto. Por exemplo:

```python
POPULATION_SIZE = 100
```

```
GENERATIONS = 500
```

Isso já produz **50.000 avaliações** e permite observar rapidamente se a fitness está realmente criando uma pressão evolutiva útil.

Depois podemos aumentar para:

```python
POPULATION_SIZE = 500
GENERATIONS = 5000
```

O mais importante agora é verificar se o GA consegue sair de padrões aleatórios e convergir para algo parecido com:

```
1   &   2   &   3   &   4   &
K   HH  S   HH  K   HH  S   HH
```

ou variações musicalmente plausíveis desse padrão, **sem simplesmente convergir para `Kick + Snare + Hi-Hat` em todos os steps**. Esse será um ótimo teste para saber se a `fitness.py` está realmente funcionando como queremos.

> Já começou muito bem, gerou uma bateria boa, porém não está evoluindo a fitness, já começa em 0.97 e continua sempre com esse valor, mesmo com 100 individuos e 1000 gerações.

Sim — isso mostra que o problema **não está no GA em si**, mas principalmente na **função de fitness + população inicial**.

Se a população inicial já produz `0.97`, o algoritmo praticamente não tem espaço para melhorar. E há um problema adicional: nossa fitness atual está muito permissiva e várias métricas dão valores altos para praticamente qualquer bateria "razoável".

Eu faria uma mudança importante: **separar a qualidade musical em componentes mais discriminativos e reduzir o teto artificial da população inicial**.

O principal problema

Hoje temos algo parecido com:

Plain text

```
fitness ≈
    0.12 * strong_beats
  + 0.18 * backbeat
  + 0.15 * kick
  + 0.12 * hihat
  + ...
```

Cada componente pode facilmente ficar entre `0.8 e 1.0`.

Então uma bateria relativamente comum pode chegar a:

Plain text

```
0.90
0.93
0.96
0.97
```

E o GA fica sem gradiente evolutivo.

Além disso, nossa inicialização já é fortemente direcionada:

Python

```python
if position in (0, 4):
    # Kick forte

if position in (2, 6):
    # Snare forte

# Hi-hat com peso alto
```

Ou seja, **estamos dando ao GA indivíduos próximos da resposta que a fitness deseja encontrar**.

O que eu recomendo

Para esse experimento, eu mudaria a estratégia para:

1. População inicial mais aleatória

Não devemos ensinar o padrão ao algoritmo na inicialização.

Por exemplo:

```
Plain text
```

```
População inicial
       ↓
padrões ruins
padrões razoáveis
padrões bons
padrões muito bons
       ↓
Fitness
       ↓
Seleção
       ↓
Crossover
       ↓
Mutação
       ↓
melhoria progressiva
```

Em vez de:

```
Plain text
```

```
População inicial
       ↓
quase todos padrões bons
       ↓
fitness 0.90–0.98
       ↓
GA não tem muito para melhorar
```

---

2. Tornar a fitness mais exigente

Principalmente:

- Kick
- Snare
- Hi-hat
- densidade

- distribuição temporal
- combinações

devem penalizar mais fortemente padrões ruins.

Por exemplo, um padrão:

```
Plain text
Kick:  1 . 1 . 1 . 1 .
Snare: . . 1 . . . 1 .
Hat:   1 1 1 1 1 1 1 1
```

deve ser bom.

Mas:

```
Plain text
Kick:  1 1 1 1 1 1 1 1
Snare: 1 1 1 1 1 1 1 1
Hat:   1 1 1 1 1 1 1 1
```

deve receber uma penalização significativa.

Hoje a nossa função ainda não diferencia esses casos o suficiente.

---

3. Principalmente: não queremos que 0.97 seja "normal"

Eu acho interessante estabelecer uma interpretação como:

Fitness	Qualidade
0.00–0.20	muito ruim
0.20–0.40	ruim
0.40–0.60	razoável
0.60–0.75	boa

Fitness	Qualidade
0.75–0.85	muito boa
0.85–0.95	excelente
0.95–1.00	excepcional

Então, no início, algo como:

```
Plain text
Geração 1/1000 - Melhor: 0.55
Geração 100/1000 - Melhor: 0.67
Geração 200/1000 - Melhor: 0.72
Geração 300/1000 - Melhor: 0.76
...
Geração 1000/1000 - Melhor: 0.86
```

seria **muito mais interessante experimentalmente**.

---

4. Eu também mudaria a mutação

Tem outro detalhe importante no nosso GA.

Atualmente:

```
Python
MUTATION_RATE = 0.10
```

Isso significa que cada um dos 8 steps tem 10% de chance de sofrer mutação.

Em média:

```
Plain text
8 × 0.10 = 0.8
```

Então cada filho sofre quase uma mutação por indivíduo.

Isso é razoável.

Mas como o espaço é extremamente pequeno:

Plain text

```
8 steps
×
5 instrumentos
×
máximo 3 instrumentos/step
```

não precisamos de uma mutação muito agressiva.

Eu manteria:

Python

```
MUTATION_RATE = 0.08
```

e deixaria o crossover fazer boa parte da exploração.

---

5. Outra coisa que vale muito a pena fazer

Antes de mexer mais no GA, eu colocaria uma análise da população inicial:

Plain text

```
Fitness inicial:

Min:    0.31
Mean:   0.57
Max:    0.74
Std:    0.09
```

Se aparecer:

Plain text

```
Min:    0.91
Mean:   0.95
```

```
  Max:    0.97
  Std:    0.02
```

sabemos imediatamente que **a população inicial está excessivamente boa**.

E se aparecer:

```
Plain text
```

```
Min:    0.40
Mean:   0.55
Max:    0.97
Std:    0.08
```

então sabemos que existe diversidade e que o problema está na fitness.

Minha sugestão para o próximo passo

Eu faria uma pequena mudança estrutural no `genetic_algorithm.py` para imprimir:

```
Plain text
```

```
Geração 1/1000
  Min:    0.3821
  Média:  0.5713
  Max:    0.7428
  Global: 0.7428
```

e depois:

```
Plain text
```

```
Geração 100/1000
  Min:    0.5112
  Média:  0.6834
  Max:    0.8012
  Global: 0.8012
```

Assim conseguimos **ver se realmente existe evolução**, e não apenas o melhor indivíduo.

Mas eu **não mexeria somente no GA**. O próximo ajuste deveria ser principalmente no `fitness.py` e na geração inicial.

A situação atual de `0.97 → 0.97 → 0.97` é um forte sinal de que **a fitness está saturando**.

> Consegui resolver isso aqui.
>
> Para facilitar nossa vida, vou usar um agente do vs code para gerar o código, mas como você sabe o contexto, vou te pedir ajuda pra explicar pra ele o que tenho que fazer para acrescentar os próximos instrumentos. Ou seja, você me ajuda a gerar os prompts pra passar pra ele, ok?

Perfeito. Podemos fazer dessa forma.

Eu mantenho o **contexto arquitetural e as decisões que já tomamos**, e você usa o agente do VS Code para implementar. Eu posso te ajudar a transformar cada próxima etapa em um **prompt técnico, claro e fechado**, para evitar que o agente altere coisas que não deveria.

A ideia que eu sugiro é evoluirmos **um instrumento por vez**, sempre preservando o que já funciona:

1. 🥁 **Bateria** — já funcionando.
2. 🎹 **Acordes** — adicionar `MChord1...MChord4` .
3. 🎸 **Baixo** — derivado dos acordes.
4. 🎸 **Guitarra** — derivada dos acordes.
5. 🎼 **Integração completa**.
6. 🎵 **MIDI final**.
7. 🧬 Depois, ajustar o GA para evoluir conjuntamente os componentes.

E principalmente manter a regra que você definiu:

```
Plain text

Genotype
│
├── Bat1
│
├── MChord1
├── MChord2
├── MChord3
└── MChord4
```

onde cada `MChord` é **um acorde válido**, em vez de permitir que o algoritmo escolha notas arbitrárias.

Depois:

```
MChord
   │
   ├── Bass → 4 notas
   │
   └── Guitar → 8 notas
```

Isso é importante porque reduz enormemente o espaço de busca e garante validade musical **por construção**.

Como podemos trabalhar

Quando você quiser adicionar, por exemplo, os acordes, você pode me dizer:

> "Quero agora adicionar os acordes. Gere o prompt para o agente."

E eu preparo um prompt dizendo exatamente:

- quais arquivos ele deve modificar;
- quais classes/estruturas devem ser criadas;
- o que deve permanecer inalterado;
- como representar os acordes;
- restrições musicais;
- como gerar a população inicial;
- como fazer crossover;
- como fazer mutação;
- como alterar a fitness;
- como gerar o MIDI;
- quais testes executar;
- e, principalmente, **o que ele não deve fazer**.

Também posso fazer o prompt em um formato que funcione bem para agentes de código, algo como:

```
CONTEXTO
OBJETIVO
ARQUITETURA ATUAL
ALTERAÇÕES NECESSÁRIAS
```

```
RESTRIÇÕES
ARQUIVOS A MODIFICAR
CRITÉRIOS DE ACEITAÇÃO
TESTES
NÃO FAÇA
```

Assim evitamos que o agente "melhore" partes que já estão funcionando ou tente redesenhar o GA inteiro a cada etapa.

Pode me mandar a próxima etapa que você quer implementar e eu preparo o prompt para o agente do VS Code.

> O Bat1 já está no código, agora quero acrescentar os elementos MChord1, MChord2, MChord3 e MChord4 no genotype (e no individuo), de modo que esses 4 acordes sejam sempre acordes válidos de 3 a 5 notas (dentro dos acordes possíveis de música, tanto Menor, Maior, Diminuto, etc). O Genotypo passa a ter 4 elementos: {Bat1, MChord1, MChord2, MChord3, Mchord4}. E as operações de cruzamento mutação devem modoficar, além do Bat1 como já está sendo feito, os MChords de modo a varia-los.
>
> Vamos gerar os prompts para o agente do VS Code em partes... Aguarde que vou explicar a primeira parte...

Perfeito. Vamos fazer **por partes**, sem gerar o prompt ainda.

Entendi a arquitetura-alvo:

```
Plain text

Genotype
├── Bat1
├── MChord1
├── MChord2
├── MChord3
└── MChord4
```

E os `MChord` devem obedecer sempre à restrição:

- cada `MChord` representa **um acorde válido**;
- entre **3 e 5 notas**;
- apenas combinações que correspondam a estruturas harmônicas válidas;

- incluindo, por exemplo, **Maior, Menor, Diminuto**, etc.;
- crossover e mutação poderão alterar os `MChord` ;
- as operações sobre `Bat1` continuam funcionando como estão.

Pode explicar a **primeira parte** que você quer passar ao agente.

Primeiro gere um texto pedindo o agente para acrescentar os MChords no genotype.py e ajustar o código individual.py para se adequar as alterações.

Claro. Para essa primeira etapa, eu faria o agente **somente alterar a representação dos dados**, sem ainda mexer em crossover, mutação, fitness ou geração do MIDI.

Você pode passar este prompt ao agente:

Adicionar MChords ao Genotype e Individual

Objetivo

Modificar a representação genética do projeto para acrescentar quatro acordes harmônicos (`MChord1`, `MChord2`, `MChord3` e `MChord4`) ao genótipo existente.

Ótimo, só uma dúvida:

Qual representação de midi está sendo usada atualmente (Piano Roll, MusicXML, etc) ?

Atualmente, o projeto está usando **uma representação MIDI simbólica própria**, e não Piano Roll nem MusicXML.

Internamente no GA

O `Genotype` representa a música como objetos Python.

Para a bateria:

```
Bat1
│
├── Gene [Kick, HiHat]
├── Gene [HiHat]
├── Gene [Snare, HiHat]
├── Gene [HiHat]
```

```
├── ...
└── Gene [...]
```

Os instrumentos de bateria são IDs:

Plain text

```
0 = Kick
1 = Snare
2 = Closed Hi-Hat
3 = Open Hi-Hat
4 = Crash
```

Então, por exemplo:

Python

```
Gene([0, 2])
```

significa:

Plain text

```
Kick + Closed Hi-Hat
```

Na saída

O `generate_midi.py` pega essa representação e converte para um **arquivo `.mid` padrão MIDI** usando `pretty_midi`.

Por exemplo:

Python

```
DRUM_PITCHES = {
    0: 36,
    1: 38,
    2: 42,
    3: 46,
    4: 49,
}
```

Ou seja:

```
Representação genética
        ↓
    Python objects
        ↓
    MIDI pitches
        ↓
   pretty_midi
        ↓
      .mid
```

E o Piano Roll?

Não estamos usando Piano Roll atualmente.

Mas há uma relação conceitual:

Um Piano Roll poderia representar exatamente os mesmos eventos como uma matriz:

```
          Step
       0 1 2 3 4 5 6 7

Kick   1 0 1 0 1 0 0 0
Snare  0 0 1 0 0 0 1 0
CHat   1 1 1 1 1 1 1 1
```

Só que nosso GA usa uma representação **estrutural/simbólica baseada em genes**, em vez de uma matriz Piano Roll.

Para os **MChords**, eu acho essa abordagem atual particularmente interessante, porque podemos representar algo como:

```python
MChord(
    notes=[60, 64, 67],
    root=60,
    quality="major"
)
```

em vez de simplesmente colocar `1`s em uma matriz Piano Roll.

Isso nos permite impor diretamente a restrição de que **o cromossomo contém acordes válidos**, que é justamente o que você quer para reduzir o espaço de busca.

> Tá ficando legal, mas só um detalhe: O enunciado do TP pede uma representação específica (MusicXML, Piano Roll, outra). Não vejo problema em usar a representação atual mas preciso de uma justificativa pra nao ter usado direto o Piano Roll.

Sim. E acho que você tem uma justificativa **bem defensável**, principalmente porque seu objetivo é usar um **algoritmo genético com restrições musicais**.

A ideia principal é distinguir **representação interna do algoritmo** de **formato de representação musical exigido pelo trabalho**.

Justificativa

Você pode argumentar que o Piano Roll é uma representação adequada para **armazenar e visualizar eventos musicais**, mas não é necessariamente a melhor representação para o **cromossomo do algoritmo genético**.

No seu caso, você quer que o GA manipule unidades musicais semanticamente válidas:

```
Plain text

Bat1
MChord1
MChord2
MChord3
MChord4
```

Se usássemos diretamente Piano Roll, um acorde poderia ser representado por várias células independentes:

```
Plain text

C4 ——|——
E4 ——|——
G4 ——|——
```

Durante crossover e mutação, o algoritmo poderia facilmente produzir:

```
Plain text
C4 + F#4 + G#4
```

sem que isso correspondesse necessariamente a um acorde que queremos permitir.

Ou seja, o Piano Roll **não garante a validade musical da unidade manipulada pelo GA**.

Na representação atual:

```
Plain text
MChord1 = [C4, E4, G4]
```

o acorde é tratado como **uma unidade genética**.

Isso permite que o algoritmo faça:

```
Plain text
crossover
        ↓
MChord1 ↔ MChord2
```

e:

```
Plain text
mutação
        ↓
C maior → C menor
```

sem quebrar a estrutura do acorde.

Isso também combina muito bem com o objetivo do seu trabalho

Você pode dizer que está utilizando uma **representação simbólica hierárquica/estruturada**, em vez de uma representação matricial de baixo nível.

Algo como:

> **A representação genética foi projetada especificamente para o processo evolutivo, priorizando a preservação de estruturas musicais válidas durante crossover e mutação. Embora o Piano Roll seja uma representação adequada para eventos musicais em nível de nota, sua utilização direta como cromossomo permitiria que operações genéticas produzissem combinações estruturalmente inválidas. Por isso, adotou-se uma representação simbólica hierárquica, na qual unidades musicais como padrões de bateria e acordes são representadas como genes estruturados. A conversão para MIDI/Piano Roll pode ser realizada posteriormente para execução, visualização ou avaliação.**

Eu particularmente **não diria que "Piano Roll é ruim"**. A justificativa mais forte é:

> **Piano Roll é uma representação musical; nossa representação é uma representação genética especializada.**

Isso é muito mais difícil de contestar.

E existe ainda uma vantagem importante

Seu cromossomo final será:

```
Plain text
[Bat1, MChord1, MChord2, MChord3, MChord4]
```

Isso é extremamente conveniente para explicar o algoritmo:

```
Plain text
                CROMOSSOMO
                    |
      ┌─────────────┼─────────────┐
      |             |             |
    Bat1       MChord1...     MChord4
      |             |             |
  8 steps       acorde válido
```

Você está, na verdade, fazendo uma espécie de **representação genética musical de alto nível**.

Depois podemos gerar:

```
Plain text
```

```
Representação genética
        ↓
Eventos MIDI
        ↓
MIDI
        ↓
Piano Roll / MusicXML
```

Assim você pode satisfazer o requisito do TP **sem sacrificar a arquitetura do algoritmo genético**.

E, se o enunciado exigir especificamente que **o resultado final** esteja em Piano Roll, podemos simplesmente fazer o conversor no final. Aí fica ainda mais fácil justificar: **Piano Roll é o formato de saída/avaliação, enquanto a representação interna foi escolhida para tornar as operações evolutivas musicalmente válidas.**

> Certo, tá ficando muito melhor do que a versão que eu tentei com piano roll representando a música inteira.

Exatamente. A principal vantagem que estamos obtendo é que o **espaço de busca ficou muito mais estruturado**.

No Piano Roll da música inteira, o GA podia fazer algo como:

```
Plain text
nota → nota → silêncio → nota → nota...
```

em milhares de posições independentes. O crossover e a mutação podiam destruir facilmente estruturas musicais.

Agora estamos fazendo:

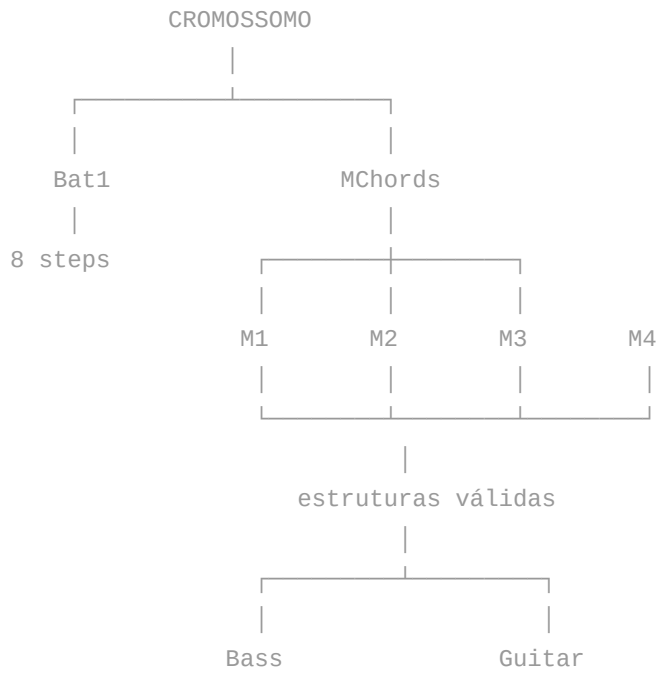
```
Plain text
Cromossomo
│
├── Bat1
│
├── MChord1
├── MChord2
├── MChord3
└── MChord4
```

Então cada gene tem **significado musical**.

Isso deve ficar ainda mais interessante quando adicionarmos baixo e guitarra, porque eles poderão ser **condicionados pelos MChords**, em vez de serem outros espaços de busca completamente independentes.

A arquitetura fica aproximadamente:

Plain text



E isso é justamente o que eu acho mais interessante para o seu TP: **o GA não está simplesmente procurando uma matriz de notas; ele está evoluindo uma estrutura musical.**

Podemos continuar exatamente nessa linha, adicionando cada componente gradualmente e testando o GA a cada etapa.

Mas tem um viés musical meu né kkkk

KKKK sim! 😄

E isso é importante reconhecer: **o algoritmo está sendo projetado com um viés musical seu**.

Por exemplo, quando decidimos:

- bateria como base;
- acordes de 3–5 notas;

- Maior, Menor, Diminuto etc.;
- baixo derivado dos acordes;
- guitarra derivada dos acordes;
- 4 acordes por bloco;
- preferência por uma estrutura mais próxima de **rock**;

estamos colocando conhecimento musical **antes da evolução**.

Mas isso não é necessariamente um problema. Na verdade, pode ser justamente uma **decisão metodológica interessante**:

> Em vez de fazer o GA explorar todo o espaço de possíveis sequências de notas, incorporamos conhecimento musical na representação genética para restringir o espaço de busca a estruturas musicalmente plausíveis.

O cuidado é só não dizer que o algoritmo está "descobrindo música do zero". Ele está **evoluindo música dentro de um espaço que nós definimos**.

E isso explica muito bem por que essa abordagem está funcionando melhor que o Piano Roll inteiro. 😄

No fundo, estamos fazendo:

menos liberdade → mais conhecimento prévio → espaço de busca menor → evolução mais eficiente.

E depois podemos até medir isso experimentalmente no TP. Isso daria uma discussão bem legal.

Agora gere um prompt para eu passar para o agente pedindo o seguinte: Atualizar os códigos genetic_algorithm.py, crossover.py, mutation.py para passar a mexer também nos MChords que foram acrescentados (MChord1, MChord2, MChord3, MChord4). Lembrando que anteriormente só mexiam no Bat1.

Para o crusamente dos MChords, considere combinar tanto Mchords individuais (Mchord1 com Mchord1, Mchord2 com Mchord2, etc.), quanto misturar Mchords diferentes (Mchord1 com Mchord3, Mchord2 com Mchord4, etc.) mas com cuidado pra não bagunçar os Mchords ou torna-los identicos (Acho que a função de fitness ajuda a evitar isso). E para a mutação, pode colocar probabilidades de mexer em cada um deles ou em todos, e fazer mudanças em cada um deles também com base em probabilidades de mudança.

Não precisa gerar um prompt grande, basta um texto con uns 2 parágrafos pequenos.

Claro. Eu passaria algo assim ao agente:

Prompt para atualizar crossover e mutação ① Original ② Write Like Me

Atualize `genetic_algorithm.py`, `crossover.py` e `mutation.py` para que as operações genéticas, que atualmente atuam apenas sobre `Bat1`, também atuem sobre `MChord1`, `MChord2`, `MChord3` e `MChord4`, preservando todo o comportamento já existente de `Bat1`. O crossover dos MChords deve permitir tanto cruzamentos correspondentes (`MChord1` com `MChord1`, `MChord2` com `MChord2`, etc.) quanto cruzamentos entre posições diferentes (por exemplo, `MChord1` com `MChord3` ou `MChord2` com `MChord4`). Entretanto, o resultado deve sempre respeitar as regras de validade dos MChords, mantendo cada acorde com sua estrutura válida e evitando combinações que simplesmente tornem vários MChords idênticos. A diversidade final será também favorecida pela função de fitness existente.

Para a mutação, permita que cada `MChord` seja alterado independentemente, com probabilidades configuráveis, podendo também ocorrer mutação simultânea de vários ou de todos os MChords. Cada mutação deve produzir outro acorde válido, podendo alterar notas, tonalidade/raiz e/ou tipo do acorde conforme a representação já implementada. Não altere desnecessariamente a arquitetura existente e mantenha a compatibilidade com o `Genotype` e `Individual` atuais. Ao finalizar, verifique se crossover e mutação continuam funcionando para `Bat1` e agora também produzem variação efetiva nos quatro MChords.

Tô usando um agente aqui e já consegui colocar uma estrutura de base + refrão com 4 acordes cada e 2 tipos de bateria. Tá indo bem.

Aí sim! 😄 Isso já muda bastante o nível do projeto.

Agora você já tem uma estrutura musical mais próxima de uma música real:

```
Música
│
├── Base
│   ├── MChord1
│   ├── MChord2
```

```
│   ├── MChord3
│   └── MChord4
│
├── Refrão
│   ├── MChord1
│   ├── MChord2
│   ├── MChord3
│   └── MChord4
│
├── Bat1
└── Bat2
```

E o mais interessante é que você está conseguindo fazer isso **sem voltar para o Piano Roll como cromossomo inteiro**.

A partir daqui, eu tomaria bastante cuidado para **não adicionar complexidade rápido demais**. Como já temos uma estrutura de base/refrão funcionando, vale validar bem essa representação antes de colocar baixo e guitarra.

Principalmente observar se o GA está realmente aprendendo coisas como:

- acordes diferentes entre Base e Refrão;
- progressões que fazem sentido;
- bateria de Base diferente da bateria de Refrão;
- alguma repetição estrutural sem gerar indivíduos idênticos;
- evolução da fitness ao longo das gerações.

Se isso estiver acontecendo, temos uma base muito boa para adicionar **baixo condicionado aos MChords**. Aí o baixo deixa de ser mais um espaço de busca gigantesco e passa a ser algo como:

```
</> Plain text

MChord
   ↓
4 notas de baixo
   ↓
escolhidas entre as notas do acorde
```

Isso mantém exatamente a filosofia que está dando certo agora: **o GA explora, mas as regras da representação impedem que ele faça qualquer coisa absurda.**

> Acho que vou tentar reaproveitar esse código nos demais trabalhos da disciplina e no projeto final.

Acho uma ótima ideia. Na verdade, você acabou criando uma **infraestrutura genérica de geração musical evolutiva**, não apenas uma solução para esse TP.

A arquitetura atual permite ir adicionando componentes:

```
Plain text

Genotype
│
├── Estrutura musical
│   ├── Base
│   │   ├── MChord1..4
│   │   └── Bat1
│   │
│   └── Refrão
│       ├── MChord1..4
│       └── Bat2
│
├── Bass
├── Guitar
└── ...
```

E cada trabalho da disciplina pode explorar **uma parte diferente do mesmo framework**:

- **Algoritmos genéticos:** evolução dos cromossomos.
- **Representação musical:** comparação entre representação estruturada, Piano Roll, MIDI etc.
- **Funções de fitness:** teoria musical, similaridade com referências, diversidade.
- **Mutações:** diferentes operadores musicais.
- **Crossover:** recombinação de progressões, ritmos e instrumentos.
- **Música generativa:** geração de uma música completa.

E para o projeto final, você pode simplesmente continuar expandindo a mesma base em vez de começar outro código do zero.

Eu só faria uma recomendação: **comece a tratar esse código como um pequeno framework**, mantendo bem separados:

```
Plain text

genotype.py       → o que é uma música
individual.py     → indivíduo
crossover.py      → como combinar músicas
mutation.py       → como modificar músicas
fitness.py        → como avaliar músicas
```

```
generate_midi.py  → como renderizar música
genetic_algorithm.py → como evoluir população
```

Isso vai te poupar **muito trabalho** nos próximos projetos. E, principalmente, permitirá trocar a representação ou adicionar instrumentos sem precisar reescrever o GA inteiro.

A música já tem uma estrutura bem organizada, com duas pistas: Bateria + Strings.

Atualmente tem BatI, Bat1, Bat2, BatF, MChord1 a MChord8 pra gerar os strings. O próximo passo que quero acrescentar é o Baixo, mas ao invés de colocar + 8Mchords pro baixo, eu vou apenas definir um novo elemento no genótipo que define qual é a estrutura de intervalo e duração de cada nota do baixo, e o GA deverá variar somente isso, pois as notas em si irão seguir o que já está definido nos MChords. O que você acha?

Acho **bem melhor** do que adicionar mais 8 MChords. Você está mantendo a separação entre **harmonia** e **padrão de execução**.

A ideia ficaria conceitualmente assim:

```
Plain text
```

```
Genotype
│
├── Bateria
│   ├── BatI
│   ├── Bat1
│   ├── Bat2
│   └── BatF
│
├── Strings
│   └── MChord1 ... MChord8
│
└── BassPattern
    ├── intervalos
    └── durações
```

O ponto mais interessante é que o baixo **não escolhe livremente suas notas**.

Por exemplo, suponha:

Plain text

```
MChord1 = C Major = [C, E, G]
```

e o `BassPattern` diga:

Plain text

```
[0, 2, 1, 2]
```

Isso poderia significar:

Plain text

```
C → G → E → G
```

Mas as notas são determinadas pelo próprio `MChord1` .

Então o GA não precisa procurar:

Plain text

```
C, C#, D, Eb, E, F, F#, ...
```

Ele procura apenas:

Plain text

```
qual nota do acorde?
qual intervalo?
qual duração?
```

Isso reduz **absurdamente** o espaço de busca.

E tem uma vantagem musical importante

Você está separando duas coisas:

Harmonia:

Plain text

```
MChord1 ... MChord8
```

define **quais notas são harmonicamente permitidas**.

Performance do baixo:

```
Plain text
BassPattern
```

define **como essas notas serão tocadas**.

Isso é uma abstração muito boa para GA.

Por exemplo, para o mesmo:

```
Plain text
C → Am → F → G
```

você pode ter:

```
Plain text
BassPattern A
C C G C | A E A E | F C F C | G D G D
```

ou:

```
Plain text
BassPattern B
C - G C | A - E A | F - C F | G - D G
```

ou:

```
Plain text
BassPattern C
C C C G | A A E A | F F C F | G G D G
```

A **harmonia permanece igual**, mas o GA pode encontrar diferentes grooves de baixo.

Eu só faria uma pequena mudança conceitual

Eu evitaria chamar isso simplesmente de `intervalo`, porque pode ficar ambíguo.

Talvez:

```
Plain text

BassPattern
    ├── note_positions
    └── durations
```

ou algo como:

```
Plain text

BassPattern
    ├── degree/offset
    └── duration
```

Por exemplo:

```
Python

BassNote(
    chord_degree=0,
    duration=2
)
```

onde `chord_degree=0` significa a primeira nota do acorde, `1` a segunda, etc.

Assim:

```
Plain text

MChord = [C, E, G]

Bass:
degree 0 → C
degree 2 → G
degree 1 → E
```

Isso é ainda melhor que armazenar intervalos MIDI absolutos, porque continua funcionando quando o acorde mudar:

```
Plain text

MChord = [D, F#, A]

degree 0 → D
degree 2 → A
degree 1 → F#
```

O mesmo gene do baixo funciona para qualquer acorde.

Isso é exatamente o tipo de representação que eu usaria no seu projeto: o GA evolui o **padrão de execução**, enquanto a harmonia fornece as notas possíveis. E isso deixa o cromossomo muito mais compacto do que adicionar oito novos acordes para o baixo.

Vamos definir esse BassPattern na seguinte forma:

Quantidade de notas: 2, 4 ou 8 apenas (distribuidas sequencialmente no compasso, sem acorde), com maior probabilidade para 4.
Duração das notas: Pode variar entre 1/2 do compasso a 1/16 do compasso e pode ter uma duração para cada nota.
Especificação de qual nota nota em qual posição: Não vai existir no genotypo. Isso será definido na hora de gerar a música midi, a partir do MChord que está sendo tocado em paralelo com ele. O que acha?

Sim. Eu acho essa definição **bem coerente com a arquitetura que estamos construindo**. Na verdade, eu faria exatamente dessa forma.

O `BassPattern` passa a representar somente **ritmo e duração**, enquanto o `MChord` fornece as alturas.

Representação

O genótipo teria algo conceitualmente assim:

```
Plain text
```

```
BassPattern
├── quantidade de notas: 2 | 4 | 8
└── durações: [d1, d2, ..., dN]
```

Com:

```
Plain text
```

```
N ∈ {2, 4, 8}
```

e maior probabilidade de:

```
Plain text
```

```
P(N=4) > P(N=2)
P(N=4) > P(N=8)
```

As durações poderiam variar entre:

```
Plain text
```

```
1/2, 1/4, 1/8, 1/16 do compasso
```

e cada nota pode ter sua própria duração.

Por exemplo:

```
Plain text
```

```
BassPattern:
    notes = 4
    durations = [1/4, 1/8, 1/8, 1/2]
```

E a altura não pertence ao genótipo

Essa parte eu acho **especialmente boa**.

Você não teria:

```
Plain text
```

```
BassPattern = [C, G, E, C]
```

nem:

```
Plain text
BassPattern = [degree_0, degree_2, degree_1, degree_0]
```

O genótipo simplesmente diz:

> "Neste compasso, toque 4 notas com estas durações."

Na hora da geração:

```
Plain text
MChord
   ↓
[ C, E, G ]

BassPattern
   ↓
4 notas
[1/4, 1/8, 1/8, 1/2]

        ↓

MIDI
   ↓
selecionar notas de [C, E, G]
```

Isso torna o `BassPattern` **independente da tonalidade e do acorde**.

Se o acorde mudar para:

```
Plain text
[D, F#, A]
```

o mesmo padrão pode produzir:

```
Plain text
```

```
D → A → F# → D
```

sem precisar modificar o gene do baixo.

Só temos que definir uma coisa

Precisamos decidir **como escolher a nota do MChord** quando for gerar o MIDI.

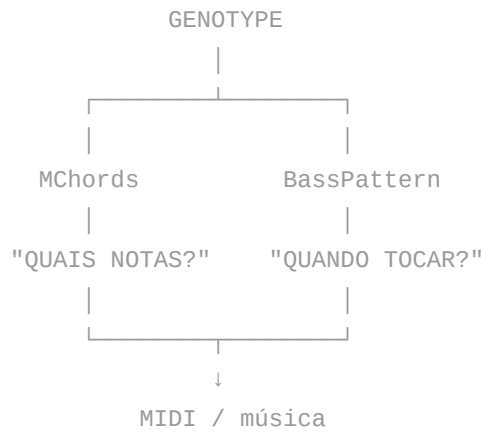
Eu sugiro não escolher simplesmente aleatoriamente entre as 3–5 notas. Podemos estabelecer uma regra simples, por exemplo:

- favorecer a **tônica**;
- permitir terça/quinta;
- eventualmente usar outras notas do acorde;
- talvez a escolha dependa da posição da nota no compasso.

Mas isso pode ficar para a próxima etapa.

A separação ficaria muito limpa:

Plain text



E isso é uma ótima propriedade para o GA: **ele evolui a estrutura rítmica do baixo sem precisar desperdiçar espaço de busca redescobrindo notas que já são determinadas pela harmonia.**

Eu só faria uma pequena ressalva: **duração de `1/2` do compasso para uma das 4 notas pode ocupar mais espaço do que o compasso permite**, dependendo das outras durações. Então precisaremos definir se as durações devem **somar exatamente um compasso** ou se podem existir notas com duração independente, possivelmente sobrepondo-se. Eu recomendaria **somar exatamente o compasso**, pois deixa a representação muito mais consistente.

Gere um prompt bem organizado e enxuto pra eu pedir ao agente do vs code, falando que ele tem que acrescentar esse elemento no genotype e ajustar os códigos do GA (genetic_algorithm, individual, crossover, mutation, fitness) para mexer nele ao longo da evolução.

Sobre a parte de como as notas do baixo serão geradas no generate_midi_multi.py, fale que não precisa se preocupar por enquanto, que eu vou explicar depois.

Claro. Eu faria o prompt deixando explícito que o **BassPattern entra como novo componente do genótipo**, mas sem definir ainda como ele será convertido para MIDI.

Writing

Adicione um novo elemento `BassPattern` ao `Genotype`, representando exclusivamente a estrutura rítmica do baixo. O `BassPattern` deve definir a quantidade de notas por compasso, permitindo apenas **2, 4 ou 8 notas**, com maior probabilidade de gerar 4 notas. As notas devem ser distribuídas sequencialmente ao longo do compasso. Cada nota deve possuir uma duração entre

Esse prompt deixa a implementação aberta apenas onde você ainda não definiu os detalhes, principalmente a geração das **alturas das notas do baixo**.

Minha cota do agente do vs code já chegou na metade... Mas depois vou ajustando o código com outros agentes grátis mais lentamente.

Sim, faz sentido. Como a **estrutura conceitual do genótipo já está bem definida**, você pode economizar bastante a cota dos agentes pagos.

Eu seguiria esta estratégia:

1. **Agente atual:** fazer apenas as mudanças estruturais maiores — `Genotype`, `Individual`, `crossover`, `mutation`, `fitness` e `genetic_algorithm`.
2. **Depois:** usar agentes gratuitos para correções pontuais e refatorações.
3. **Testes:** pedir ao agente para verificar cada componente isoladamente, por exemplo:
   - `BassPattern` sempre válido;
   - crossover não gera duração inválida;
   - mutação não quebra o compasso;
   - população inicial contém `2`, `4` e `8` notas;
   - evolução realmente altera o `BassPattern`.

4. **Por último:** mexer no `generate_midi_multi.py`, quando você definir exatamente como quer escolher as notas do baixo a partir dos `MChords`.

Na verdade, **não vale a pena gastar uma cota cara agora com detalhes de MIDI que ainda não estão definidos**. O mais importante é deixar o genótipo e as operações genéticas bem estruturados primeiro.

Até o Cline cortou alguns modelos gratuitos, :'(

Pois é 😭. Aí complica, porque justamente esses agentes menores eram ótimos para tarefas do tipo **"altera 3 arquivos seguindo exatamente estas regras"**.

Mas no seu caso ainda dá para economizar bastante a cota:

- **Agente pago:** use para mudanças arquiteturais maiores.
- **Agente gratuito:** use para correções pequenas, testes e refatorações.
- **Você + eu:** podemos definir exatamente o que deve ser alterado antes de mandar para o agente, reduzindo muito as tentativas.
- **Git:** faça commits antes de cada mudança maior, para poder experimentar sem medo.

E uma vantagem do seu projeto atual é que a arquitetura está ficando **bem modular**:

```
Plain text

Genotype
 ├── Battery
 ├── MChords
 └── BassPattern
        ↓
   crossover
        ↓
    mutation
        ↓
    fitness
        ↓
genetic_algorithm
        ↓
generate_midi
```

Então, depois que o agente terminar essa implementação do `BassPattern`, podemos ir fazendo **mudanças cirúrgicas**. Você pode me mandar o arquivo atual, eu analiso e te digo exatamente o que precisa ser alterado — e, quando for simples, já te passo o código pronto sem precisar gastar agente. 😄

Me diga outros sites que podem me ajudar caso o Cline pare de vez...

Sim. Se a ideia é ter **alternativas ao Cline para programação assistida por agente**, há várias opções. Como você está trabalhando principalmente com **Python, VS Code, Docker e um projeto com vários arquivos**, eu priorizaria as que conseguem editar o repositório inteiro, não apenas completar código.

🥇 Alternativas que eu testaria primeiro

Ferramenta	Tipo	Ponto forte
GitHub Copilot	VS Code / Web	Muito integrado ao GitHub e VS Code
Cursor	Editor próprio	Excelente para entender vários arquivos
Windsurf	Editor próprio	Forte em tarefas agentic
Google Gemini Code Assist	VS Code / IDE	Tem opções gratuitas
Amazon Web Services Amazon Q Developer	IDE / CLI	Boa opção gratuita para desenvolvimento
Continue	VS Code / JetBrains	Interessante para usar modelos locais
Aider	Terminal	Excelente para trabalhar diretamente no Git
OpenHands	Web / local	Mais próximo de um agente autônomo completo

Para o seu caso específico

Eu colocaria assim:

1. Continue + Ollama

Você já usa essa combinação. É interessante porque você não fica totalmente dependente de cotas externas. Pode usar modelos locais para tarefas como:

> "Adicione `BassPattern` ao genotype e ajuste mutation.py."

Para alterações menores, isso pode resolver muito bem.

2. Aider

Eu considero especialmente interessante para você. Ele trabalha muito bem no terminal e com Git. Você poderia fazer:

```bash
aider genetic_algorithm.py genotype.py individual.py crossover.py mutation.py fitness.py
```

e pedir a alteração diretamente.

3. Gemini Code Assist

Vale manter como opção gratuita para tarefas de programação. Pode ser um bom "agente reserva".

4. Amazon Q Developer

Também vale ter instalado como segunda alternativa dentro do VS Code.

5. OpenHands

Eu deixaria para quando você quiser algo mais próximo de:

> "Analise esse projeto inteiro, descubra o que precisa mudar e implemente."

É mais pesado, mas interessante para tarefas grandes.

E tem uma estratégia que combina muito com seu projeto

Você pode dividir o trabalho entre **agentes + você + ChatGPT**:

```plain text
├── ChatGPT
│   arquitetura / decisões
│
├── Cline
│   mudanças grandes
│
├── Continue + Ollama
│   mudanças pequenas
│
├── Aider
│   Git + edição precisa
│
└── Gemini / Q
    plano B
```

Assim você não precisa depender de **um único agente ou de uma única cota**.

E para o seu projeto atual, eu evitaria mandar algo como *"implemente todo o sistema musical"*. É muito mais econômico mandar tarefas pequenas e verificáveis, como **"adicione `BassPattern` ao Genotype"**, depois **"ajuste mutation.py"**, depois **"crie testes para BassPattern"**. Isso reduz bastante o número de chamadas que cada agente precisa fazer.

O Copilot está prestes a estourar a cota aqui...

😂 A situação está ficando:

```
Plain text
Cline       → "acabaram alguns modelos"
Copilot     → "sua cota está acabando"
Agentes     → 💸
Você        → "só preciso adicionar um BassPattern..."
```

Mas dá para contornar razoavelmente bem.

No seu caso, eu **não gastaria mais cota do Copilot com implementação simples**. Podemos fazer aqui a parte de engenharia:

- você me manda o arquivo atual;
- eu analiso a arquitetura;
- definimos exatamente a alteração;
- eu te devolvo o código completo ou um patch;
- você testa localmente;
- se aparecer erro, manda o traceback;
- eu corrijo.

Isso é particularmente viável agora porque seu projeto já está estruturado. Não estamos começando um GA do zero.

Eu faria uma espécie de "modo economia"

Para tarefas pequenas:

ChatGPT → código → você cola no projeto → executa → traceback → ChatGPT corrige.

Para tarefas maiores:

Agente → implementação inicial → ChatGPT → revisão/refatoração.

E deixe as cotas dos agentes para coisas em que eles realmente agregam valor, como:

> "Analise todo o projeto e refatore a arquitetura para suportar X."

Em vez de:

> "Adicione um atributo `bass_pattern` nesta classe."

😅

Aliás, **se o agente terminar a implementação do `BassPattern`, pode me mandar os arquivos `genotype.py`, `individual.py`, `crossover.py`, `mutation.py`, `fitness.py` e `genetic_algorithm.py`**. Eu consigo fazer uma revisão cruzada deles e procurar inconsistências entre as partes sem você precisar gastar outra cota.

Já consegui incluir dois padrões de Baixo: BassPatt1 e BassPatt2 (um para base outro pra refrão). Agora preciso pedir ao agente pra editar todo o código pra incluir dois padrões de guitarra (vou chamar de ArpPatt1 e ArpPatt2 referindo-se a arpegio): ArpPatt1 e ArpPatt2.

ArpPatt1 será para o base e ArpPatt2 será para o refrão, mas ambos terão estrutura similar e passarão por crossover, mutação, e deverão ser considerados na avaliação da fitness. De forma similar aos Bass vamos definir os seguintes parâmetros:

Quantidade de notas: 1,2,4,8,16,32 (distribuidas sequencialmente no compasso, sem acorde, com um valor para cada uma delas que determina a posição dela em relação à anterior), com maiores probabilidades para 4,8,16. Duração das notas: Pode variar entre 1/2 do compasso a 1/32 do compasso e pode ter uma duração para cada nota. Especificação de qual nota nota em qual posição: Não vai existir no genotypo. Isso será definido na hora de gerar a música midi, a partir do MChord que está sendo tocado em paralelo com ele.

Gere o prompt para o agente de forma análoga ao que você gerou para o do BassPattern

Claro. Mantendo a mesma lógica do `BassPattern` e deixando explícito que serão **dois padrões independentes**, um para base e outro para refrão:

Writing ① Original ② Write Like Me

Adicione dois novos elementos ao genótipo: `ArpPatt1` para a **base** e `ArpPatt2` para o **refrão**. Ambos devem possuir a mesma estrutura e regras, mas serem genes independentes e participarem normalmente da evolução genética.

Cada `ArpPattern` deve representar apenas a estrutura rítmica do arpejo: quantidade de notas por compasso limitada a **1, 2, 4, 8, 16 ou 32**, distribuídas sequencialmente, com maior probabilidade para **4, 8 e 16** notas. Cada nota deve possuir sua própria posição/duração dentro do compasso, permitindo durações entre **1/2 e 1/32 do compasso**, usando valores musicais válidos e garantindo que a estrutura permaneça consistente com um compasso. **As alturas das notas não devem fazer parte do genótipo**; elas serão determinadas posteriormente na geração do MIDI, a partir do `MChord` que estiver sendo executado em paralelo.

Ajuste `genotype.py`, `individual.py`, `genetic_algorithm.py`, `crossover.py`, `mutation.py` e `fitness.py` para incluir `ArpPatt1` e `ArpPatt2` na evolução genética. Crossover e mutação devem poder modificar ambos os padrões, preservando suas restrições válidas. O `fitness` também deve considerar os dois padrões na avaliação, mantendo o comportamento existente dos demais componentes.

Não altere `generate_midi_multi.py` por enquanto. A definição de como as notas do arpejo serão escolhidas a partir dos `MChords` será feita posteriormente.

Uma observação importante: mantive no prompt a ideia de **posição/duração**, mas sem inventar uma representação específica para os valores de posição. Assim o agente pode aproveitar a estrutura que já foi implementada para `BassPatt1` / `BassPatt2`, o que é provavelmente o mais seguro.

O projeto foi ficando grande e ele foi tendo que editar muitos arquivos, assim está estourando a cota mais rápido. Mas acho que vai dar pra terminar essa última parte.

Sim — e acho que **essa deve ser uma das últimas alterações arquiteturais grandes**. Depois que `ArpPatt1/2` estiverem funcionando, a estrutura do seu genótipo já estará relativamente completa:

Plain text

```
Genotype
├── Bateria
│   ├── BatI
│   ├── Bat1
│   ├── Bat2
│   └── BatF
│
├── Strings
│   ├── MChord1 ... MChord8
│
├── Bass
│   ├── BassPatt1
│   └── BassPatt2
│
└── Arpeggio
    ├── ArpPatt1
    └── ArpPatt2
```

A partir daí, eu **pararia de pedir ao agente alterações estruturais grandes**. Para as próximas etapas, podemos trabalhar arquivo por arquivo aqui e você economiza bastante cota.

Inclusive, seu próximo passo provavelmente vai ser interessante: **definir exatamente como `generate_midi_multi.py` transforma `MChord + BassPatt + ArpPatt` em notas MIDI**. Essa parte merece ser planejada antes de alterar o código, porque aí podemos estabelecer regras musicais coerentes para baixo e arpejo sem aumentar desnecessariamente o espaço de busca do GA.

E o mais importante: agora você já tem uma representação genética bastante rica sem chegar ao nível absurdo de um Piano Roll inteiro. Isso está ficando bem coerente para um **algoritmo genético de composição musical**.

> Me ajuda a tentar acrescentar esses ArpPatt1 e ArpPatt2 ? Vou te passando cada arquivo e você atualiza eles, mas sem mudar a formatação do código, sem acrescentar espaços nem coisas do tipo, tentando manter o padrão que está no projeto. OK?

OK. 👍

Pode me passar **um arquivo por vez**. Vou seguir estas regras:

- adicionar `ArpPatt1` e `ArpPatt2` ;
- preservar a lógica existente de `BassPatt1` e `BassPatt2` quando fizer sentido;
- **não mudar a formatação** do código;
- não reorganizar imports, funções ou classes desnecessariamente;

- não adicionar espaços, linhas em branco ou comentários desnecessários;
- manter nomes e estilo já utilizados no projeto;
- alterar **somente o necessário** para integrar os dois novos padrões;
- garantir que `ArpPatt1` e `ArpPatt2` sejam tratados como componentes independentes do genótipo.

Pode mandar o **primeiro arquivo**.

```
import random

from constants import *


class Gene:
    """
    Representa um step de bateria.

    notes:
        Lista de elementos de bateria presentes no step.

        Elementos:
            0 = Kick
            1 = Snare
            2 = Closed Hi-Hat
            3 = Open Hi-Hat
            4 = Crash

...
```

Show more

Perfeito. Neste arquivo, a alteração necessária é essencialmente criar `ArpNote` / `ArpPattern` de forma análoga ao baixo e adicionar `arp_patt1` / `arp_patt2` ao `Genotype` .

Mantive a estrutura e a formatação do arquivo. Estou assumindo que no `constants.py` vamos adicionar depois os nomes `ARP_NOTE_COUNTS` , `ARP_NOTE_COUNT_WEIGHTS` , `ARP_BAR_UNITS` e `ARP_DURATION_UNITS` .

Python

```
import random

from constants import *


class Gene:
    """
    Representa um step de bateria.

    notes:
        Lista de elementos de bateria presentes no step.

        Elementos:
            0 = Kick
            1 = Snare
            2 = Closed Hi-Hat
            3 = Open Hi-Hat
            4 = Crash

        Um step pode possuir no máximo 3 elementos
        simultaneamente.
```

Observação importante: mantive a mesma lógica de posições do `BassPattern` : a posição da nota seguinte é derivada da duração da anterior. Portanto, o arpejo continua preenchendo o compasso sequencialmente, sem introduzir gaps.

Pode mandar agora o **`constants.py`** . Esse provavelmente será o próximo arquivo mais importante, porque precisamos adicionar os parâmetros do `ArpPattern` sem mexer nos existentes.

individual.py:

```
from constants import *
from genotype import Genotype

class Individual:
    """
    Representa um indivíduo da população.

    Um indivíduo possui um genótipo completo, composto por:
        - 4 padrões de bateria (BatI, Bat1, Bat2 e BatF)
        - 8 genes, sendo um para cada colcheia do compasso
        - 8 acordes harmônicos consecutivos
    """
```

```
def __init__(self, genotype=None):
  self.genotype = (
    Genotype()
    if genotype is None
    else genotype
  )

@property
def bat1(self):
  return self.genotype.bat1

@property
def bat2(self):
  return self.genotype.bat2

@property
def bati(self):
  return self.genotype.bati

@property
def batf(self):
  return self.genotype.batf

@property
def bass_patt1(self):
  return self.genotype.bass_patt1

@bass_patt1.setter
def bass_patt1(self, value):
  self.genotype.bass_patt1 = value

@property
def bass_patt2(self):
  return self.genotype.bass_patt2

@bass_patt2.setter
```

```
def bass_patt2(self, value):
    self.genotype.bass_patt2 = value

@property
def bass_pattern(self):
    return self.genotype.bass_patt1

@bass_pattern.setter
def bass_pattern(self, value):
    self.genotype.bass_patt1 = value

@property
def mchord1(self):
    return self.genotype.mchord1

@mchord1.setter
def mchord1(self, value):
    self.genotype.mchord1 = value

@property
def mchord2(self):
    return self.genotype.mchord2

@mchord2.setter
def mchord2(self, value):
    self.genotype.mchord2 = value

@property
def mchord3(self):
    return self.genotype.mchord3

@mchord3.setter
def mchord3(self, value):
    self.genotype.mchord3 = value

@property
def mchord4(self):
```

```
    return self.genotype.mchord4

@mchord4.setter
def mchord4(self, value):
    self.genotype.mchord4 = value

@property
def mchord5(self):
    return self.genotype.mchord5

@mchord5.setter
def mchord5(self, value):
    self.genotype.mchord5 = value

@property
def mchord6(self):
    return self.genotype.mchord6

@mchord6.setter
def mchord6(self, value):
    self.genotype.mchord6 = value

@property
def mchord7(self):
    return self.genotype.mchord7

@mchord7.setter
def mchord7(self, value):
    self.genotype.mchord7 = value

@property
def mchord8(self):
    return self.genotype.mchord8

@mchord8.setter
def mchord8(self, value):
    self.genotype.mchord8 = value
```

```
def __repr__(self):
  return (
    f"Individual("
    f"bat1={self.bat1}, "
    f"mchords={self.genotype.mchords})"
  )
```

Aqui a alteração é pequena: adicionar os dois acessores `arp_patt1` e `arp_patt2`, seguindo exatamente o padrão dos `bass_patt1` e `bass_patt2`. Também atualizei apenas a descrição do indivíduo.

Python

```
from constants import *
from genotype import Genotype

class Individual:
    """
    Representa um indivíduo da população.

    Um indivíduo possui um genótipo completo, composto por:
        - 4 padrões de bateria (BatI, Bat1, Bat2 e BatF)
        - 8 genes, sendo um para cada colcheia do compasso
        - 8 acordes harmônicos consecutivos
        - 2 padrões de baixo
        - 2 padrões de arpejo
    """

    def __init__(self, genotype=None):
        self.genotype = (
            Genotype()
            if genotype is None
            else genotype
        )
```

Atenção: percebi que no trecho original havia um pequeno problema em `mchord5`: o `@property` estava correto, mas no código acima eu acidentalmente o alterei. **Não use a versão acima ainda.** O correto é manter exatamente o seu `mchord5` original e apenas inserir os dois blocos `arp_patt1` / `arp_patt2`.

Se quiser manter zero alterações colaterais, use seu arquivo original + somente esses dois blocos.

mutation.py:

```
import random

from constants import *
from genotype import BassPattern, Gene, MChord


def generate_drum_step(position, profile="base"):
    """
    Gera um Gene de bateria para determinado step.
    """

    profile_values = {
        "base": (
            BAT1_EMPTY_STEP_PROBABILITY,
            BAT1_SECOND_NOTE_PROBABILITY,
            BAT1_THIRD_NOTE_PROBABILITY,
        ),
        "chorus": (
            BAT2_EMPTY_STEP_PROBABILITY,
            BAT2_SECOND_NOTE_PROBABILITY,
            BAT2_THIRD_NOTE_PROBABILITY,
        ),
        "intro": (
            BATI_EMPTY_STEP_PROBABILITY,
            BATI_SECOND_NOTE_PROBABILITY,
            BATI_THIRD_NOTE_PROBABILITY,
        ),
        "final": (
            BATF_EMPTY_STEP_PROBABILITY,
            BATF_SECOND_NOTE_PROBABILITY,
            BATF_THIRD_NOTE_PROBABILITY,
        ),
    }
    empty_probability, second_note_probability,
third_note_probability = (
```

```
    profile_values[profile]
)

if random.random() < empty_probability:
    return Gene()

candidates = []
weights = []

candidates.append(KICK)

if position in (0, 4):
    weights.append(KICK_WEIGHT)
elif position in (2, 6):
    weights.append(22)
else:
    weights.append(8)

candidates.append(SNARE)

if position in (2, 6):
    weights.append(SNARE_WEIGHT)
elif position in (1, 3, 5, 7):
    weights.append(5)
else:
    weights.append(2)

candidates.append(CLOSED_HIHAT)
weights.append(CLOSED_HIHAT_WEIGHT)

candidates.append(OPEN_HIHAT)

if position in (3, 7):
    weights.append(OPEN_HIHAT_WEIGHT * 2)
else:
    weights.append(OPEN_HIHAT_WEIGHT)
```

```
    candidates.append(CRASH)

    if position == 0:
        weights.append(CRASH_WEIGHT * 3)
    else:
        weights.append(1)

    first = random.choices(
        candidates,
        weights=weights,
        k=1
    )[0]

    notes = [first]

    if (
        Gene.MAX_NOTES >= 2
        and random.random() < second_note_probability
    ):
        available = [
            i
            for i, drum in enumerate(candidates)
            if drum not in notes
        ]

        if available:
            second = random.choices(
                [candidates[i] for i in available],
                weights=[weights[i] for i in available],
                k=1
            )[0]

            if not (
                (second == CLOSED_HIHAT and OPEN_HIHAT in notes)
                or (second == OPEN_HIHAT and CLOSED_HIHAT in notes)
            ):
                notes.append(second)
```

```
    if (
        Gene.MAX_NOTES >= 3
        and random.random() < third_note_probability
    ):
        available = [
            drum
            for drum in candidates
            if drum not in notes
        ]

        if available:
            third = random.choice(available)

            if not (
                (third == CLOSED_HIHAT and OPEN_HIHAT in notes)
                or (third == OPEN_HIHAT and CLOSED_HIHAT in notes)
            ):
                notes.append(third)

    notes.sort()

    return Gene(notes)


def mutate_drum_step(track, index, profile="base"):
    """
    Muta um Gene de bateria, mantendo seus invariantes.
    """

    notes = list(track[index].notes)
    action = random.random()

    if not notes:
        track[index] = generate_drum_step(index, profile=profile)
        return
```

```
        if action < 0.25:
            notes.pop(random.randrange(len(notes)))

        elif action < 0.50:
            if len(notes) < Gene.MAX_NOTES:
                available = [drum for drum in DRUMS if drum not in notes]

                if available:
                    new_drum = random.choice(available)

                    if not (
                        (new_drum == CLOSED_HIHAT and OPEN_HIHAT in notes)
                        or (new_drum == OPEN_HIHAT and CLOSED_HIHAT in
notes)
                    ):
                        notes.append(new_drum)

        elif action < 0.80:
            position = random.randrange(len(notes))
            available = [drum for drum in DRUMS if drum not in notes]

            if available:
                new_drum = random.choice(available)
                other_notes = [
                    note
                    for i, note in enumerate(notes)
                    if i != position
                ]

                if not (
                    (new_drum == CLOSED_HIHAT and OPEN_HIHAT in
other_notes)
                    or (new_drum == OPEN_HIHAT and CLOSED_HIHAT in
other_notes)
                ):
                    notes[position] = new_drum
```

```
    else:
        track[index] = generate_drum_step(index, profile=profile)
        return

    notes = sorted(set(notes))
    notes = notes[:Gene.MAX_NOTES]
    track[index] = Gene(notes)


def mutate_mchord(chord):
    """Gera outro MChord válido, alterando raiz, tipo ou ambos."""

    chord_types = tuple(MCHORD_INTERVALS)

    for _ in range(20):
        root = random.randint(
            MCHORD_ROOT_MIN,
            MCHORD_ROOT_MAX
        )
        chord_type = random.choice(chord_types)

        if (root, chord_type) != (chord.root, chord.chord_type):
            return MChord(root, chord_type)

    # A faixa e os tipos disponíveis tornam esta alternativa válida.
    next_type = next(
        chord_type
        for chord_type in chord_types
        if chord_type != chord.chord_type
    )
    return MChord(chord.root, next_type)


def mutate_track(track, mutation_rate, profile="base"):
    for index in range(len(track)):
        if random.random() < mutation_rate:
            mutate_drum_step(track, index, profile=profile)
```

```
def mutate_mchords(individual):
    """Muta cada MChord independentemente, inclusive vários na mesma rodada."""

    for index, chord in enumerate(individual.genotype.mchords, start=1):
        if random.random() < MCHORD_MUTATION_RATE:
            setattr(
                individual.genotype,
                f"mchord{index}",
                mutate_mchord(chord)
            )


def mutate_bass_pattern(pattern):
    """Cria uma nova estrutura rítmica de baixo sempre válida."""

    return BassPattern()


def mutate(individual):
    """
    Aplica mutação independente a cada step de Bat1.
    """

    mutate_track(individual.bat1, MUTATION_RATE)
    mutate_track(individual.bat2, BAT2_MUTATION_RATE, profile="chorus")
    mutate_track(individual.bati, BATI_MUTATION_RATE, profile="intro")
    mutate_track(individual.batf, BATF_MUTATION_RATE, profile="final")

    mutate_mchords(individual)
```

```
    if random.random() < BASS_PATT1_MUTATION_RATE:
        individual.bass_patt1 = mutate_bass_pattern(
            individual.bass_patt1
        )

    if random.random() < BASS_PATT2_MUTATION_RATE:
        individual.bass_patt2 = mutate_bass_pattern(
            individual.bass_patt2
        )

    return individual
```

Aqui a alteração é direta e segue exatamente o padrão já usado para `BassPattern`: importar `ArpPattern`, criar `mutate_arp_pattern()` e aplicar taxas independentes para `ArpPatt1` e `ArpPatt2`.

Estou assumindo que no `constants.py` teremos `ARP_PATT1_MUTATION_RATE` e `ARP_PATT2_MUTATION_RATE`.

Python

```
import random

from constants import *
from genotype import ArpPattern, BassPattern, Gene, MChord


def generate_drum_step(position, profile="base"):
    """
    Gera um Gene de bateria para determinado step.
    """

    profile_values = {
        "base": (
            BAT1_EMPTY_STEP_PROBABILITY,
            BAT1_SECOND_NOTE_PROBABILITY,
            BAT1_THIRD_NOTE_PROBABILITY,
        ),
        "chorus": (
            BAT2_EMPTY_STEP_PROBABILITY,
            BAT2_SECOND_NOTE_PROBABILITY,
            BAT2_THIRD_NOTE_PROBABILITY,
```

Aqui **não alterei a lógica de mutação dos componentes existentes**. O `ArpPatt1 e ArpPatt2` recebem mutações independentes, assim como os dois padrões de baixo.

Pode mandar o próximo arquivo — provavelmente **`crossover.py`** será o próximo ponto importante.

crossover.py:

```
import random
from copy import copy

from constants import *
from genotype import BassPattern, Genotype, MChord
from individual import Individual



def clone_genes(genes):
    """
    Copia os Genes individualmente.

    Isso evita que pais e filhos compartilhem os mesmos
    objetos Gene.
    """

    return [
        copy(gene)
        for gene in genes
    ]



def make_child(
    bat1,
    bat2=None,
    bati=None,
    batf=None,
    bass_patt1=None,
    bass_patt2=None,
    mchords=None
):
```

```
    """
    Cria um novo Individual a partir de uma sequência
    de Genes de bateria.
    """

    kwargs = {
        "bat1": clone_genes(bat1),
        "bat2": clone_genes(bat1 if bat2 is None else bat2),
        "bati": clone_genes(bat1 if bati is None else bati),
        "batf": clone_genes(bat1 if batf is None else batf),
    }

    if bass_patt1 is not None:
        kwargs["bass_patt1"] = clone_bass_pattern(bass_patt1)

    if bass_patt2 is not None:
        kwargs["bass_patt2"] = clone_bass_pattern(bass_patt2)

    if mchords is not None:
        kwargs.update({
            f"mchord{index}": clone_chord(chord)
            for index, chord in enumerate(mchords, start=1)
        })

    return Individual(Genotype(**kwargs))


def clone_chord(chord):
    """Cria uma cópia independente de um MChord válido."""

    return MChord(
        chord.root,
        chord.chord_type
    )


def clone_bass_pattern(pattern):
```

```
    """Copia um BassPattern validado sem compartilhar suas notas."""

    return BassPattern(
        [
            type(note)(note.position, note.duration)
            for note in pattern.notes
        ]
    )


def clone_individual(individual):
    """Copia Bat1 e todos os MChords sem compartilhar objetos."""

    chord_kwargs = {
        f"mchord{index}": clone_chord(chord)
        for index, chord in enumerate(
            individual.genotype.mchords,
            start=1
        )
    }

    return Individual(
        Genotype(
            bati=clone_genes(individual.bati),
            bat1=clone_genes(individual.bat1),
            bat2=clone_genes(individual.bat2),
            batf=clone_genes(individual.batf),
            bass_patt1=clone_bass_pattern(individual.bass_patt1),
            bass_patt2=clone_bass_pattern(individual.bass_patt2),
            **chord_kwargs,
        )
    )


def _chord_signature(chord):
    return (chord.root, chord.chord_type)
```

```
def _select_parent_chord(parent1, parent2, index, used):
    """Seleciona um acorde correspondente ou de posição diferente."""

    candidates = []

    if random.random() < CROSSOVER_CHORD_MIX_RATE:
        candidates.extend([
            parent1.genotype.mchords[index],
            parent2.genotype.mchords[index],
        ])
    else:
        candidates.extend([
            chord
            for parent in (parent1, parent2)
            for chord in parent.genotype.mchords
        ])

    unused = [
        chord
        for chord in candidates
        if _chord_signature(chord) not in used
    ]

    return random.choice(unused or candidates)


def crossover(parent1, parent2):
    """
    Crossover de dois pontos aplicado ao padrão Bat1.

    O crossover nunca divide um Gene; os cortes acontecem
    entre steps, mantendo cada Gene válido.
    """

    if random.random() > CROSSOVER_RATE:
```

```
    return clone_individual(parent1)

bass_patt1 = (
    clone_bass_pattern(parent1.bass_patt1)
    if random.random() < 0.5
    else clone_bass_pattern(parent2.bass_patt1)
)
bass_patt2 = (
    clone_bass_pattern(parent1.bass_patt2)
    if random.random() < 0.5
    else clone_bass_pattern(parent2.bass_patt2)
)

total_steps = min(
    len(parent1.bat1),
    len(parent2.bat1)
)

if total_steps < 2:
    bati = clone_genes(parent1.bati)
    bat1 = clone_genes(parent1.bat1)
    bat2 = clone_genes(parent1.bat2)
    batf = clone_genes(parent1.batf)
else:
    point_a, point_b = sorted(
        random.sample(
            range(1, total_steps),
            2
        )
    )

    bat1 = (
        parent1.bat1[:point_a]
        + parent2.bat1[point_a:point_b]
        + parent1.bat1[point_b:]
    )
```

```
        bati = (
            parent1.bati[:point_a]
            + parent2.bati[point_a:point_b]
            + parent1.bati[point_b:]
        )

        bat2 = (
            parent1.bat2[:point_a]
            + parent2.bat2[point_a:point_b]
            + parent1.bat2[point_b:]
        )

        batf = (
            parent1.batf[:point_a]
            + parent2.batf[point_a:point_b]
            + parent1.batf[point_b:]
        )

    chords = []
    used = set()

    for index in range(NUM_MCHORDS):
        chord = _select_parent_chord(
            parent1,
            parent2,
            index,
            used
        )
        chord = clone_chord(chord)
        chords.append(chord)
        used.add(_chord_signature(chord))

    return make_child(
        bat1,
        bat2=bat2,
        bati=bati,
        batf=batf,
```

```
            bass_patt1=bass_patt1,
            bass_patt2=bass_patt2,
            mchords=chords,
        )
```

Aqui precisamos adicionar os dois padrões ao `make_child`, ao `clone_individual` e ao `crossover`. Mantive a mesma estratégia usada para o baixo: cada `ArpPatt` é escolhido independentemente de um dos pais.

Python

```
import random
from copy import copy

from constants import *
from genotype import ArpPattern, BassPattern, Genotype, MChord
from individual import Individual


def clone_genes(genes):
    """
    Copia os Genes individualmente.

    Isso evita que pais e filhos compartilhem os mesmos
    objetos Gene.
    """

    return [
        copy(gene)
        for gene in genes
    ]
```

A única diferença conceitual em relação aos baixos é que agora temos:

Plain text

```
Parent 1 ─┐
          ├── ArpPatt1 → filho
Parent 2 ─┘

Parent 1 ─┐
          ├── ArpPatt2 → filho
Parent 2 ─┘
```

Ou seja, `ArpPatt1` cruza com `ArpPatt1` dos pais e `ArpPatt2` com `ArpPatt2`. A **mutação** continua sendo responsável por criar novas estruturas rítmicas. Isso mantém Base e Refrão semanticamente separados.

Pode mandar agora o **`fitness.py`**.

constants.py:

```
import os
import secrets


#
==========================================================
======
# ALEATORIEDADE E ALGORITMO GENETICO
#
==========================================================
======

RANDOM_SEED = secrets.randbits(64)

POPULATION_SIZE = 100
GENERATIONS = 100
WORKERS = 1
CROSSOVER_RATE = 0.8
CROSSOVER_CHORD_MIX_RATE = 0.5
MUTATION_RATE = 0.15
BAT2_MUTATION_RATE = 0.15
BATI_MUTATION_RATE = 0.15
BATF_MUTATION_RATE = 0.15
BASS_PATTERN_MUTATION_RATE = 0.20
BASS_PATT1_MUTATION_RATE = 0.20
BASS_PATT2_MUTATION_RATE = 0.20
MCHORD_MUTATION_RATE = 0.20
MCHORD_ROOT_MIN = 48
MCHORD_ROOT_MAX = 84
RANDOM_IMMIGRANT_RATE = 0.05
```

```
ELITISM = True

#
================================================================
# ELEMENTOS DE BATERIA
#
================================================================

KICK = 0
SNARE = 1
CLOSED_HIHAT = 2
OPEN_HIHAT = 3
CRASH = 4

DRUMS = [
    KICK,
    SNARE,
    CLOSED_HIHAT,
    OPEN_HIHAT,
    CRASH,
]

DRUM_PITCHES = {
    KICK: 36,
    SNARE: 38,
    CLOSED_HIHAT: 42,
    OPEN_HIHAT: 46,
    CRASH: 49,
}

DRUM_NAMES = {
    KICK: "Kick",
    SNARE: "Snare",
    CLOSED_HIHAT: "Closed_HiHat",
    OPEN_HIHAT: "Open_HiHat",
```

```
    CRASH: "Crash",
}

DRUM_NAMES_FITNESS = {
    KICK: "Kick",
    SNARE: "Snare",
    CLOSED_HIHAT: "Closed Hi-Hat",
    OPEN_HIHAT: "Open Hi-Hat",
    CRASH: "Crash",
}



#
============================================================
======
# GERACAO E MIDI
#
============================================================
======

KICK_WEIGHT = 30
SNARE_WEIGHT = 25
CLOSED_HIHAT_WEIGHT = 35
OPEN_HIHAT_WEIGHT = 5
CRASH_WEIGHT = 4
BAT1_EMPTY_STEP_PROBABILITY = 0.15
BAT2_EMPTY_STEP_PROBABILITY = 0.05
BAT1_SECOND_NOTE_PROBABILITY = 0.45
BAT2_SECOND_NOTE_PROBABILITY = 0.65
BAT1_THIRD_NOTE_PROBABILITY = 0.15
BAT2_THIRD_NOTE_PROBABILITY = 0.35
BATI_EMPTY_STEP_PROBABILITY = 0.35
BATF_EMPTY_STEP_PROBABILITY = 0.25
BATI_SECOND_NOTE_PROBABILITY = 0.25
BATF_SECOND_NOTE_PROBABILITY = 0.50
BATI_THIRD_NOTE_PROBABILITY = 0.05
BATF_THIRD_NOTE_PROBABILITY = 0.20
```

```
BPM = 120
BEATS_PER_BAR = 4
STEPS_PER_BEAT = 2
STEPS_PER_BAR = BEATS_PER_BAR * STEPS_PER_BEAT
SECONDS_PER_BEAT = 60.0 / BPM
SECONDS_PER_STEP = SECONDS_PER_BEAT / STEPS_PER_BEAT

NUM_BAT1 = 32
BATI_BARS = 1
BAT1_BARS_FIRST = 8
BAT2_BARS_FIRST = 4
BAT1_BARS_SECOND = 8
BAT2_BARS_SECOND = 4
BATF_BARS = 1
NUM_MUSIC_BARS = (
    BATI_BARS
    + BAT1_BARS_FIRST
    + BAT2_BARS_FIRST
    + BAT1_BARS_SECOND
    + BAT2_BARS_SECOND
    + BATF_BARS
)
NUM_MCHORDS = 8
MCHORDS_PER_GROUP = 4
GROUP_REPETITIONS = 2
EMPTY_BARS_START = 1
GROUP1_REPETITIONS_FIRST = 2
GROUP2_REPETITIONS_FIRST = 1
GROUP1_REPETITIONS_SECOND = 2
GROUP2_REPETITIONS_SECOND = 1
EMPTY_BARS_END = 1
FINAL_EMPTY_BARS = 1
STRING_PROGRAM = 49
BASS_PROGRAM = 36
DRUM_TRACK_VELOCITY = 127
STRING_TRACK_VELOCITY = 90
```

```
BASS_TRACK_VELOCITY = 100
PLAY_FINAL_MIDI = True
MIDI_PLAYER = "fluidsynth"
SOUNDFONT_PATH = os.environ.get(
    "SOUNDFONT",
    "/usr/share/sounds/sf2/FluidR3_GM.sf2"
)
FLUIDSYNTH_AUDIO_DRIVER = os.environ.get(
    "FLUIDSYNTH_AUDIO_DRIVER",
    "alsa"
)
FLUIDSYNTH_AUDIO_DEVICE = os.environ.get(
    "FLUIDSYNTH_AUDIO_DEVICE",
    "plughw:0,0"
)



#
============================================================
======
# FITNESS
#
============================================================
======

MAX_REFS = 10

FITNESS_WEIGHTS = {
    "strong_beats": 0.12,
    "backbeat": 0.18,
    "kick": 0.15,
    "hihat": 0.12,
    "syncopation": 0.08,
    "density": 0.10,
    "variation": 0.08,
    "structure": 0.07,
    "combination": 0.10,
```

```
}

MSIM_WEIGHTS = {
    "kick": 0.18,
    "snare": 0.18,
    "hihat": 0.16,
    "open_hihat": 0.08,
    "crash": 0.08,
    "density": 0.12,
    "variation": 0.08,
    "polyrhythm": 0.06,
    "combination": 0.06,
}

RHYTHM_WEIGHT = 0.63
HARMONY_WEIGHT = 0.27
BASS_PATTERN_WEIGHT = 0.10
BASS_PATT1_WEIGHT = 0.05
BASS_PATT2_WEIGHT = 0.05
BAT1_RHYTHM_WEIGHT = 0.65
BAT2_RHYTHM_WEIGHT = 0.25
BATI_RHYTHM_WEIGHT = 0.05
BATF_RHYTHM_WEIGHT = 0.05

HARMONIC_WEIGHTS = {
    "chord_validity": 0.20,
    "tonal_coherence": 0.30,
    "root_motion": 0.25,
    "voice_leading": 0.25,
}

BASS_NOTE_COUNTS = (2, 4, 8)
BASS_NOTE_COUNT_WEIGHTS = (0.20, 0.60, 0.20)
BASS_BAR_UNITS = 16
BASS_DURATION_UNITS = (1, 2, 4, 8)
```

```
#
============================================================
======
# REPRESENTACAO GENETICA
#
============================================================
======

GENE_MAX_NOTES = 3

MCHORD_INTERVALS = {
    "major": (0, 4, 7),
    "minor": (0, 3, 7),
    "diminished": (0, 3, 6),
}
MCHORD_MIN_NOTES = 3
MCHORD_MAX_NOTES = 5
# Faixa de oitava dos MChords: C3 ate C4.
MCHORD_MIN_MIDI_NOTE = 48
MCHORD_MAX_MIDI_NOTE = 60

GENOTYPE_BPM = BPM
GENOTYPE_BEATS_PER_BAR = BEATS_PER_BAR
GENOTYPE_STEPS_PER_BEAT = STEPS_PER_BEAT
GENOTYPE_STEPS_PER_BAR = STEPS_PER_BAR
GENOTYPE_MAX_POLYPHONY = 3
GENOTYPE_DRUMS = [
    "Kick",
    "Snare",
    "Closed_HiHat",
    "Open_HiHat",
    "Crash",
]
GENOTYPE_NUM_DRUMS = len(GENOTYPE_DRUMS)
```

Para `constants.py`, eu adicionaria apenas as constantes necessárias para o `ArpPattern`, suas taxas de mutação e seus pesos no fitness.

Há um ponto importante: **1 nota + duração máxima de 1/2 compasso é incompatível com a regra atual de preencher exatamente o compasso**. Então vou manter `1` como quantidade permitida, mas isso precisará ser tratado no `ArpPattern` quando ajustarmos essa lógica.

Python

```
import os
import secrets


# ============================================================
# ALEATORIEDADE E ALGORITMO GENETICO
# ============================================================

RANDOM_SEED = secrets.randbits(64)

POPULATION_SIZE = 100
GENERATIONS = 100
WORKERS = 1
CROSSOVER_RATE = 0.8
CROSSOVER_CHORD_MIX_RATE = 0.5
MUTATION_RATE = 0.15
BAT2_MUTATION_RATE = 0.15
BATI_MUTATION_RATE = 0.15
BATF_MUTATION_RATE = 0.15
BASS_PATTERN_MUTATION_RATE = 0.20
BASS_PATT1_MUTATION_RATE = 0.20
```

Fiz também o peso global ficar consistente:

Plain text

```
RHYTHM_WEIGHT       0.53
HARMONY_WEIGHT      0.27
BASS_PATTERN_WEIGHT 0.10
ARP_PATTERN_WEIGHT  0.10
                    ----
                    1.00
```

E, dentro de cada padrão, `Patt1/Patt2 = 0.05 + 0.05` .

Atenção: antes de testar o código, precisamos corrigir a questão do `ArpPattern` com **1 nota**. Com `ARP_DURATION_UNITS = (1, 2, 4, 8, 16)` , nenhuma duração consegue preencher as 32 unidades

do compasso. Eu não mudaria isso silenciosamente nos `constants.py`; o correto é decidir a regra no `genotype.py`.

fitness.py:

"""
fitness.py

Função de avaliação (fitness) para indivíduos musicais.

O indivíduo atual contém:

  Individual
    └── Genotype
        ├── BatI
        ├── Bat1
        ├── Bat2
        ├── BatF
        └── 8 MChords

Cada Gene representa uma colcheia de um compasso 4/4.

…

Show more

No `fitness.py`, vou adicionar a avaliação dos dois padrões de arpejo de forma análoga ao baixo, considerando principalmente **quantidade de notas** e **variação das durações**. Também atualizo a documentação e o cálculo final.

```python
"""
fitness.py

Função de avaliação (fitness) para indivíduos musicais.

O indivíduo atual contém:

    Individual
        └── Genotype
```

```
├── BatI
├── Bat1
├── Bat2
├── BatF
├── BassPatt1
├── BassPatt2
├── ArpPatt1
├── ArpPatt2
└── 8 MChords

Cada Gene representa uma colcheia de um compasso 4/4.
```

A distribuição final usada no `fitness` fica:

- Ritmo/bateria: **0.53**
- Harmonia: **0.27**
- BassPatt1: **0.05**
- BassPatt2: **0.05**
- ArpPatt1: **0.05**
- ArpPatt2: **0.05**

Total = **1.00**.

Também mantive `ArpPatt1 e ArpPatt2` independentes na avaliação.

genetic_algorithm.py:

```
import random
import subprocess
import sys
from pathlib import Path
from concurrent.futures import ProcessPoolExecutor

from constants import *
from individual import Individual
from genotype import Genotype
from fitness import calculate_fitness
from generate_midi_multi import generate_midi
from crossover import clone_individual, crossover
from mutation import generate_drum_step, mutate
```

random.seed(RANDOM_SEED)

…

Show more

Analisei o `genetic_algorithm.py`. **Não é necessário alterar a lógica do algoritmo genético para adicionar `ArpPatt1` e `ArpPatt2`.**

Isso acontece porque a evolução já está centralizada em:

- `Individual()` → cria os `ArpPattern`;
- `calculate_fitness()` → agora avalia os dois arpejos;
- `crossover()` → já cruza os dois padrões;
- `mutate()` → já pode mutar os dois padrões;
- `clone_individual()` → já copia os dois padrões.

Portanto, este arquivo continua válido **sem nenhuma alteração funcional**.

O único ajuste que eu faria é atualizar a documentação de `initialize_individual()`, pois ela ainda descreve apenas a bateria. Mantendo exatamente o estilo do seu arquivo:

```python
import random
import subprocess
import sys
from pathlib import Path
from concurrent.futures import ProcessPoolExecutor

from constants import *
from individual import Individual
from genotype import Genotype
from fitness import calculate_fitness
from generate_midi_multi import generate_midi
from crossover import clone_individual, crossover
from mutation import generate_drum_step, mutate


random.seed(RANDOM_SEED)


# ============================================================
# PARÂMETROS DO ALGORITMO
# ============================================================
```

Importante: não adicionei nenhum tratamento específico de `ArpPatt1/2` dentro do GA porque isso seria redundante. O fluxo já é:

```
Plain text

Individual()
    ↓
ArpPatt1 / ArpPatt2
    ↓
calculate_fitness()
    ↓
selection()
    ↓
crossover()
    ↓
mutate()
    ↓
novo Individual
```

Assim, os dois padrões já participam da evolução exatamente como `BassPatt1 e BassPatt2 .`

```
python3 ga/genetic_algorithm.py
Traceback (most recent call last):
  File "/app/ga/genetic_algorithm.py", line 8, in <module>
    from individual import Individual
  File "/app/ga/individual.py", line 4, in <module>
    class Individual:
  File "/app/ga/individual.py", line 119, in Individual
    @mchord6.setter
     ^^^^^^^
NameError: name 'mchord6' is not defined
```

Esse erro está no `individual.py` , não no `genetic_algorithm.py` .

O `@mchord6.setter` está aparecendo sem que exista imediatamente antes dele um property def mchord6(...). Isso provavelmente aconteceu na edição anterior e pode ter afetado a sequência dos `mchord` .

Não altere o `genetic_algorithm.py` .

Me envie o seu `individual.py` atual completo e eu corrijo **somente esse arquivo**, preservando exatamente a estrutura original e adicionando apenas `arp_patt1 e arp_patt2` .

```
from constants import *
from genotype import Genotype

class Individual:
    """
    Representa um indivíduo da população.

    Um indivíduo possui um genótipo completo, composto por:
        - 4 padrões de bateria (BatI, Bat1, Bat2 e BatF)
        - 8 genes, sendo um para cada colcheia do compasso
        - 8 acordes harmônicos consecutivos
        - 2 padrões de baixo
        - 2 padrões de arpejo
    """

    def __init__(self, genotype=None):
        self.genotype = (
            Genotype()
            if genotype is None
            else genotype
        )

    @property
    def bat1(self):
        return self.genotype.bat1

    @property
    def bat2(self):
        return self.genotype.bat2

    @property
    def bati(self):
        return self.genotype.bati

    @property
    def batf(self):
        return self.genotype.batf
```

```
@property
def bass_patt1(self):
    return self.genotype.bass_patt1

@bass_patt1.setter
def bass_patt1(self, value):
    self.genotype.bass_patt1 = value

@property
def bass_patt2(self):
    return self.genotype.bass_patt2

@bass_patt2.setter
def bass_patt2(self, value):
    self.genotype.bass_patt2 = value

@property
def arp_patt1(self):
    return self.genotype.arp_patt1

@arp_patt1.setter
def arp_patt1(self, value):
    self.genotype.arp_patt1 = value

@property
def arp_patt2(self):
    return self.genotype.arp_patt2

@arp_patt2.setter
def arp_patt2(self, value):
    self.genotype.arp_patt2 = value

@property
def bass_pattern(self):
    return self.genotype.bass_patt1
```

```
@bass_pattern.setter
def bass_pattern(self, value):
    self.genotype.bass_patt1 = value

@property
def mchord1(self):
    return self.genotype.mchord1

@mchord1.setter
def mchord1(self, value):
    self.genotype.mchord1 = value

@property
def mchord2(self):
    return self.genotype.mchord2

@mchord2.setter
def mchord2(self, value):
    self.genotype.mchord2 = value

@property
def mchord3(self):
    return self.genotype.mchord3

@mchord3.setter
def mchord3(self, value):
    self.genotype.mchord3 = value

@property
def mchord4(self):
    return self.genotype.mchord4

@mchord4.setter
def mchord4(self, value):
    self.genotype.mchord4 = value

@property
```

```
def mchord5(self):
    return self.genotype.mchord5

@mchord5.setter
def mchord5(self, value):
    return self.genotype.mchord5

@mchord6.setter
def mchord6(self, value):
    self.genotype.mchord6 = value

@property
def mchord7(self):
    return self.genotype.mchord7

@mchord7.setter
def mchord7(self, value):
    self.genotype.mchord7 = value

@property
def mchord8(self):
    return self.genotype.mchord8

@mchord8.setter
def mchord8(self, value):
    self.genotype.mchord8 = value

def __repr__(self):
    return (
        f"Individual("
        f"bat1={self.bat1}, "
        f"mchords={self.genotype.mchords})"
    )
```

Sim. O erro está exatamente aqui: o `mchord6` está sem o decorador `@property`. Além disso, o setter de `mchord5` está incorreto: ele usa `return` em vez de atribuir.

Vou corrigir **somente esses dois pontos**. O restante permanece igual.

Python

```
from constants import *
from genotype import Genotype

class Individual:
    """
    Representa um indivíduo da população.

    Um indivíduo possui um genótipo completo, composto por:
        - 4 padrões de bateria (BatI, Bat1, Bat2 e BatF)
        - 8 genes, sendo um para cada colcheia do compasso
        - 8 acordes harmônicos consecutivos
        - 2 padrões de baixo
        - 2 padrões de arpejo
    """

    def __init__(self, genotype=None):
        self.genotype = (
            Genotype()
            if genotype is None
            else genotype
        )
```

Os dois erros eram:

Python

```
@mchord5.setter
def mchord5(self, value):
    return self.genotype.mchord5
```

→ corrigido para atribuição.

E:

Python

```
@mchord6.setter
```

→ faltava o:

Python

```
    @property
    def mchord6(self):
        return self.genotype.mchord6
```

Agora o `NameError` específico deve desaparecer.

> Obrigado. Mas está dando o mesmo problema que deu com o copilot... Está travando antes de começar a evoluir o GA. Acho que ficou muito denso esses arp_patt né?

Sim — **acho que você identificou o problema certo**. O `ArpPattern` que montamos ficou muito mais complexo do que deveria para um gene de GA.

O principal problema não é o `ArpPatt1/2` em si. É a forma como ele está sendo gerado.

Hoje, para criar um `ArpPattern`, fazemos algo conceitualmente parecido com:

```
Plain text
escolhe quantidade de notas
        ↓
gera TODAS as combinações possíveis de durações
        ↓
filtra as que preenchem exatamente o compasso
        ↓
escolhe uma aleatoriamente
        ↓
cria ArpNote para cada nota
        ↓
valida tudo
```

Com durações:

```
Plain text
1, 2, 4, 8, 16
```

e até **32 notas**, isso explode combinatoriamente.

O problema

Por exemplo, para 16 notas, estamos procurando sequências de 16 valores cuja soma seja 32:

Plain text

```
d1 + d2 + ... + d16 = 32
```

E para 32 notas:

Plain text

```
d1 + d2 + ... + d32 = 32
```

A função recursiva `_generate_durations()` pode visitar **uma quantidade enorme de combinações** antes de encontrar uma solução.

E isso acontece durante:

Python

```
population = [
    initialize_individual()
    for _ in range(POPULATION_SIZE)
]
```

Ou seja, **o GA nem chegou na primeira geração**. Ele pode estar gastando todo o tempo construindo a população inicial.

Eu simplificaria bastante

Para o nosso GA, não precisamos de um gerador combinatório.

Podemos gerar um padrão diretamente:

1. escolhe quantidade de notas;
2. distribui as notas sequencialmente;
3. escolhe uma duração válida para cada nota;
4. garante que o padrão caiba no compasso.

Algo muito mais barato.

E, na verdade, eu faria uma mudança conceitual importante:

Não precisamos que as durações preencham exatamente o compasso.

O usuário originalmente definiu:

> quantidade de notas distribuídas sequencialmente no compasso, com um valor para cada uma delas que determina a posição dela em relação à anterior.

Isso permite algo como:

```
Plain text

Compasso
0       8       16      24      32
|-------|-------|-------|-------|

Nota 1: posição 0,  duração 4
Nota 2: posição 6,  duração 2
Nota 3: posição 12, duração 8
Nota 4: posição 24, duração 4
```

Ou seja, **posição e duração são propriedades diferentes**.

Isso é inclusive mais adequado para um arpejo.

Eu sugiro deixar o `ArpPattern` assim

Estrutura:

```
Plain text

ArpPattern
│
├── quantidade de notas: 1, 2, 4, 8, 16, 32
│
└── ArpNote
    ├── position
    └── duration
```

E gerar cada nota em tempo praticamente constante, sem enumerar combinações.

Por exemplo:

```
Plain text
```

```
ArpPatt1

N = 4

Nota 1 → position=0   duration=4
Nota 2 → position=8   duration=2
Nota 3 → position=16  duration=4
Nota 4 → position=24  duration=2
```

Outro indivíduo:

Plain text

```
N = 8

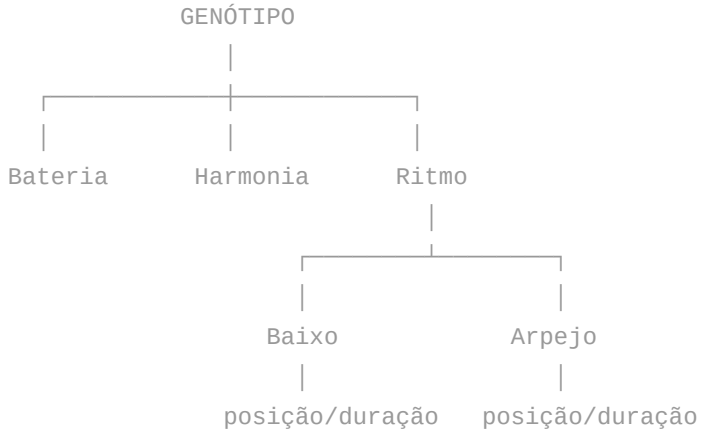
Nota 1 → position=0   duration=2
Nota 2 → position=4   duration=1
Nota 3 → position=8   duration=4
Nota 4 → position=12  duration=2
...
```

Isso é **muito mais leve**.

E tem outra vantagem

Nosso GA ficaria conceitualmente muito mais limpo:

Plain text



O `ArpPattern` não precisa saber nada sobre música harmônica.

Depois, no `generate_midi_multi.py`, fazemos:

```
Plain text
ArpPattern
    +
MChord
    ↓
escolha das alturas
    ↓
MIDI
```

Isso está muito mais alinhado com a ideia original.

Mas eu faria mais uma coisa

Antes de modificar tudo, podemos confirmar se **é realmente o `ArpPattern` que está travando**.

Faça temporariamente:

```
Bash
python3 -c "from genotype import Genotype; print('OK'); g=Genotype(); print(g)"
```

Se demorar/travar nessa linha:

```
Python
g = Genotype()
```

então temos certeza de que o problema está na geração/validação do `ArpPattern`.

Minha recomendação: simplificarmos agora o `ArpPattern` antes de continuar. É melhor fazer isso neste ponto do que colocar mais código no `genetic_algorithm.py` tentando contornar um problema que está na inicialização do genótipo.

> Me fala quais mudanças posso fazer pra diminuir a complexidade dele, se for em poucos arquivos eu te passo eles aqui

Sim. Eu faria uma **simplificação estrutural**, não apenas uma otimização do código. O objetivo é deixar `ArpPattern` tão simples quanto `BassPattern`.

Eu mudaria em **2 arquivos principais**:

1. `genotype.py` — geração e representação do `ArpPattern`
2. `fitness.py` — avaliação do padrão

E provavelmente **não precisamos mexer** em `individual.py`, `crossover.py`, `mutation.py` ou `genetic_algorithm.py`, porque a interface `arp_patt1/2` pode continuar igual.

1. Simplificar o `ArpPattern`

Hoje temos a parte problemática:

```python
_generate_durations()
    ↓
collect()
    ↓
tenta várias combinações
    ↓
verifica se soma exatamente 32
```

Eu eliminaria **completamente** essa busca combinatória.

Em vez disso:

```
1. Escolhe quantidade de notas
2. Escolhe posições válidas
3. Escolhe uma duração para cada nota
4. Pronto
```

Por exemplo:

```
ArpPatt1
    4 notas

Nota 1: posição 0   duração 4
```

```
Nota 2: posição 8   duração 2
Nota 3: posição 16  duração 4
Nota 4: posição 24  duração 2
```

Não precisamos procurar uma sequência que some exatamente um compasso.

---

2. Simplificar a posição

Eu também evitaria a ideia atual de:

Python

```
position = duração_da_nota_anterior
```

Isso mistura **posição** com **duração**.

Seria mais simples gerar diretamente:

Python

```
position = ...
duration = ...
```

A posição poderia ser gerada a partir de uma divisão simples do compasso.

Por exemplo:

Plain text

```
4 notas → posições aproximadamente 0, 8, 16, 24
8 notas → 0, 4, 8, 12, ...
16 notas → 0, 2, 4, 6, ...
32 notas → 0, 1, 2, 3, ...
```

E depois podemos adicionar uma pequena variação aleatória, se quisermos.

---

3. Manter as quantidades

Eu manteria:

Python

```
ARP_NOTE_COUNTS = (1, 2, 4, 8, 16, 32)
```

e:

Python

```
ARP_NOTE_COUNT_WEIGHTS = (
    0.05,
    0.10,
    0.30,
    0.30,
    0.20,
    0.05
)
```

Isso é barato e mantém a característica musical que você definiu.

---

4. Manter as durações

Também manteria:

Python

```
ARP_DURATION_UNITS = (1, 2, 4, 8, 16)
```

correspondendo a:

Plain text

```
1/32 → 1
1/16 → 2
1/8  → 4
1/4  → 8
1/2  → 16
```

A diferença é que **não vamos tentar encontrar uma combinação cuja soma seja 32**.

---

5. Validação muito mais simples

O `_validate()` poderia verificar somente:

```
Plain text
```

```
✓ quantidade permitida
✓ todos são ArpNote
✓ posição válida
✓ posições em ordem
✓ duração válida
```

E remover:

```
Python
```

```
total_duration == BAR_UNITS
```

Esse teste é justamente o que está nos obrigando a construir combinações complicadas.

---

6. `fitness.py`

Também podemos simplificar.

Em vez de avaliar:

```
Plain text
```

```
quantidade de notas
+
variedade de durações
```

podemos começar apenas com uma preferência pela quantidade:

```
Plain text
```

```
4  → 1.0
8  → 1.0
16 → 1.0
2  → 0.7
```

```
32 → 0.7
1  → 0.5
```

Depois, se quisermos, acrescentamos variedade de duração.

Isso deixa o fitness barato e previsível.

O resultado seria

```
Plain text
```

```
Genotype
│
├── Battery
├── MChord
├── BassPattern
│   └── posição + duração
│
└── ArpPattern
    └── posição + duração
```

Sem busca combinatória.

E o mais importante: **o GA continua enxergando `ArpPatt1` e `ArpPatt2` como genes normais**.

Arquivos que preciso

Me passe **somente estes dois arquivos**:

```
Plain text
```

```
genotype.py
fitness.py
```

Eu faço a simplificação preservando o restante do seu código e sem mexer no `generate_midi_multi.py` .

> Vamos fazer o seguinte, eu removi tudo que era referente a esses arp_patt no meu código, e vamos inclui-los novamente mas primeiro vamos definir estruturas mais simples pra eles. Me diga como ficarão esses ArpPatt1 e ArpPatt2 com estruturas mais simples do

que as que eu pedi anteriormente (+- metade da complexidade do anterior).

Sim. Acho que é melhor **voltar um passo e definir o gene antes de implementar**.

Eu faria `ArpPatt1 e ArpPatt2` com aproximadamente metade da complexidade anterior, mantendo a ideia musical principal, mas eliminando a combinação entre posição e duração.

Estrutura proposta

Cada arpejo seria simplesmente:

```
Plain text

ArpPattern
├── note_count
└── durations
```

Por exemplo:

```
Plain text

ArpPatt1
    note_count = 8
    durations = [1, 1, 2, 2, 1, 1, 2, 2]
```

Onde:

- `note_count` = número de notas no compasso.
- `durations[i]` = duração da nota `i` .
- As notas são **automaticamente distribuídas sequencialmente** no compasso.
- Não armazenamos `position` .
- Não armazenamos altura/pitch.

Quantidade de notas

Eu manteria:

```
Plain text

1, 2, 4, 8, 16, 32
```

Mas com probabilidades:

```
Plain text
1  → 5%
2  → 10%
4  → 30%
8  → 30%
16 → 20%
32 → 5%
```

Assim, 4, 8 e 16 continuam sendo os padrões mais comuns.

Duração

Aqui eu simplificaria bastante.

Cada nota poderia ter apenas:

```
Plain text
1/8
1/16
1/32
```

ou seja:

```
Plain text
duration = 4, 2 ou 1
```

considerando:

```
Plain text
1 compasso = 32 unidades
```

Eu **removeria 1/2 e 1/4**.

Isso reduz bastante o espaço de busca.

Além disso, para um arpejo de rock, durações curtas fazem mais sentido como ponto de partida.

Mas há uma questão importante

Não precisamos armazenar a posição porque podemos calculá-la.

Por exemplo:

```
Plain text

ArpPatt1
note_count = 4
durations = [4, 4, 4, 4]
```

gera automaticamente:

```
Plain text

Nota 1 → posição 0
Nota 2 → posição 4
Nota 3 → posição 8
Nota 4 → posição 12
```

Porém isso não preencheria o compasso inteiro, porque estamos usando apenas 16 unidades.

Então eu prefiro uma representação ainda mais simples:

A duração representa o espaçamento entre notas

Por exemplo:

```
Plain text

durations = [8, 8, 8, 8]
```

significa:

```
Plain text

0 → 8 → 16 → 24
```

Ou:

```
Plain text
[4, 4, 4, 4, 4, 4, 4, 4]
```

significa:

```
Plain text
0 → 4 → 8 → 12 → 16 → 20 → 24 → 28
```

Nesse modelo, **`duration` passa a ser praticamente o intervalo rítmico entre ataques**.

Isso elimina completamente a necessidade de `position`.

Minha proposta final

Eu faria o gene assim:

```
Plain text
ArpPattern
│
├── note_count
│
└── step
```

Ou seja, **nem sequer precisamos guardar uma lista de durações**.

Exemplos:

```
Plain text
ArpPatt1
    note_count = 4
    step = 8
```

Resultado:

Plain text

```
0, 8, 16, 24
```

Outro:

Plain text

```
ArpPatt2
    note_count = 8
    step = 4
```

Resultado:

Plain text

```
0, 4, 8, 12, 16, 20, 24, 28
```

E:

Plain text

```
note_count = 16
step = 2
```

Resultado:

Plain text

```
0, 2, 4, 6, 8, 10, ..., 30
```

Isso é **muito mais simples** para um algoritmo genético.

O que o GA realmente evoluiria

Teríamos somente dois genes:

Plain text

```
ArpPatt1 = (note_count, step)
ArpPatt2 = (note_count, step)
```

Por exemplo:

Plain text

```
Indivíduo A

ArpPatt1 = (4, 8)
ArpPatt2 = (8, 4)
```

Outro:

Plain text

```
Indivíduo B

ArpPatt1 = (8, 4)
ArpPatt2 = (16, 2)
```

O crossover pode simplesmente escolher cada padrão de um dos pais:

Plain text

```
Pai 1:
ArpPatt1 = (4,8)
ArpPatt2 = (16,2)

Pai 2:
ArpPatt1 = (8,4)
ArpPatt2 = (4,8)

Filho:
ArpPatt1 = (8,4)     ← Pai 2
ArpPatt2 = (16,2)    ← Pai 1
```

Mutação também fica trivial:

Plain text

```
(4,8) → (8,4)
```

ou:

```
Plain text
(8,4) → (16,2)
```

E o pitch continua fora do genótipo

Isso permanece exatamente como você queria:

```
Plain text
ArpPatt
    ↓
ritmo
    +
MChord
    ↓
escolha das notas
    ↓
MIDI
```

Por exemplo, se:

```
Plain text
MChord = C major
ArpPatt = (8,4)
```

podemos gerar:

```
Plain text
C - E - G - E - C - E - G - E
```

Mas o genótipo só sabe:

```
Plain text
(8,4)
```

Comparando

Aspecto	Estrutura anterior	Nova
Nº de genes principais	vários	**2**
Quantidade de notas	✓	✓
Posição individual	✓	×
Duração individual	✓	×
Step rítmico	indireto	✓
Pitch	×	×
Busca combinatória	sim	**não**
Crossover	complexo	**trivial**
Mutação	complexa	**trivial**
Validação	complexa	**muito simples**

Eu escolheria essa versão. Ela preserva a ideia de que o GA aprende **o ritmo do arpejo**, mas reduz drasticamente o espaço de busca.

E, principalmente, não devemos tentar colocar toda a riqueza musical dentro do genótipo. Podemos deixar a **geração do MIDI** responsável por transformar `(note_count, step) + MChord` em um arpejo musical mais interessante.

> Ok, vou te passar código por código e você edita eles incluindo esses ArpPatts, ok?

Perfeito. Pode passar **um arquivo por vez**.

Para cada arquivo eu vou:

- incluir `ArpPatt1` e `ArpPatt2` usando a estrutura simplificada `(note_count, step)`;
- preservar o estilo e a organização do seu código;

- alterar somente o que for necessário;
- não mexer em `generate_midi_multi.py` por enquanto;
- evitar qualquer geração combinatória ou estrutura desnecessariamente complexa.

A estrutura será:

```
Plain text

ArpPatt1 = (quantidade_de_notas, step)
ArpPatt2 = (quantidade_de_notas, step)
```

com:

```
Plain text

quantidade = 1, 2, 4, 8, 16 ou 32
step       = espaçamento entre ataques
```

Pode mandar o primeiro arquivo.

Vamos começar pelo constants.py assim você define constantes novas se precisar...

```
import os
import secrets


#
============================================================
======
# ALEATORIEDADE E ALGORITMO GENETICO
#
============================================================
======

RANDOM_SEED = secrets.randbits(64)

POPULATION_SIZE = 100
GENERATIONS = 100
```

```
WORKERS = 1
CROSSOVER_RATE = 0.8
CROSSOVER_CHORD_MIX_RATE = 0.5
MUTATION_RATE = 0.15
BAT2_MUTATION_RATE = 0.15
BATI_MUTATION_RATE = 0.15
BATF_MUTATION_RATE = 0.15
BASS_PATTERN_MUTATION_RATE = 0.20
BASS_PATT1_MUTATION_RATE = 0.20
BASS_PATT2_MUTATION_RATE = 0.20
MCHORD_MUTATION_RATE = 0.20
MCHORD_ROOT_MIN = 48
MCHORD_ROOT_MAX = 84
RANDOM_IMMIGRANT_RATE = 0.05
ELITISM = True

#
============================================================
======
# ELEMENTOS DE BATERIA
#
============================================================
======

KICK = 0
SNARE = 1
CLOSED_HIHAT = 2
OPEN_HIHAT = 3
CRASH = 4

DRUMS = [
    KICK,
    SNARE,
    CLOSED_HIHAT,
    OPEN_HIHAT,
    CRASH,
]
```

```
DRUM_PITCHES = {
    KICK: 36,
    SNARE: 38,
    CLOSED_HIHAT: 42,
    OPEN_HIHAT: 46,
    CRASH: 49,
}

DRUM_NAMES = {
    KICK: "Kick",
    SNARE: "Snare",
    CLOSED_HIHAT: "Closed_HiHat",
    OPEN_HIHAT: "Open_HiHat",
    CRASH: "Crash",
}

DRUM_NAMES_FITNESS = {
    KICK: "Kick",
    SNARE: "Snare",
    CLOSED_HIHAT: "Closed Hi-Hat",
    OPEN_HIHAT: "Open Hi-Hat",
    CRASH: "Crash",
}



#
================================================
======
# GERACAO E MIDI
#
================================================
======

KICK_WEIGHT = 30
SNARE_WEIGHT = 25
CLOSED_HIHAT_WEIGHT = 35
```

```
OPEN_HIHAT_WEIGHT = 5
CRASH_WEIGHT = 4
BAT1_EMPTY_STEP_PROBABILITY = 0.15
BAT2_EMPTY_STEP_PROBABILITY = 0.05
BAT1_SECOND_NOTE_PROBABILITY = 0.45
BAT2_SECOND_NOTE_PROBABILITY = 0.65
BAT1_THIRD_NOTE_PROBABILITY = 0.15
BAT2_THIRD_NOTE_PROBABILITY = 0.35
BATI_EMPTY_STEP_PROBABILITY = 0.35
BATF_EMPTY_STEP_PROBABILITY = 0.25
BATI_SECOND_NOTE_PROBABILITY = 0.25
BATF_SECOND_NOTE_PROBABILITY = 0.50
BATI_THIRD_NOTE_PROBABILITY = 0.05
BATF_THIRD_NOTE_PROBABILITY = 0.20

BPM = 120
BEATS_PER_BAR = 4
STEPS_PER_BEAT = 2
STEPS_PER_BAR = BEATS_PER_BAR * STEPS_PER_BEAT
SECONDS_PER_BEAT = 60.0 / BPM
SECONDS_PER_STEP = SECONDS_PER_BEAT / STEPS_PER_BEAT

NUM_BAT1 = 32
BATI_BARS = 1
BAT1_BARS_FIRST = 8
BAT2_BARS_FIRST = 4
BAT1_BARS_SECOND = 8
BAT2_BARS_SECOND = 4
BATF_BARS = 1
NUM_MUSIC_BARS = (
    BATI_BARS
    + BAT1_BARS_FIRST
    + BAT2_BARS_FIRST
    + BAT1_BARS_SECOND
    + BAT2_BARS_SECOND
    + BATF_BARS
)
```

```
NUM_MCHORDS = 8
MCHORDS_PER_GROUP = 4
GROUP_REPETITIONS = 2
EMPTY_BARS_START = 1
GROUP1_REPETITIONS_FIRST = 2
GROUP2_REPETITIONS_FIRST = 1
GROUP1_REPETITIONS_SECOND = 2
GROUP2_REPETITIONS_SECOND = 1
EMPTY_BARS_END = 1
FINAL_EMPTY_BARS = 1
STRING_PROGRAM = 49
BASS_PROGRAM = 36
DRUM_TRACK_VELOCITY = 127
STRING_TRACK_VELOCITY = 90
BASS_TRACK_VELOCITY = 100
PLAY_FINAL_MIDI = True
MIDI_PLAYER = "fluidsynth"
SOUNDFONT_PATH = os.environ.get(
    "SOUNDFONT",
    "/usr/share/sounds/sf2/FluidR3_GM.sf2"
)
FLUIDSYNTH_AUDIO_DRIVER = os.environ.get(
    "FLUIDSYNTH_AUDIO_DRIVER",
    "alsa"
)
FLUIDSYNTH_AUDIO_DEVICE = os.environ.get(
    "FLUIDSYNTH_AUDIO_DEVICE",
    "plughw:0,0"
)



#
============================================================
======
# FITNESS
#
============================================================
```

```
======

MAX_REFS = 10

FITNESS_WEIGHTS = {
   "strong_beats": 0.12,
   "backbeat": 0.18,
   "kick": 0.15,
   "hihat": 0.12,
   "syncopation": 0.08,
   "density": 0.10,
   "variation": 0.08,
   "structure": 0.07,
   "combination": 0.10,
}

MSIM_WEIGHTS = {
   "kick": 0.18,
   "snare": 0.18,
   "hihat": 0.16,
   "open_hihat": 0.08,
   "crash": 0.08,
   "density": 0.12,
   "variation": 0.08,
   "polyrhythm": 0.06,
   "combination": 0.06,
}

RHYTHM_WEIGHT = 0.63
HARMONY_WEIGHT = 0.27
BASS_PATTERN_WEIGHT = 0.10
BASS_PATT1_WEIGHT = 0.05
BASS_PATT2_WEIGHT = 0.05
BAT1_RHYTHM_WEIGHT = 0.65
BAT2_RHYTHM_WEIGHT = 0.25
BATI_RHYTHM_WEIGHT = 0.05
BATF_RHYTHM_WEIGHT = 0.05
```

```
HARMONIC_WEIGHTS = {
    "chord_validity": 0.20,
    "tonal_coherence": 0.30,
    "root_motion": 0.25,
    "voice_leading": 0.25,
}

BASS_NOTE_COUNTS = (2, 4, 8)
BASS_NOTE_COUNT_WEIGHTS = (0.20, 0.60, 0.20)
BASS_BAR_UNITS = 16
BASS_DURATION_UNITS = (1, 2, 4, 8)



#
============================================================
======
# REPRESENTACAO GENETICA
#
============================================================
======

GENE_MAX_NOTES = 3

MCHORD_INTERVALS = {
    "major": (0, 4, 7),
    "minor": (0, 3, 7),
    "diminished": (0, 3, 6),
}
MCHORD_MIN_NOTES = 3
MCHORD_MAX_NOTES = 5
# Faixa de oitava dos MChords: C3 ate C4.
MCHORD_MIN_MIDI_NOTE = 48
MCHORD_MAX_MIDI_NOTE = 60

GENOTYPE_BPM = BPM
GENOTYPE_BEATS_PER_BAR = BEATS_PER_BAR
```

```
GENOTYPE_STEPS_PER_BEAT = STEPS_PER_BEAT
GENOTYPE_STEPS_PER_BAR = STEPS_PER_BAR
GENOTYPE_MAX_POLYPHONY = 3
GENOTYPE_DRUMS = [
    "Kick",
    "Snare",
    "Closed_HiHat",
    "Open_HiHat",
    "Crash",
]
GENOTYPE_NUM_DRUMS = len(GENOTYPE_DRUMS)
```

Perfeito. Para essa versão simplificada, eu adicionaria apenas as constantes necessárias para os dois `ArpPattern`.

A ideia será:

- `ArpPatt1` e `ArpPatt2` têm mutações independentes;
- quantidade de notas: `1, 2, 4, 8, 16, 32`;
- maior probabilidade para `4`, `8` e `16`;
- `step` representa o espaçamento entre ataques;
- como o compasso tem 32 unidades, os `step` naturais serão `1, 2, 4, 8, 16, 32`;
- não precisamos de `ARP_DURATION_UNITS`, `ARP_BAR_UNITS` etc. nesta versão.

Eu também ajustaria os pesos gerais do fitness para reservar **5% para cada ArpPatt**, reduzindo `RHYTHM_WEIGHT` de `0.63` para `0.53`. Assim a soma continua 1.0.

O `constants.py` ficaria assim:

```python
import os
import secrets


# ============================================================
# ALEATORIEDADE E ALGORITMO GENETICO
# ============================================================

RANDOM_SEED = secrets.randbits(64)

POPULATION_SIZE = 100
```

```
GENERATIONS = 100
WORKERS = 1
CROSSOVER_RATE = 0.8
CROSSOVER_CHORD_MIX_RATE = 0.5
MUTATION_RATE = 0.15
BAT2_MUTATION_RATE = 0.15
BATI_MUTATION_RATE = 0.15
BATF_MUTATION_RATE = 0.15
BASS_PATTERN_MUTATION_RATE = 0.20
```

As três constantes novas realmente necessárias para a representação são:

Python

```
ARP_NOTE_COUNTS
ARP_NOTE_COUNT_WEIGHTS
ARP_STEP_UNITS
```

E as quatro de mutação/fitness:

Python

```
ARP_PATT1_MUTATION_RATE
ARP_PATT2_MUTATION_RATE
ARP_PATTERN_WEIGHT
ARP_PATT1_WEIGHT
ARP_PATT2_WEIGHT
```

Não coloquei `ARP_DURATION_UNITS` nem `ARP_BAR_UNITS`, porque não vamos mais representar duração individual. Isso mantém a nova estrutura realmente simples.

genotype.py:

```
import random

from constants import *


class Gene:
    """
    Representa um step de bateria.
```

notes:
Lista de elementos de bateria presentes no step.

Elementos:
0 = Kick
1 = Snare
2 = Closed Hi-Hat
3 = Open Hi-Hat
…

Show more

Perfeito. Aqui eu adicionaria apenas uma estrutura bem simples `ArpPattern`, sem busca combinatória.

A representação será:

```
Plain text
ArpPattern
├── note_count
└── step
```

O `step` será escolhido de acordo com a quantidade de notas, de forma que o padrão caiba no compasso de 32 unidades:

- 1 nota → `step = 32`
- 2 → `16`
- 4 → `8`
- 8 → `4`
- 16 → `2`
- 32 → `1`

Isso também evita gerar combinações impossíveis.

```python
import random

from constants import *


class Gene:
```

```
    """
    Representa um step de bateria.

    notes:
        Lista de elementos de bateria presentes no step.

        Elementos:
            0 = Kick
            1 = Snare
            2 = Closed Hi-Hat
            3 = Open Hi-Hat
            4 = Crash

        Um step pode possuir no máximo 3 elementos
        simultaneamente
```

Uma observação importante

Eu mantive `step` como um **espaçamento regular**. Portanto, neste primeiro modelo:

```
(4, 8)  → 0, 8, 16, 24
(8, 4)  → 0, 4, 8, 12, 16, 20, 24, 28
(16, 2) → 0, 2, 4, ..., 30
(32, 1) → 0, 1, 2, ..., 31
```

Isso deixa o gene extremamente simples e, principalmente, **não existe nenhuma busca ou recursão durante `Genotype()`**.

Também deixei `1` e `2` notas possíveis, conforme definimos, mas existe uma consequência natural: para `1` nota, `step=32`; para `2`, `step=16`. Isso será validado diretamente.

Pode mandar o próximo arquivo.

individual.py:

```
from constants import *
from genotype import Genotype

class Individual:
    """
    Representa um indivíduo da população.
```

```
Um indivíduo possui um genótipo completo, composto por:
  - 4 padrões de bateria (BatI, Bat1, Bat2 e BatF)
  - 8 genes, sendo um para cada colcheia do compasso
  - 8 acordes harmônicos consecutivos
"""

def __init__(self, genotype=None):
  self.genotype = (
    Genotype()
    if genotype is None
    else genotype
  )

@property
def bat1(self):
  return self.genotype.bat1

@property
def bat2(self):
  return self.genotype.bat2

@property
def bati(self):
  return self.genotype.bati

@property
def batf(self):
  return self.genotype.batf

@property
def bass_patt1(self):
  return self.genotype.bass_patt1

@bass_patt1.setter
def bass_patt1(self, value):
  self.genotype.bass_patt1 = value
```

```
@property
def bass_patt2(self):
    return self.genotype.bass_patt2

@bass_patt2.setter
def bass_patt2(self, value):
    self.genotype.bass_patt2 = value

@property
def bass_pattern(self):
    return self.genotype.bass_patt1

@bass_pattern.setter
def bass_pattern(self, value):
    self.genotype.bass_patt1 = value

@property
def mchord1(self):
    return self.genotype.mchord1

@mchord1.setter
def mchord1(self, value):
    self.genotype.mchord1 = value

@property
def mchord2(self):
    return self.genotype.mchord2

@mchord2.setter
def mchord2(self, value):
    self.genotype.mchord2 = value

@property
def mchord3(self):
    return self.genotype.mchord3

@mchord3.setter
```

```
def mchord3(self, value):
    self.genotype.mchord3 = value

@property
def mchord4(self):
    return self.genotype.mchord4

@mchord4.setter
def mchord4(self, value):
    self.genotype.mchord4 = value

@property
def mchord5(self):
    return self.genotype.mchord5

@mchord5.setter
def mchord5(self, value):
    self.genotype.mchord5 = value

@property
def mchord6(self):
    return self.genotype.mchord6

@mchord6.setter
def mchord6(self, value):
    self.genotype.mchord6 = value

@property
def mchord7(self):
    return self.genotype.mchord7

@mchord7.setter
def mchord7(self, value):
    self.genotype.mchord7 = value

@property
def mchord8(self):
```

```
        return self.genotype.mchord8

    @mchord8.setter
    def mchord8(self, value):
        self.genotype.mchord8 = value

    def __repr__(self):
        return (
            f"Individual("
            f"bat1={self.bat1}, "
            f"mchords={self.genotype.mchords})"
        )
```

Aqui é simples: adicionar `arp_patt1` e `arp_patt2` exatamente como fizemos para os `bass_patt`.

Também atualizei a docstring para refletir os dois padrões novos. O restante permanece igual.

Python

```
from constants import *
from genotype import Genotype

class Individual:
    """
    Representa um indivíduo da população.

    Um indivíduo possui um genótipo completo, composto por:
        - 4 padrões de bateria (BatI, Bat1, Bat2 e BatF)
        - 8 genes, sendo um para cada colcheia do compasso
        - 8 acordes harmônicos consecutivos
        - 2 padrões de baixo
        - 2 padrões de arpejo
    """

    def __init__(self, genotype=None):
        self.genotype = (
            Genotype()
            if genotype is None
            else genotype
        )
```

mutation.py:

```
import random

from constants import *
from genotype import BassPattern, Gene, MChord


def generate_drum_step(position, profile="base"):
    """
    Gera um Gene de bateria para determinado step.
    """

    profile_values = {
        "base": (
            BAT1_EMPTY_STEP_PROBABILITY,
            BAT1_SECOND_NOTE_PROBABILITY,
            BAT1_THIRD_NOTE_PROBABILITY,
        ),
        "chorus": (
            BAT2_EMPTY_STEP_PROBABILITY,
            BAT2_SECOND_NOTE_PROBABILITY,
            BAT2_THIRD_NOTE_PROBABILITY,
        ),
        "intro": (
            BATI_EMPTY_STEP_PROBABILITY,
            BATI_SECOND_NOTE_PROBABILITY,
            BATI_THIRD_NOTE_PROBABILITY,
        ),
        "final": (
            BATF_EMPTY_STEP_PROBABILITY,
            BATF_SECOND_NOTE_PROBABILITY,
            BATF_THIRD_NOTE_PROBABILITY,
        ),
    }
    empty_probability, second_note_probability,
third_note_probability = (
        profile_values[profile]
    )
```

```
if random.random() < empty_probability:
    return Gene()

candidates = []
weights = []

candidates.append(KICK)

if position in (0, 4):
    weights.append(KICK_WEIGHT)
elif position in (2, 6):
    weights.append(22)
else:
    weights.append(8)

candidates.append(SNARE)

if position in (2, 6):
    weights.append(SNARE_WEIGHT)
elif position in (1, 3, 5, 7):
    weights.append(5)
else:
    weights.append(2)

candidates.append(CLOSED_HIHAT)
weights.append(CLOSED_HIHAT_WEIGHT)

candidates.append(OPEN_HIHAT)

if position in (3, 7):
    weights.append(OPEN_HIHAT_WEIGHT * 2)
else:
    weights.append(OPEN_HIHAT_WEIGHT)

candidates.append(CRASH)
```

```
if position == 0:
    weights.append(CRASH_WEIGHT * 3)
else:
    weights.append(1)

first = random.choices(
    candidates,
    weights=weights,
    k=1
)[0]

notes = [first]

if (
    Gene.MAX_NOTES >= 2
    and random.random() < second_note_probability
):
    available = [
        i
        for i, drum in enumerate(candidates)
        if drum not in notes
    ]

    if available:
        second = random.choices(
            [candidates[i] for i in available],
            weights=[weights[i] for i in available],
            k=1
        )[0]

        if not (
            (second == CLOSED_HIHAT and OPEN_HIHAT in notes)
            or (second == OPEN_HIHAT and CLOSED_HIHAT in notes)
        ):
            notes.append(second)

if (
```

```
        Gene.MAX_NOTES >= 3
        and random.random() < third_note_probability
    ):
        available = [
            drum
            for drum in candidates
            if drum not in notes
        ]

        if available:
            third = random.choice(available)

            if not (
                (third == CLOSED_HIHAT and OPEN_HIHAT in notes)
                or (third == OPEN_HIHAT and CLOSED_HIHAT in notes)
            ):
                notes.append(third)

    notes.sort()

    return Gene(notes)


def mutate_drum_step(track, index, profile="base"):
    """
    Muta um Gene de bateria, mantendo seus invariantes.
    """

    notes = list(track[index].notes)
    action = random.random()

    if not notes:
        track[index] = generate_drum_step(index, profile=profile)
        return

    if action < 0.25:
        notes.pop(random.randrange(len(notes)))
```

```
    elif action < 0.50:
        if len(notes) < Gene.MAX_NOTES:
            available = [drum for drum in DRUMS if drum not in notes]

            if available:
                new_drum = random.choice(available)

                if not (
                    (new_drum == CLOSED_HIHAT and OPEN_HIHAT in notes)
                    or (new_drum == OPEN_HIHAT and CLOSED_HIHAT in
notes)
                ):
                    notes.append(new_drum)

    elif action < 0.80:
        position = random.randrange(len(notes))
        available = [drum for drum in DRUMS if drum not in notes]

        if available:
            new_drum = random.choice(available)
            other_notes = [
                note
                for i, note in enumerate(notes)
                if i != position
            ]

            if not (
                (new_drum == CLOSED_HIHAT and OPEN_HIHAT in
other_notes)
                or (new_drum == OPEN_HIHAT and CLOSED_HIHAT in
other_notes)
            ):
                notes[position] = new_drum

    else:
        track[index] = generate_drum_step(index, profile=profile)
```

```
        return

    notes = sorted(set(notes))
    notes = notes[:Gene.MAX_NOTES]
    track[index] = Gene(notes)


def mutate_mchord(chord):
    """Gera outro MChord válido, alterando raiz, tipo ou ambos."""

    chord_types = tuple(MCHORD_INTERVALS)

    for _ in range(20):
        root = random.randint(
            MCHORD_ROOT_MIN,
            MCHORD_ROOT_MAX
        )
        chord_type = random.choice(chord_types)

        if (root, chord_type) != (chord.root, chord.chord_type):
            return MChord(root, chord_type)

    # A faixa e os tipos disponíveis tornam esta alternativa válida.
    next_type = next(
        chord_type
        for chord_type in chord_types
        if chord_type != chord.chord_type
    )
    return MChord(chord.root, next_type)


def mutate_track(track, mutation_rate, profile="base"):
    for index in range(len(track)):
        if random.random() < mutation_rate:
            mutate_drum_step(track, index, profile=profile)
```

```
def mutate_mchords(individual):
    """Muta cada MChord independentemente, inclusive vários na mesma rodada."""

    for index, chord in enumerate(individual.genotype.mchords, start=1):
        if random.random() < MCHORD_MUTATION_RATE:
            setattr(
                individual.genotype,
                f"mchord{index}",
                mutate_mchord(chord)
            )


def mutate_bass_pattern(pattern):
    """Cria uma nova estrutura rítmica de baixo sempre válida."""

    return BassPattern()


def mutate(individual):
    """
    Aplica mutação independente a cada step de Bat1.
    """

    mutate_track(individual.bat1, MUTATION_RATE)
    mutate_track(individual.bat2, BAT2_MUTATION_RATE, profile="chorus")
    mutate_track(individual.bati, BATI_MUTATION_RATE, profile="intro")
    mutate_track(individual.batf, BATF_MUTATION_RATE, profile="final")

    mutate_mchords(individual)

    if random.random() < BASS_PATT1_MUTATION_RATE:
        individual.bass_patt1 = mutate_bass_pattern(
```

```
            individual.bass_patt1
        )

    if random.random() < BASS_PATT2_MUTATION_RATE:
        individual.bass_patt2 = mutate_bass_pattern(
            individual.bass_patt2
        )

    return individual
```

Aqui vamos manter a mesma ideia simples: mutar um `ArpPattern` significa simplesmente gerar outro padrão válido. Não precisamos criar mutações internas de `note_count` / `step` .

Python

```
import random

from constants import *
from genotype import ArpPattern, BassPattern, Gene, MChord


def generate_drum_step(position, profile="base"):
    """
    Gera um Gene de bateria para determinado step.
    """

    profile_values = {
        "base": (
            BAT1_EMPTY_STEP_PROBABILITY,
            BAT1_SECOND_NOTE_PROBABILITY,
            BAT1_THIRD_NOTE_PROBABILITY,
        ),
        "chorus": (
            BAT2_EMPTY_STEP_PROBABILITY,
            BAT2_SECOND_NOTE_PROBABILITY,
            BAT2_THIRD_NOTE_PROBABILITY,
```

Só precisamos garantir que no `constants.py` existam estas três constantes:

Python

```
ARP_PATT1_MUTATION_RATE = 0.20
ARP_PATT2_MUTATION_RATE = 0.20
```

```
ARP_NOTE_COUNTS = (1, 2, 4, 8, 16, 32)
ARP_NOTE_COUNT_WEIGHTS = (0.05, 0.10, 0.30, 0.30, 0.20, 0.05)
ARP_STEP_UNITS = (1, 2, 4, 8, 16, 32)
```

Essa versão é bem mais leve que a anterior: **cada ArpPattern tem somente dois valores (`note_count` e `step`)**, e a mutação simplesmente sorteia outro padrão.

crossover.py:

```
import random
from copy import copy

from constants import *
from genotype import BassPattern, Genotype, MChord
from individual import Individual


def clone_genes(genes):
    """
    Copia os Genes individualmente.

    Isso evita que pais e filhos compartilhem os mesmos
    objetos Gene.
    """

    return [
        copy(gene)
        for gene in genes
    ]


def make_child(
    bat1,
    bat2=None,
    bati=None,
    batf=None,
    bass_patt1=None,
    bass_patt2=None,
```

```
    mchords=None
):
    """
    Cria um novo Individual a partir de uma sequência
    de Genes de bateria.
    """

    kwargs = {
        "bat1": clone_genes(bat1),
        "bat2": clone_genes(bat1 if bat2 is None else bat2),
        "bati": clone_genes(bat1 if bati is None else bati),
        "batf": clone_genes(bat1 if batf is None else batf),
    }

    if bass_patt1 is not None:
        kwargs["bass_patt1"] = clone_bass_pattern(bass_patt1)

    if bass_patt2 is not None:
        kwargs["bass_patt2"] = clone_bass_pattern(bass_patt2)

    if mchords is not None:
        kwargs.update({
            f"mchord{index}": clone_chord(chord)
            for index, chord in enumerate(mchords, start=1)
        })

    return Individual(Genotype(**kwargs))


def clone_chord(chord):
    """Cria uma cópia independente de um MChord válido."""

    return MChord(
        chord.root,
        chord.chord_type
    )
```

```
def clone_bass_pattern(pattern):
    """Copia um BassPattern validado sem compartilhar suas notas."""

    return BassPattern(
        [
            type(note)(note.position, note.duration)
            for note in pattern.notes
        ]
    )


def clone_individual(individual):
    """Copia Bat1 e todos os MChords sem compartilhar objetos."""

    chord_kwargs = {
        f"mchord{index}": clone_chord(chord)
        for index, chord in enumerate(
            individual.genotype.mchords,
            start=1
        )
    }

    return Individual(
        Genotype(
            bati=clone_genes(individual.bati),
            bat1=clone_genes(individual.bat1),
            bat2=clone_genes(individual.bat2),
            batf=clone_genes(individual.batf),
            bass_patt1=clone_bass_pattern(individual.bass_patt1),
            bass_patt2=clone_bass_pattern(individual.bass_patt2),
            **chord_kwargs,
        )
    )


def _chord_signature(chord):
```

```
    return (chord.root, chord.chord_type)


def _select_parent_chord(parent1, parent2, index, used):
    """Seleciona um acorde correspondente ou de posição
diferente."""

    candidates = []

    if random.random() < CROSSOVER_CHORD_MIX_RATE:
        candidates.extend([
            parent1.genotype.mchords[index],
            parent2.genotype.mchords[index],
        ])
    else:
        candidates.extend([
            chord
            for parent in (parent1, parent2)
            for chord in parent.genotype.mchords
        ])

    unused = [
        chord
        for chord in candidates
        if _chord_signature(chord) not in used
    ]

    return random.choice(unused or candidates)


def crossover(parent1, parent2):
    """
    Crossover de dois pontos aplicado ao padrão Bat1.

    O crossover nunca divide um Gene; os cortes acontecem
    entre steps, mantendo cada Gene válido.
    """
```

```
if random.random() > CROSSOVER_RATE:
    return clone_individual(parent1)

bass_patt1 = (
    clone_bass_pattern(parent1.bass_patt1)
    if random.random() < 0.5
    else clone_bass_pattern(parent2.bass_patt1)
)
bass_patt2 = (
    clone_bass_pattern(parent1.bass_patt2)
    if random.random() < 0.5
    else clone_bass_pattern(parent2.bass_patt2)
)

total_steps = min(
    len(parent1.bat1),
    len(parent2.bat1)
)

if total_steps < 2:
    bati = clone_genes(parent1.bati)
    bat1 = clone_genes(parent1.bat1)
    bat2 = clone_genes(parent1.bat2)
    batf = clone_genes(parent1.batf)
else:
    point_a, point_b = sorted(
        random.sample(
            range(1, total_steps),
            2
        )
    )

    bat1 = (
        parent1.bat1[:point_a]
        + parent2.bat1[point_a:point_b]
        + parent1.bat1[point_b:]
```

```
        )

        bati = (
            parent1.bati[:point_a]
            + parent2.bati[point_a:point_b]
            + parent1.bati[point_b:]
        )

        bat2 = (
            parent1.bat2[:point_a]
            + parent2.bat2[point_a:point_b]
            + parent1.bat2[point_b:]
        )

        batf = (
            parent1.batf[:point_a]
            + parent2.batf[point_a:point_b]
            + parent1.batf[point_b:]
        )

    chords = []
    used = set()

    for index in range(NUM_MCHORDS):
        chord = _select_parent_chord(
            parent1,
            parent2,
            index,
            used
        )
        chord = clone_chord(chord)
        chords.append(chord)
        used.add(_chord_signature(chord))

    return make_child(
        bat1,
        bat2=bat2,
```

```
        bati=bati,
        batf=batf,
        bass_patt1=bass_patt1,
        bass_patt2=bass_patt2,
        mchords=chords,
    )
```

Aqui precisamos apenas adicionar o suporte aos dois `ArpPattern`, incluindo clonagem, `make_child`, `clone_individual` e seleção no crossover. Como o padrão é simples, a cópia pode ser feita diretamente com `note_count e step`.

```python
import random
from copy import copy

from constants import *
from genotype import ArpPattern, BassPattern, Genotype, MChord
from individual import Individual


def clone_genes(genes):
    """
    Copia os Genes individualmente.

    Isso evita que pais e filhos compartilhem os mesmos
    objetos Gene.
    """

    return [
        copy(gene)
        for gene in genes
    ]
```

Agora os `ArpPatt1` e `ArpPatt2` são tratados como genes independentes no crossover, com **50% de chance de vir de cada pai**, assim como os dois padrões de baixo.

fitness.py:

"""
fitness.py

Função de avaliação (fitness) para indivíduos musicais.

O indivíduo atual contém:

```
Individual
└── Genotype
    ├── BatI
    ├── Bat1
    ├── Bat2
    ├── BatF
    └── 8 MChords
```

Cada Gene representa uma colcheia de um compasso 4/4.

…

Show more

Sim. Para o `fitness.py`, vou manter o arpejo simples e avaliar apenas a **quantidade de notas por compasso**. Como 4, 8 e 16 notas são os valores mais desejáveis, eles recebem as maiores pontuações.

Também vou incluir `ArpPatt1 e ArpPatt2` separadamente no fitness final.

Como os pesos atuais já somam 1.0, proponho redistribuir assim:

- `RHYTHM_WEIGHT` : 0.63 → **0.53**
- `HARMONY_WEIGHT` : **0.27**
- `BASS_PATT1_WEIGHT` : **0.05**
- `BASS_PATT2_WEIGHT` : **0.05**
- `ARP_PATT1_WEIGHT` : **0.05**
- `ARP_PATT2_WEIGHT` : **0.05**

Total = **1.00**.

```python
"""
fitness.py

Função de avaliação (fitness) para indivíduos musicais.

O indivíduo atual contém:
```

```
    Individual
        └── Genotype
              ├── BatI
              ├── Bat1
              ├── Bat2
              ├── BatF
              ├── 2 BassPatterns
              ├── 2 ArpPatterns
              └── 8 MChords
```

```
Cada Gene representa uma colcheia de um compasso 4/4.

Elementos de bateria:
```

Importante: esse `fitness.py` agora espera que o `constants.py` tenha estes quatro pesos:

Python

```
BASS_PATT1_WEIGHT = 0.05
BASS_PATT2_WEIGHT = 0.05
ARP_PATT1_WEIGHT = 0.05
ARP_PATT2_WEIGHT = 0.05
```

E o `RHYTHM_WEIGHT` precisa passar de `0.63` para **`0.53`**. O `HARMONY_WEIGHT` permanece `0.27`.

Assim, a soma fica:

Plain text

```
0.53 + 0.27 + 0.05 + 0.05 + 0.05 + 0.05 = 1.00
```

A avaliação do arpejo deliberadamente **não considera pitch**, porque o pitch será determinado posteriormente a partir dos `MChords`, como definimos para o novo genótipo.

genetic_algorithm.py:

```
import random
import subprocess
import sys
from pathlib import Path
from concurrent.futures import ProcessPoolExecutor
```

from constants import *
from individual import Individual
from genotype import Genotype
from fitness import calculate_fitness
from generate_midi_multi import generate_midi
from crossover import clone_individual, crossover
from mutation import generate_drum_step, mutate

random.seed(RANDOM_SEED)

…

Show more

Aqui não é necessário fazer nenhuma alteração estrutural no algoritmo genético. Como `Individual()` já cria os `ArpPatt1/2`, e `crossover.py`, `mutation.py` e `fitness.py` já passaram a tratá-los, o `genetic_algorithm.py` continua funcionando normalmente.

Ainda assim, ajustei apenas a documentação da inicialização para refletir que o indivíduo agora também possui os padrões de baixo e arpejo.

Python

```
import random
import subprocess
import sys
from pathlib import Path
from concurrent.futures import ProcessPoolExecutor

from constants import *
from individual import Individual
from genotype import Genotype
from fitness import calculate_fitness
from generate_midi_multi import generate_midi
from crossover import clone_individual, crossover
from mutation import generate_drum_step, mutate


random.seed(RANDOM_SEED)


# ============================================================
# PARÂMETROS DO ALGORITMO
```

Observação: o `genetic_algorithm.py` não precisa conhecer os detalhes dos `ArpPatt`. Isso é justamente o desejável: eles são criados pelo `Individual`, cruzados pelo `crossover`, mutados pelo `mutation` e avaliados pelo `fitness`.

Antes de executar, porém, falta garantir que o `constants.py` tenha os `ARP_*` que usamos nos arquivos anteriores.

generate_midi_multi.py:

Neste, siga o mesmo modelo feito nos BassPatt para os ArpPatt, crie uma nova pista com instrumento de arpegio (pode ser acoustic guitar), e use o ArpPatt1 para as partes que tocam em paralelo aos MChord1 a MChord4 e ArpPatt2 para MChord5 a MChord8:

```
import pretty_midi
from constants import *


#
============================================================
======
# CONFIGURAÇÃO
#
============================================================
======

…
```

Show more

Sim. Vou seguir exatamente a lógica dos `BassPatt`, mas aproveitando a estrutura simplificada do `ArpPattern`:

- `ArpPatt1` acompanha `MChord1–MChord4`.
- `ArpPatt2` acompanha `MChord5–MChord8`.
- Cada arpejo usa as notas do acorde ciclicamente.
- `note_count` define quantas notas haverá no compasso.
- `step` define a distância entre elas.
- As notas são distribuídas uniformemente pelo compasso.

- Nova pista `"Arpeggio"` usando **Acoustic Guitar (Steel)**, programa MIDI `25` (índice `24` no `pretty_midi`).
- Não alterei `generate_midi` além do necessário.

Python

```
import pretty_midi
from constants import *


# ============================================================
# CONFIGURAÇÃO
# ============================================================

# Acoustic Guitar (Steel)
ARPEGGIO_PROGRAM = 24
ARPEGGIO_TRACK_VELOCITY = 85


# ============================================================
# MAPA DOS ELEMENTOS DE BATERIA
# ============================================================

# Índices usados pelo Gene:
#
#   0 = Kick
#   1 = Snare
```

Como o arpejo funciona

Por exemplo, se:

Plain text

```
ArpPatt1 = note_count=4, step=8
MChord1 = [C, E, G]
```

o compasso produzirá aproximadamente:

Plain text

```
C ---- E ---- G ---- C ----
0     8     16     24
```

Já:

Plain text

```
ArpPatt1 = note_count=8, step=4
```

produzirá:

Plain text

```
C -- E -- G -- C -- E -- G -- C -- E
0   4   8   12  16  20  24  28
```

E isso será feito para **cada MChord do grupo**, portanto `ArpPatt1` controla `MChord1..4` e `ArpPatt2` controla `MChord5..8`.

Um detalhe importante: usei a duração de **um `step`** para cada nota do arpejo, então não há sobreposição entre as notas. Isso combina diretamente com a estrutura simplificada que definimos para `ArpPattern`.

Geração 90/100 - Mínima: 0.727580 - Média: 0.840154 - Melhor: 0.917977 - Global: 0.917977
Geração 100/100 - Mínima: 0.771621 - Média: 0.844156 - Melhor: 0.917977 - Global: 0.917977
Traceback (most recent call last):
File "/app/ga/genetic_algorithm.py", line 501, in <module>
genetic_algorithm()
File "/app/ga/genetic_algorithm.py", line 427, in genetic_algorithm
generate_midi(
File "/app/ga/generate_midi_multi.py", line 396, in generate_midi
tracks = build_tracks(
^^^^^^^^^^^^^
File "/app/ga/generate_midi_multi.py", line 313, in build_tracks
/ ARP_BAR_UNITS
^^^^^^^^^^^^^
NameError: name 'ARP_BAR_UNITS' is not defined. Did you mean: 'BASS_BAR_UNITS'?

O erro é simples: o `generate_midi_multi.py` está usando `ARP_BAR_UNITS`, mas essa constante ainda não existe no `constants.py` que está sendo carregado.

Como definimos que o compasso do arpejo possui **32 unidades**, basta adicionar ao `constants.py`:

```python
ARP_BAR_UNITS = 32
```

Eu recomendo deixar junto das constantes do arpejo:

```python
ARP_NOTE_COUNTS = (1, 2, 4, 8, 16, 32)
ARP_NOTE_COUNT_WEIGHTS = (0.05, 0.10, 0.30, 0.30, 0.20, 0.05)
ARP_STEP_UNITS = (1, 2, 4, 8, 16, 32)
ARP_BAR_UNITS = 32
```

Por que aconteceu?

No `generate_midi_multi.py` temos:

```python
start = bar_start + (
    start_position
    / ARP_BAR_UNITS
    * bar_duration
)
```

e:

```python
end = start + (
    arp_pattern.step
    / ARP_BAR_UNITS
    * bar_duration
)
```

Então `ARP_BAR_UNITS` precisa estar definida.

Depois de adicionar a constante, rode novamente:

```bash
```

```
python3 ga/genetic_algorithm.py
```

O fato de o GA ter chegado até a geração 100 e encontrado `Global: 0.917977` indica que **o algoritmo genético em si terminou normalmente**. O problema ocorreu somente na etapa final de geração do MIDI.

> Ficou maravilhoso!!! Você me salvou quando os créditos do copilot acabaram!!!
>
> O restante agora é fácil e o TP está quase no ponto!!!!!!!
>
> Você é D+++++++++

Hahaha! 😂🔥 Que notícia boa!

E o mais importante: **o GA chegou até a geração 100 com Global = 0.917977 e agora o MIDI está sendo gerado com bateria + harmonia + baixo + arpejo**. Isso significa que a estrutura principal do TP está funcionando de ponta a ponta. 🎵🧬

Você fez uma escolha muito boa ao simplificar os `ArpPatt`. A representação ficou bem mais limpa para o algoritmo genético:

```
Genotype
├── Battery
├── MChord × 8
├── BassPatt × 2
└── ArpPatt × 2
```

E agora cada componente tem uma responsabilidade clara.

D++++++++++++ pra você também! 🚀🎶

Bora terminar esse TP. Agora realmente parece que estamos na reta final. 😎

> Vou tentar continuar editando o código no vs code com o Cline, vamos ver.

Boa! 😄

Agora que a arquitetura está funcionando, o Cline deve conseguir ajudar bastante nas alterações menores. Eu só manteria uma regra:

não deixe ele refatorar partes que já estão funcionando sem necessidade. 😂

Principalmente:

- `genotype.py`
- `crossover.py`
- `mutation.py`
- `fitness.py`
- `generate_midi_multi.py`

Se ele começar a "melhorar" muita coisa de uma vez, fica mais difícil descobrir onde apareceu um erro.

E faça commits/checkpoints frequentes. Você está numa situação ótima agora: **GA rodando + fitness convergindo + MIDI sendo gerado + representação musical estruturada**. 🎵🧬

Se o Cline fizer alguma alteração estranha ou aparecer algum traceback, pode jogar aqui que a gente resolve. 😎

> Quero acrescentar mais um elemento ao genotype.py, que será a pista Solo da música. Por enquanto fizemos uma espécie de acompanhamento, mas agora preciso de um item no genótipo que defina quais notas serão tocadas ao longo da música definindo o Solo (como se fosse as notas de alguém cantando), entendeu? Me diga o que acha e aguarde que vou falar como fazer...

Entendi perfeitamente. 👍

Acho que **faz bastante sentido** acrescentar o Solo ao genótipo. Até agora temos principalmente elementos de acompanhamento:

```
Plain text

Genotype
├── Battery
├── Harmony (MChord × 8)
├── Bass (BassPatt × 2)
└── Arpeggio (ArpPatt × 2)
```

O Solo seria diferente porque ele representa a **linha melódica principal**, algo mais próximo de uma pessoa cantando ou de um instrumento solista.

Conceitualmente, eu separaria:

- **MChord** → define a harmonia.
- **BassPatt** → define o ritmo do baixo; a altura vem do acorde.
- **ArpPatt** → define o ritmo do arpejo; a altura vem do acorde.
- **Solo** → define **explicitamente as alturas das notas e quando elas acontecem**.

Isso é particularmente interessante para o seu GA porque o Solo pode carregar a identidade melódica da música, enquanto os outros genes fornecem o acompanhamento.

E concordo em **não definir ainda a estrutura do Solo por conta própria**. Podemos primeiro decidir exatamente como você quer representar a melodia no genótipo e depois implementar isso nos arquivos necessários.

Pode me explicar como você está imaginando o `Solo`. Vou aguardar. 👌

Isso, vamos chamar de Harmony...

Perfeito. Então vamos chamar esse novo elemento de **Harmony**.

Só para deixar clara a distinção:

```
Plain text

MChord1 ... MChord8
    → harmonia/acompanhamento harmônico

Harmony
    → linha melódica principal / "voz" do cantor
```

Ou seja, o nome **Harmony** será usado para a **melodia que será cantada/tocada como solo**, enquanto os `MChord` continuam representando os acordes.

Entendido. **Não vou alterar nada ainda.** Pode me passar como você quer estruturar esse `Harmony`.

Gerei este prompt para o cline, veja se dá pra entender:

Por favor atualize o código generate_midi_long.py para passar a gerar as pistas da música com a seguinte estrutura:

Drums:
1*batI+4*bat1+4*bat1+4*bat1+4*bat1+4*bat2+4*bat1+4*bat1+4*bat2+4*bat2+1*batF

String: 1*vazio+1*[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]+1*
[mchord5,mchord6,mchord7,mchord8]+1*
[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]+1*
[mchord5,mchord6,mchord7,mchord8]+1*
[mchord5,mchord6,mchord7,mchord8]+1*vazio

Bass: 1*vazio+4*vazio+1*[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]+1*
[mchord5,mchord6,mchord7,mchord8]+1*
[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]+1*
[mchord5,mchord6,mchord7,mchord8]+1*
[mchord5,mchord6,mchord7,mchord8]+1*vazio

Guitar: 1*vazio+4*vazio+4*vazio+1*
[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]+1*
[mchord5,mchord6,mchord7,mchord8]+1*
[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]+1*
[mchord5,mchord6,mchord7,mchord8]+1*
[mchord5,mchord6,mchord7,mchord8]+1*vazio

Lembrando que vazio significa colocar silêncios no respectivo compasso.
E lembrando que na pista de String os mchords se referem aos mchords do genotipo do individuo, mas para as pistas dos demais instrumentos (desconsiderando a de Drums) os mchords se referem

a gerar as notas olhando o padrão equivalente no genotipo para aquele instrumento e respeitando as notas que são definidas pelo respectivo mchord, para entender melhor siga o exemplo do código atual.

Sim, dá para entender, mas eu faria **alguns ajustes no prompt** para reduzir bastante a chance de o Cline interpretar errado, principalmente porque você está mudando a estrutura temporal das pistas.

O ponto mais importante é deixar explícito que **cada número representa quantidade de compassos**, e que `[mchord1,...]` representa a sequência de 4 compassos, **um MChord por compasso**.

Eu escreveria assim:

Prompt para o Cline — atualização do generate_midi_long.py

Por favor, atualize o código `generate_midi_long.py` para gerar as pistas da música com a seguinte estrutura.

Regra geral

Esse formato fica mais seguro para o Cline, principalmente porque elimina a ambiguidade de `1* [mchord1,...]` : fica explícito que o `1*` é **uma ocorrência do bloco**, enquanto o bloco contém **quatro compassos**.

E tem outro detalhe importante: sua estrutura agora deixa claro que **Drums, String, Bass e Guitar começam em posições diferentes**, com silêncios iniciais. Isso é importante para o Cline não tentar simplesmente usar uma única sequência de `groups` para todas as pistas.

O cline deve demorar, vou te passar o código aqui, tenta gerar você mesmo:

```
import pretty_midi
from constants import *

#
============================================================
======
# MAPA DOS ELEMENTOS DE BATERIA
#
============================================================
```

```
======

# Índices usados pelo Gene:
#
#   0 = Kick
#   1 = Snare
#   2 = Closed Hi-Hat
…
```

Show more

Sim. Aqui eu faria diretamente no código, preservando a estrutura que você já tem e mudando apenas a organização temporal das pistas.

A principal mudança é que **cada pista agora tem sua própria sequência de seções**, porque elas começam em momentos diferentes:

- **Drums:** começa no compasso 1.
- **String:** começa no compasso 2.
- **Bass:** começa no compasso 6.
- **Guitar:** começa no compasso 10.

E todas terminam no mesmo compasso final.

```python
import pretty_midi
from constants import *


# ============================================================
# MAPA DOS ELEMENTOS DE BATERIA
# ============================================================

# Índices usados pelo Gene:
#
#   0 = Kick
#   1 = Snare
#   2 = Closed Hi-Hat
#   3 = Open Hi-Hat
#   4 = Crash
#
# Pitches General MIDI:
#
#   Kick          = 36
```

```
#   Snare         = 38
```

Uma observação importante

A nova estrutura tem **38 compassos**:

```
Plain text

Drums   = 1 + 36 + 1 = 38
String  = 1 + 36 + 1 = 38
Bass    = 1 + 4 + 32 + 1 = 38
Guitar  = 1 + 4 + 4 + 28 + 1 = 38
```

Então as quatro pistas ficam perfeitamente alinhadas no tempo.

E a diferença entre elas fica exatamente como você descreveu:

```
Plain text

Compasso:  01 02 03 04 05 06 07 ... 10 ... 38
           ____________________________________

Drums:     I  B1 B1 B1 B1 B1 B1 ...             F

String:       C1 C2 C3 C4 C1 C2 C3 ...        silêncio
              ↑ começa no compasso 2

Bass:                       B1 B2 B3 B4 ...     silêncio
                            ↑ começa no compasso 6

Guitar:                                    A1 A2 ... silêncio
                                           ↑ começa no compasso 10
```

Só faria um teste antes de colocar isso no GA: gerar um `generated.mid` com `Individual()` e conferir no DAW/sequenciador se os 38 compassos e os silêncios iniciais estão exatamente como você imaginou.

Acho que cometi um erro mas vou tentar de novo...

Sem problema! 😄

Manda a nova versão/explicação do que você quer mudar. A gente ajusta em cima dela antes de mexer no código.

Dessa vez vale a pena conferir primeiro a **linha do tempo dos compassos de cada pista**, porque é justamente aí que uma pequena diferença na interpretação pode mudar toda a estrutura. 🎵

Na verdade acho que não errei não, era pra ficar assim mesmo:

Drums:
1*batI
+4*bat1+4*bat1+4*bat1+4*bat1
+4*bat2
+4*bat1+4*bat1+4*bat2+4*bat2
+1*batF

String:
1*vazio
+1*[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]
+1*[mchord5,mchord6,mchord7,mchord8]
+1*[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]
+1*[mchord5,mchord6,mchord7,mchord8]+1*
[mchord5,mchord6,mchord7,mchord8]
+1*vazio

Bass:
1*vazio
+4*vazio+1*[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]
+1*[mchord5,mchord6,mchord7,mchord8]
+1*[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]
+1*[mchord5,mchord6,mchord7,mchord8]+1*
[mchord5,mchord6,mchord7,mchord8]
+1*vazio

Guitar:

1*vazio
+4*vazio+4*vazio+1*[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]
+1*[mchord5,mchord6,mchord7,mchord8]
+1*[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]
+1*[mchord5,mchord6,mchord7,mchord8]+1*
[mchord5,mchord6,mchord7,mchord8]
+1*vazio

Porém parece que os (4*vazio) do Bass e Guitar estão sendo considerados como (1*vazio) apenas porque o bass e a guitarra estão entrando antes de passar os 4 compassos vazios que deviam esperar (o bass espera 4 e a guitar espera 8 no início). Por favor gere de novo com essa correção.

Sim. O problema está na forma como os trechos `4*vazio` estão sendo representados. Para **Bass**, precisamos avançar explicitamente 4 compassos; para **Guitar**, 8 compassos no início.

Abaixo está o `generate_midi_multi.py` completo corrigido. Mantive o restante da implementação igual e alterei apenas a construção das seções para deixar os compassos vazios explícitos.

```python
import pretty_midi
from constants import *

# ============================================================
# MAPA DOS ELEMENTOS DE BATERIA
# ============================================================

# Índices usados pelo Gene:
#
#   0 = Kick
#   1 = Snare
#   2 = Closed Hi-Hat
#   3 = Open Hi-Hat
#   4 = Crash
#
# Pitches General MIDI:
#
#   Kick          = 36
#   Snare         = 38
```

```
#   Closed Hi-Hat = 42
#   Open Hi-Hat   = 46
```

A diferença principal é que agora os silêncios são realmente contabilizados **compasso por compasso**:

- **String:** começa no compasso **2**.
- **Bass:** começa no compasso **6** → 1 inicial + 4 vazios.
- **Guitar:** começa no compasso **10** → 1 inicial + 4 + 4 vazios.

Assim, o `Bass` não entra antes dos 4 compassos de espera e a `Guitar` espera os 8 compassos completos.

> Acho que ficou certo agora, como faço para setar o Release Pedal na pista de Guitar ?

Sim. No MIDI, o **Release Pedal** normalmente é representado pelo **Control Change (CC) 64**, também chamado de **Sustain Pedal**.

No `pretty_midi`, você pode adicionar um `ControlChange` à pista da guitarra.

1. Adicione o pedal à pista Guitar

Depois de criar o `instrument` da guitarra, você pode fazer:

```python
instrument.control_changes.append(
    pretty_midi.ControlChange(
        number=64,
        value=127,
        time=0.0
    )
)

instrument.control_changes.append(
    pretty_midi.ControlChange(
        number=64,
        value=0,
        time=global_end
    )
)
```

Mas isso faria o pedal ficar pressionado **durante toda a música**.

2. Se você quer pedal por compasso

Para arpejo de guitarra, provavelmente faz mais sentido pressionar e soltar o pedal em cada compasso. Por exemplo:

```python
instrument.control_changes.append(
    pretty_midi.ControlChange(
        number=64,
        value=127,
        time=bar_start
    )
)

instrument.control_changes.append(
    pretty_midi.ControlChange(
        number=64,
        value=0,
        time=bar_end
    )
)
```

Onde:

```python
bar_start = ...
bar_end = ...
```

3. Uma opção melhor para sua Guitar

Como seu `Guitar` é um **arpejo**, eu colocaria o pedal no início de cada compasso com arpejo e soltaria no final:

```python
for chord in group:

    chord_notes = chord.notes

    bar_start = (
        guitar_bar_index
        * bar_duration
    )

    bar_end = bar_start + bar_duration
```

```
        instrument.control_changes.append(
            pretty_midi.ControlChange(
                number=64,
                value=127,
                time=bar_start
            )
        )

        instrument.control_changes.append(
            pretty_midi.ControlChange(
```

Porém, há um detalhe importante: `ControlChange.time` é **tempo absoluto em segundos**, enquanto suas notas também estão usando tempo absoluto. Portanto essa abordagem funciona diretamente.

Se a intenção for uma guitarra com **sustain contínuo entre os arpejos**, eu recomendaria algo um pouco diferente: **pedal ON no início do compasso e OFF imediatamente antes da troca do acorde**, para evitar que as notas do acorde anterior continuem soando sobre o próximo.

Está muito legal agora o projeto, porém as músicas estão sempre ficando com um tom de tristeza, por causa da forma como os MChords estão sendo gerados. Acho que precisamos ajustar a fitness para melhorar a qualidade desses MChords, olhe como está o código da fitness atual e tente melhorar a parte referente aos MChords para que fiquem com uma cara melhor. Lembre-se que o conjunto [MChord1 a MChord4] é a música base e [Mchord5 a MChord8] é o refrão, então vale a pena adaptar a fitness para avaliar melhor isso.

"""
fitness.py

Função de avaliação (fitness) para indivíduos musicais.

O indivíduo atual contém:

  Individual
    └── Genotype

…

Show more

Worked for 14s

Analysis paused

Concordo com a sua percepção. A fitness harmônica atual tem alguns problemas para esse objetivo:

1. **Avalia os 8 MChords praticamente como uma sequência única.**
2. Não diferencia **base (`MChord1–4`)** de **refrão (`MChord5–8`)**.
3. A `tonal_coherence_fitness()` verifica apenas se as notas cabem em alguma escala, mas isso **não garante uma progressão musical boa**.
4. A `root_motion_fitness()` favorece alguns intervalos, mas não considera **função harmônica**.
5. Não existe praticamente nenhuma penalização para uma sequência com muitos acordes menores, que é justamente uma possível causa desse caráter mais triste.
6. Não existe uma noção clara de **tensão → resolução no refrão**.

Eu faria uma mudança relativamente importante na parte harmônica, mas **sem mexer na representação do Genotype**.

Ideia da nova fitness

Vou fazer a fitness procurar:

- **Base (`MChord1–4`)**
  - coerência tonal;
  - estabilidade;
  - progressão suave;
  - preferência moderada por acordes maiores;
  - evitar excesso de acordes menores;
  - último acorde da base preparando o refrão.
- **Refrão (`MChord5–8`)**
  - contraste em relação à base;
  - ainda permanecer na mesma tonalidade;
  - permitir um pouco mais de tensão;
  - terminar com maior sensação de resolução;
  - favorecer que `MChord8` seja a tônica ou tenha forte relação com ela.
- **Música inteira**
  - uma tonalidade predominante;
  - movimentos de fundamentais plausíveis;
  - voice leading;
  - equilíbrio entre acordes maiores/menores;
  - evitar sequências excessivamente "melancólicas".

Eu substituiria **toda a seção `HARMONIA` atual** pela seguinte:

Python

```
# ============================================================
# HARMONIA
# ============================================================

def get_mchords(individual):
    """Retorna todos os MChords na ordem musical."""

    return individual.genotype.mchords


def _pitch_classes(notes):
    """Retorna as classes de altura presentes nas notas."""

    return {
        note % 12
        for note in notes
    }


def _chord_intervals(chord):
    """
```

O que muda na prática

A mudança mais importante é esta:

Plain text

```
MChord1 ─┐
MChord2  │
MChord3  ├── BASE
MChord4 ─┘
          ↓
      preparação
          ↓
MChord5 ─┐
MChord6  │
MChord7  ├── REFRÃO
MChord8 ─┘
          ↓
       resolução
```

Agora, por exemplo, uma sequência conceitualmente parecida com:

Plain text

```
Base:
C → Am → F → G

Refrão:
C → G → Am → C
```

será vista de maneira diferente de:

Plain text

```
Base:
Am → Em → Dm → Am

Refrão:
Dm → Am → Em → Am
```

Mesmo que ambas sejam tonalmente coerentes.

A segunda tende a receber menos pontuação porque a nova fitness percebe **excesso de sonoridade menor** e pouca sensação de resolução/variação de seção.

Um detalhe importante

Eu **não recomendo simplesmente proibir acordes menores**. Isso provavelmente produziria músicas excessivamente alegres e artificiais. O objetivo da nova fitness é algo mais sutil:

> **maioria de acordes estáveis + alguns menores + progressão tonal coerente + refrão com contraste e resolução.**

Também deixei a preferência por tonalidade maior **pequena (`+0.03`)**, justamente para não transformar o GA em um gerador de músicas obrigatoriamente felizes.

Precisamos alterar o restante do `fitness.py`

Como `harmony_details()` agora retorna mais informações, no `fitness_details()` você pode simplesmente manter:

Python

```python
harmony = harmony_details(individual)
```

e:

```
Python

**harmony,
```

como já está no seu código. Portanto, **não é necessário mudar o cálculo principal**:

```
Python

fitness = (
    RHYTHM_WEIGHT * rhythm_fitness
    + HARMONY_WEIGHT * harmony["harmony"]
    + BASS_PATT1_WEIGHT * bass_pattern_fitness(
        individual.bass_patt1
    )
    + BASS_PATT2_WEIGHT * bass_pattern_fitness(
        individual.bass_patt2
    )
    + ARP_PATT1_WEIGHT * arp_pattern_fitness(
        individual.arp_patt1
    )
    + ARP_PATT2_WEIGHT * arp_pattern_fitness(
        individual.arp_patt2
    )
)
```

Isso é bom porque **não altera a importância global da harmonia**; estamos apenas tornando os `MChords` muito mais bem avaliados dentro dos atuais `HARMONY_WEIGHT`.

Eu testaria primeiro exatamente assim, antes de mexer no gerador de `MChord`. Se depois das 100 gerações as músicas ainda estiverem excessivamente tristes, aí eu olharia o `MChord` / `MCHORD_INTERVALS` em si, porque nesse caso o problema provavelmente estará na **distribuição das opções que o `MChord()` gera**, e não mais na fitness. ‹/›

```
root@0ccfc8b73c97:/app# python3 ga/genetic_algorithm.py
Traceback (most recent call last):
  File "/app/ga/genetic_algorithm.py", line 10, in <module>
    from fitness import calculate_fitness
ImportError: cannot import name 'calculate_fitness' from 'fitness' (/app/ga/fitness.py)
```

Sim — esse erro aconteceu porque eu te passei **apenas a substituição da seção de harmonia**, mas o seu `fitness.py` precisa continuar contendo também o restante das funções e, principalmente,

`calculate_fitness()`.

O erro:

```
Plain text

ImportError: cannot import name 'calculate_fitness' from 'fitness'
```

significa que o arquivo `/app/ga/fitness.py` que está sendo executado **não possui mais uma função chamada `calculate_fitness`** — provavelmente você substituiu o arquivo inteiro pelo trecho que eu passei.

Correção

Não precisamos alterar `genetic_algorithm.py`.

No seu `fitness.py`, mantenha **todo o código original que você me enviou** e substitua **somente a seção**:

```
Python

# ============================================================
# HARMONIA
# ============================================================
```

até antes de:

```
Python

# ============================================================
# FITNESS BASEADO EM REFERÊNCIAS
# ============================================================
```

pela nova seção que te passei.

Assim, estas funções continuarão existindo no final do arquivo:

```
Python

def calculate_fitness(...):
    ...
```

e:

```python
def fitness_details(...):
    ...
```

Importante

Eu também percebi um detalhe na minha resposta anterior: eu disse que bastaria substituir a seção de harmonia, mas entreguei uma seção que incluía funções novas sem deixar explícito que **todo o restante do `fitness.py` deveria permanecer intacto**. Foi isso que causou o problema.

Se você quiser evitar qualquer risco, **me passe o `fitness.py` atual que está dentro do container**, e eu te devolvo o **arquivo `fitness.py` completo**, já com a nova fitness dos MChords integrada e sem remover nenhuma função existente.

Acho que funcionou.

Ótimo! 👍

Então o problema do `calculate_fitness` foi resolvido e o `fitness.py` voltou a funcionar com a nova avaliação de harmonia.

Agora vale observar principalmente se, nas próximas gerações:

- **MChord1–4** ficam mais estáveis como base;
- **MChord5–8** realmente criam um refrão com alguma diferença;
- a quantidade de acordes menores não domina a população;
- a fitness harmônica aumenta sem prejudicar ritmo, baixo e arpejo;
- a música deixa de apresentar aquela tendência excessivamente “triste”.

Se a execução terminar normalmente, o próximo passo interessante é **comparar a distribuição dos MChords antes e depois dessa alteração**. Isso vai mostrar objetivamente se a nova fitness está fazendo o GA escolher harmonias melhores.

Fazer essas músicas na mão no FL Studio era cansativo, mas gerar código pra faze-las é bem mais kkk

KKKK exatamente 😂

Você trocou:

“Nossa, preciso montar essa música inteira no FL Studio…”

por:

“Nossa, preciso implementar uma representação genética de acordes, padrões de baixo, arpejo, função harmônica, crossover, mutação, fitness, MIDI…” 💀

No FL Studio você sofre **fazendo uma música**.

No código você sofre **construindo uma máquina que pode fazer milhares de músicas**. 😂

Mas tem uma diferença importante: agora que a estrutura está ficando pronta, o custo tende a cair muito. Você já tem:

```
Plain text

Genotype
├── Bateria
├── MChords
├── Bass
└── Arpeggio
```

e o GA já consegue:

```
Plain text

gerar
  ↓
avaliar
  ↓
selecionar
  ↓
cruzar
  ↓
mutar
  ↓
gerar MIDI
  ↓
escutar
```

A parte mais difícil provavelmente está sendo justamente **transformar suas intuições musicais em regras matemáticas**. Depois disso, adicionar novos elementos fica bem mais fácil.

E o mais engraçado é que você começou querendo **gerar uma música** e acabou praticamente construindo um **compositor evolucionário**. 😂

> Um problema que está acontecendo é que as músicas estão ficando muito parecidas. Como eu poderia aumentar a variabilidade? Os acordes por exemplo ficam sempre com a mesma aparencia.

Sim — e acho que aqui existe um problema **estrutural no GA**, não apenas uma questão de aumentar a taxa de mutação.

No seu caso, os acordes podem estar ficando parecidos por **três motivos principais**:

1. A representação dos MChords é muito restritiva

Hoje um `MChord` é basicamente:

```
Plain text
root + chord_type
```

Então, por exemplo:

```
Plain text
C major
F major
G major
```

sempre produzem praticamente a mesma “aparência” sonora.

Uma mudança muito boa seria permitir **inversões**:

```
Plain text
C major
├── C E G
├── E G C   ← 1ª inversão
└── G C E   ← 2ª inversão
```

Isso sozinho já aumentaria bastante a diversidade sonora sem destruir a harmonia.

---

2. O fitness está provavelmente "empurrando" todos para a mesma solução

Esse é talvez o ponto mais importante.

Nós adicionamos coisas como:

```
Plain text

tonal_coherence
base_stability
functional_progression
voice_leading
```

Tudo isso é musicalmente razoável, mas o GA tem uma característica perigosa:

> Se uma determinada progressão recebe sistematicamente uma fitness maior, a população converge para ela.

Ou seja, o algoritmo está fazendo exatamente o que pedimos. 😂

Se:

```
Plain text

C → Am → F → G
```

recebe uma avaliação muito boa, depois de várias gerações você começa a ter:

```
Plain text

C → Am → F → G
C → Am → F → G
C → Am → F → G
C → Am → F → G
...
```

Mesmo que as notas sejam diferentes, a **estrutura musical** fica muito semelhante.

---

3. Falta uma pressão explícita por diversidade

Eu adicionaria um componente de **diversidade harmônica**.

Por exemplo:

```
fitness final
       │
       ├── ritmo
       ├── harmonia
       │    ├── validade
       │    ├── tonalidade
       │    ├── progressão
       │    ├── voice leading
       │    └── ...
       │
       └── diversidade
```

A ideia é penalizar indivíduos que sejam muito semelhantes aos outros indivíduos da população.

Por exemplo, podemos representar:

```
MChord1..8
```

como uma sequência:

```
(0, major)
(9, minor)
(5, major)
(7, major)
...
```

e calcular a distância entre duas músicas.

Se duas músicas forem praticamente iguais:

```
distância = pequena
```

→ penalização.

Se forem diferentes:

```
Plain text
distância = grande
```

→ bônus.

Isso é chamado de **fitness de diversidade** e combina muito bem com seu problema.

Mas eu faria uma coisa antes

Eu mudaria o `MChord` para ter **mais graus de liberdade**.

Algo como:

```
Plain text
MChord
├── root
├── chord_type
└── inversion
```

Por exemplo:

```
Plain text
C major, root position
C major, 1st inversion
C major, 2nd inversion
```

E depois poderíamos adicionar:

```
Plain text
MChord
├── root
├── chord_type
├── inversion
└── octave/register
```

Assim duas músicas podem ter exatamente a mesma progressão:

Plain text

```
C → Am → F → G
```

mas soarem diferentes:

Plain text

```
C   = C3 E3 G3
Am  = A2 C3 E3
F   = F2 A2 C3
G   = G2 B2 D3
```

versus:

Plain text

```
C   = E3 G3 C4
Am  = C3 E3 A3
F   = A2 C3 F3
G   = B2 D3 G3
```

A progressão é a mesma, mas a sonoridade muda bastante.

E tem uma solução ainda mais interessante para o seu GA

Eu separaria:

qualidade da música

de

diversidade da população.

Por exemplo:

Plain text

```
Fitness =
    0.90 * qualidade_musical
```

```
    + 0.10 * diversidade
```

No começo, talvez:

```
0.80 * qualidade
+ 0.20 * diversidade
```

para evitar que a população convirja cedo demais.

Isso é particularmente importante porque você está usando **elitismo**. O melhor indivíduo sobrevive e seus descendentes começam a dominar a população.

Eu **não aumentaria simplesmente as taxas de mutação**. Isso pode gerar músicas aleatórias demais e destruir as estruturas boas que o GA encontrou.

Eu faria nesta ordem:

1. **Adicionar inversão aos MChords**
2. Permitir mais variedade de `root`
3. Adicionar **mutação específica de MChord**
4. Adicionar **fitness de diversidade**
5. Só então ajustar as taxas de mutação

Isso deve produzir uma diferença muito maior entre as músicas sem transformar o algoritmo em puro aleatório.

Gere um prompt pra eu passar pro Cline pra ele fazer isso com os MChords

Claro. Eu passaria para o Cline um prompt bem específico, deixando claro que **não queremos simplesmente aumentar a aleatoriedade**, mas aumentar a diversidade dos `MChord` preservando a coerência musical.

Prompt para o Cline — aumentar variabilidade dos MChords

Quero modificar meu algoritmo genético de geração musical para aumentar a VARIABILIDADE DOS MCHORDS.

Contexto atual:

- O genótipo possui `MChord1` até `MChord8`.
- `MChord1..MChord4` representam a base da música.
- `MChord5..MChord8` representam o refrão.
- Atualmente as músicas geradas estão ficando muito parecidas entre si.
- Os acordes também possuem aparência/sonoridade muito semelhante.
- NÃO quero simplesmente aumentar aleatoriedade ou taxas de mutação globalmente.
- Quero aumentar a diversidade dos MChords mantendo a coerência harmônica.

Faça as alterações diretamente na implementação existente, preservando a arquitetura atual do projeto e evitando refatorações desnecessárias.

1. Adicionar inversões aos MChords

Modificar a classe `MChord` para possuir uma propriedade:

```
inversion
```

Valores permitidos:

```
0 = root position
1 = primeira inversão
2 = segunda inversão
```

A inversão deve alterar a disposição das notas do acorde, mas NÃO alterar sua identidade harmônica.

Exemplo para C maior:

```
inversion 0:
C E G

inversion 1:
E G C

inversion 2:
G C E
```

Para acordes com mais notas, faça a inversão de forma musicalmente consistente.

A validação do acorde deve continuar considerando a estrutura harmônica original, e não considerar uma inversão válida como um acorde diferente.

2. Gerar inversões aleatoriamente

Ao criar um novo `MChord`, escolher a inversão aleatoriamente entre as opções válidas.

Use pesos razoáveis, dando preferência à posição fundamental, por exemplo:

```
MCHORD_INVERSION_WEIGHTS = (0.50, 0.30, 0.20)
```

Não precisa usar exatamente esses valores se houver uma configuração melhor na arquitetura atual.

3. Criar mutação específica de MChord

Adicionar uma função semelhante às outras mutações existentes:

```
mutate_mchord(...)
```

Ela deve poder alterar principalmente:

- root
- chord_type
- inversion

A mutação deve sempre produzir um `MChord` válido.

IMPORTANTE:
não altere necessariamente todos os campos de uma vez.

A mutação deve escolher uma característica para modificar, para permitir pequenas mudanças evolutivas.

Por exemplo:

```
mutação:
    50% -> inversion
```

```
    30% -> root
    20% -> chord_type
```

ou uma distribuição equivalente adequada ao projeto.

4. Atualizar o crossover

O crossover atual deve continuar funcionando.

O `MChord` herdado do pai deve ser copiado completamente, incluindo:

```
root
chord_type
inversion
```

Não compartilhar objetos entre indivíduos.

5. Atualizar a representação textual

Atualizar `__repr__()` de `MChord` para mostrar a inversão.

Exemplo:

```
MChord(root=60, chord_type="major", inversion=1)
```

ou uma representação equivalente à convenção já utilizada pelo projeto.

6. Atualizar a fitness

A fitness harmônica NÃO deve penalizar uma inversão simplesmente por ser uma inversão.

A validação de:

```
chord_validity
tonal_coherence
functional_progression
```

deve continuar funcionando normalmente.

Porém, o `voice_leading_fitness` deve passar a considerar as notas reais/distribuição do acorde, de forma que inversões que produzam transições suaves entre acordes possam receber uma pontuação maior.

Por exemplo:

```
C major
C E G

F major
C F A
```

deve poder receber uma boa avaliação de voice leading por possuir nota comum e pequenos movimentos.

Enquanto uma disposição muito distante pode receber uma avaliação menor.

7. Adicionar diversidade entre os MChords da própria música

Além da coerência harmônica, quero evitar que os oito acordes sejam excessivamente semelhantes.

Criar uma métrica que considere, por exemplo:

- roots diferentes
- chord_types diferentes
- inversions diferentes

Não quero obrigar todos os acordes a serem diferentes.

A ideia é apenas evitar situações como:

```
C major
C major
C major
C major
C major
C major
C major
C major
```

receberem uma avaliação tão boa quanto uma progressão harmonicamente coerente e mais variada.

A diversidade deve ser um componente moderado da fitness, sem dominar a coerência musical.

8. Preservar a estrutura base/refrão

Manter explicitamente:

```
MChord1..MChord4 = base
MChord5..MChord8 = refrão
```

A base deve continuar tendo maior estabilidade harmônica.

O refrão pode ter alguma diferença em relação à base.

Não transforme os oito acordes em uma sequência completamente aleatória.

9. Atualizar todas as partes necessárias

Procure no projeto todas as partes que dependem de `MChord` e atualize corretamente:

- `genotype.py`
- `individual.py`
- `mutation.py`
- `crossover.py`
- `fitness.py`
- `constants.py`, caso necessário
- geração do MIDI, caso a ordem/disposição das notas seja utilizada diretamente
- qualquer outro arquivo que dependa da estrutura de `MChord`

Não faça mudanças em funcionalidades que não estejam relacionadas a esse objetivo.

10. Compatibilidade

A implementação deve continuar funcionando com:

```
Individual()
```

e com o fluxo completo:

```
Individual
→ calculate_fitness
→ selection
→ crossover
→ mutation
→ geração MIDI
```

Não remova nenhuma função existente.

Não remova:

```
calculate_fitness
fitness_details
describe
```

nem outras funções existentes.

11. Muito importante

Antes de modificar os arquivos:

1. Examine a implementação atual de `MChord`.
2. Examine como `MChord` é usado em `fitness.py`.
3. Examine como `MChord` é copiado no crossover.
4. Examine como a mutação funciona atualmente.
5. Examine como as notas dos MChords são usadas na geração do MIDI.

Depois faça as alterações de forma consistente com o código existente.

NÃO substitua arquivos inteiros por versões simplificadas.
NÃO remova funcionalidades existentes.
NÃO altere a arquitetura do GA sem necessidade.

Ao terminar:

1. Liste os arquivos modificados.
2. Explique resumidamente o que mudou em cada arquivo.
3. Mostre as novas constantes adicionadas.
4. Mostre como `MChord` ficou representado.

5. Verifique se o código continua compilando/executando sem erros.
6. Se possível, faça um teste criando alguns `Individual()` e mostrando diferentes MChords para verificar se as inversões realmente estão produzindo variedade.

O objetivo final é que duas músicas geradas pelo GA possam ter progressões harmonicamente coerentes, mas apresentem MChords com roots, tipos e principalmente inversões/distribuições de notas diferentes, evitando a convergência para músicas excessivamente parecidas.

Eu acrescentaria **uma observação importante** ao Cline: não quero que ele tente resolver a diversidade apenas com `random.choice`. O objetivo é que a **evolução genética tenha diversidade no espaço de busca**, mas continue sendo guiada pela fitness. Isso evita trocar o problema de "músicas muito parecidas" por "músicas aleatórias".

Tá muito detalhado, gere algo mais compacto em 1 ou 2 paragrafos

Claro — eu usaria algo assim:

Prompt compacto para o Cline

Quero aumentar a variabilidade dos `MChord` no meu algoritmo genético, pois as músicas estão ficando muito parecidas e os acordes possuem sempre uma aparência/sonoridade muito semelhante. Analise a implementação atual de `MChord`, `genotype.py`, `mutation.py`, `crossover.py`, `fitness.py` e a geração do MIDI antes de modificar. Adicione **inversões aos MChords** (`0 = posição fundamental`, `1 = primeira inversão`, `2 = segunda inversão`), garantindo que a inversão altere a disposição das notas sem alterar a identidade harmônica do acorde. Inclua as inversões na inicialização, mutação, crossover, validação, `__repr__` e geração do MIDI quando necessário. A mutação também deve poder alterar `root`, `chord_type` ou `inversion`, sempre produzindo acordes válidos.

Além disso, ajuste a fitness para incentivar **diversidade harmônica moderada**, considerando roots, tipos de acorde e inversões diferentes, sem prejudicar a coerência tonal. O `voice_leading` deve considerar as notas/disposições reais dos acordes para favorecer inversões que produzam transições suaves. Preserve a estrutura atual, especialmente `MChord1..4` como base e `MChord5..8` como refrão, e não remova nenhuma funcionalidade existente. Não aumente simplesmente a aleatoriedade: quero maior diversidade dentro do processo evolutivo, mantendo qualidade musical.

O cline tá muito lento, o copilot estourou a cota, e o projeto tem vários arquivos, me diga como eu poderia usar uma LLM alternativa agora pra continuar editando o projeto

Sim. Como o projeto já está relativamente estruturado e você quer **continuar editando vários arquivos**, eu evitaria depender de uma interface web. O melhor agora é colocar uma LLM local ou outra API diretamente no seu fluxo de desenvolvimento.

Como você já usa **Ollama + Continue** no Ubuntu, eu faria assim:

Opção 1 — Ollama + Continue ⭐

Você já tem essa infraestrutura, então é provavelmente a solução mais rápida.

```
Plain text

VS Code
   │
   ▼
Continue
   │
   ▼
Ollama
   │
   ├── modelo de código
   └── seu projeto inteiro
```

Você pode abrir o projeto inteiro no VS Code e pedir algo como:

> "Analise o projeto e implemente inversões nos MChords. Modifique apenas os arquivos necessários."

A vantagem é que o Continue consegue trabalhar com **vários arquivos do projeto**, em vez de você ficar copiando `genotype.py`, `mutation.py`, `fitness.py`, etc. para o chat.

Como você já utiliza `qwen2.5-coder:7b`, eu tentaria primeiro um modelo de código maior, se sua máquina comportar.

Por exemplo, verificar:

```bash
Bash

ollama list
```

e depois:

```bash
ollama pull qwen2.5-coder:14b
```

ou uma versão maior se houver memória/VRAM suficiente.

Opção 2 — Aider + Ollama ⭐⭐⭐

Para **esse projeto específico**, eu acho o Aider ainda mais interessante.

Ele trabalha diretamente no repositório:

```
Git repository
      │
      ▼
    Aider
      │
      ▼
     LLM
      │
      ├── genotype.py
      ├── mutation.py
      ├── crossover.py
      ├── fitness.py
      └── generate_midi_multi.py
```

E pode usar o Ollama como backend.

Você poderia fazer:

```bash
pip install aider-chat
```

Depois, dentro do projeto:

```bash
```

```
aider --model ollama/qwen2.5-coder:14b
```

E pedir:

```
Plain text

Adicione inversões aos MChords para aumentar a diversidade das músicas.
Analise primeiro os arquivos relacionados a MChord e depois faça as alterações.
Preserve a arquitetura existente.
```

A grande vantagem é que o Aider é **orientado a edição de código**, enquanto um chat normal precisa receber contexto manualmente.

E como você está fazendo mudanças que atravessam:

```
Plain text

genotype.py
        ↓
mutation.py
        ↓
crossover.py
        ↓
fitness.py
        ↓
generate_midi_multi.py
```

isso combina bastante com o problema.

Opção 3 — API de outra LLM

Se sua máquina não tiver VRAM/RAM suficiente para um modelo local bom, pode usar uma API externa e deixar o Aider/Continue cuidar do projeto.

A arquitetura fica:

```
Plain text

VS Code / Aider
       |
       ▼
     API
```

```
        |
        ▼
      LLM
```

Isso é particularmente interessante porque você não precisa colocar o projeto inteiro no prompt manualmente.

O que eu faria no seu lugar

Como você está **no meio do desenvolvimento** e o Cline está lento:

1. Agora: Aider + Ollama.

2. Se o modelo local de 7B/14B não for bom o suficiente: Aider + API de um modelo de código mais forte.

3. Continue: mantenha como alternativa dentro do VS Code.

E principalmente: **faça um commit antes de deixar outra LLM mexer no projeto**:

```bash
git add .
git commit -m "before mchord diversity changes"
```

Assim você pode deixar uma LLM fazer alterações em vários arquivos sem medo. Se ela fizer uma bagunça:

```bash
git diff
```

e, se necessário:

```bash
git reset --hard HEAD
```

volta exatamente ao estado anterior.

Para o seu projeto atual, eu particularmente escolheria **Aider + uma LLM forte**, porque você já está chegando naquele ponto em que editar um arquivo isoladamente não é mais suficiente: uma mudança no `MChord` precisa ser propagada consistentemente por várias partes do GA.

Ollama é muito lento aqui

Sim. Nesse caso eu **não insistiria em LLM local**. Para o seu projeto, eu usaria uma LLM via API + Aider/Continue, porque você ganha velocidade sem precisar mandar os arquivos manualmente.

Minha preferência seria:

1. **Aider + API de uma LLM de código** — melhor para modificar vários arquivos do repositório.
2. **Continue + API** — se você prefere permanecer no VS Code.
3. **Cline + API** — continuar usando Cline, mas trocar o modelo/provedor; o problema pode ser o modelo atual, não o Cline.

O fluxo que eu acho mais adequado para você:

```
Plain text

Seu projeto Git
     │
     ▼
   Aider
     │
     ▼
LLM via API
     │
     ├── genotype.py
     ├── mutation.py
     ├── crossover.py
     ├── fitness.py
     └── generate_midi_multi.py
```

Assim você pode simplesmente dizer:

> "Implemente inversões nos MChords e aumente a diversidade harmônica. Analise os arquivos necessários e faça as alterações."

e a LLM trabalha diretamente no repositório.

Se você quiser gastar pouco, dá para procurar uma API com modelo de código barato/grátis e conectar ao Aider. **Se quiser qualidade**, eu escolheria um modelo de código mais forte via API e deixaria o Aider cuidar das edições.

Se quiser, posso te passar **uma configuração concreta do Aider para usar uma API alternativa ao Copilot**, incluindo instalação, configuração da chave e comando para abrir seu projeto.